

Tema 1 (1)

Tema 2

Tema 3 (1)

Tema 5

Tema 6

SRS a validar

Tema 7

Tema 8

Epílogo



Universidad de Oviedo

Tema 1

Fundamentos de la Ingeniería de Requisitos

1

Jorge Álvarez Fidalgo

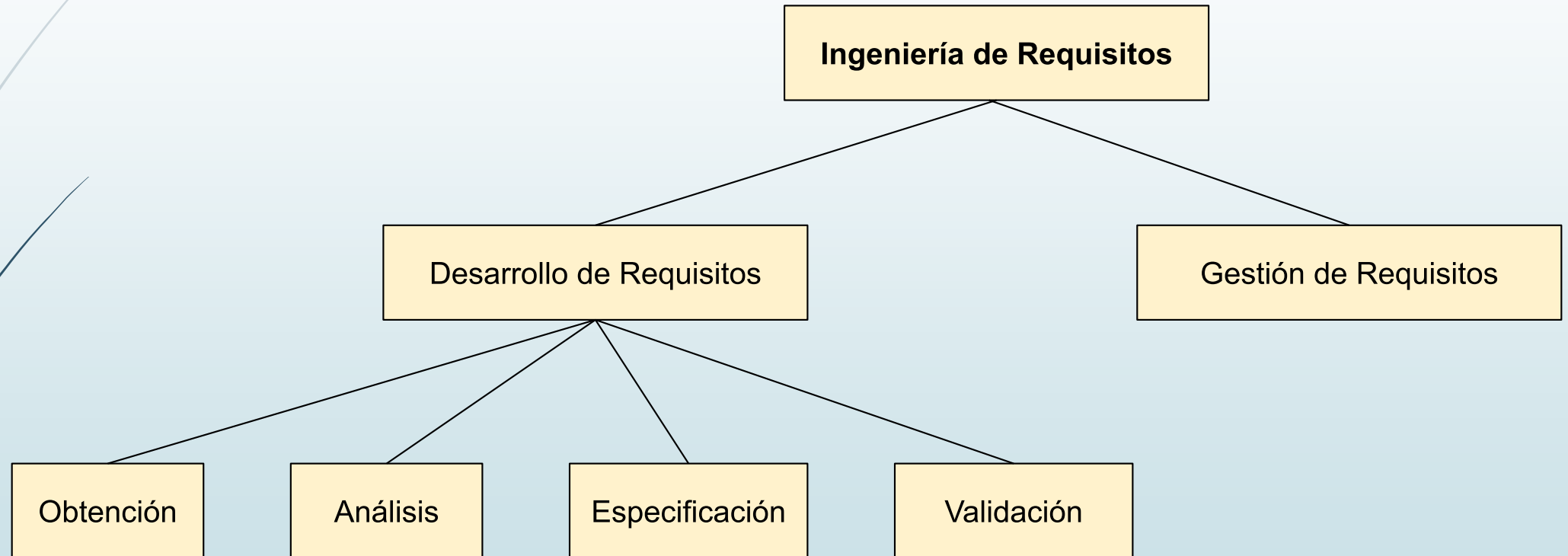
Introducción

- Los **requisitos** son una **especificación** de lo que se debería **implementar**: descripciones de cómo debería **comportarse** el sistema, o de una **propiedad** o atributo del sistema. También podrían ser una **restricción** en el proceso de desarrollo del sistema. [Sommerville y Sawyer]
- Abarcan tanto el comportamiento **externo** del sistema (*usuario*) como las características **internas** (*desarrollador*).

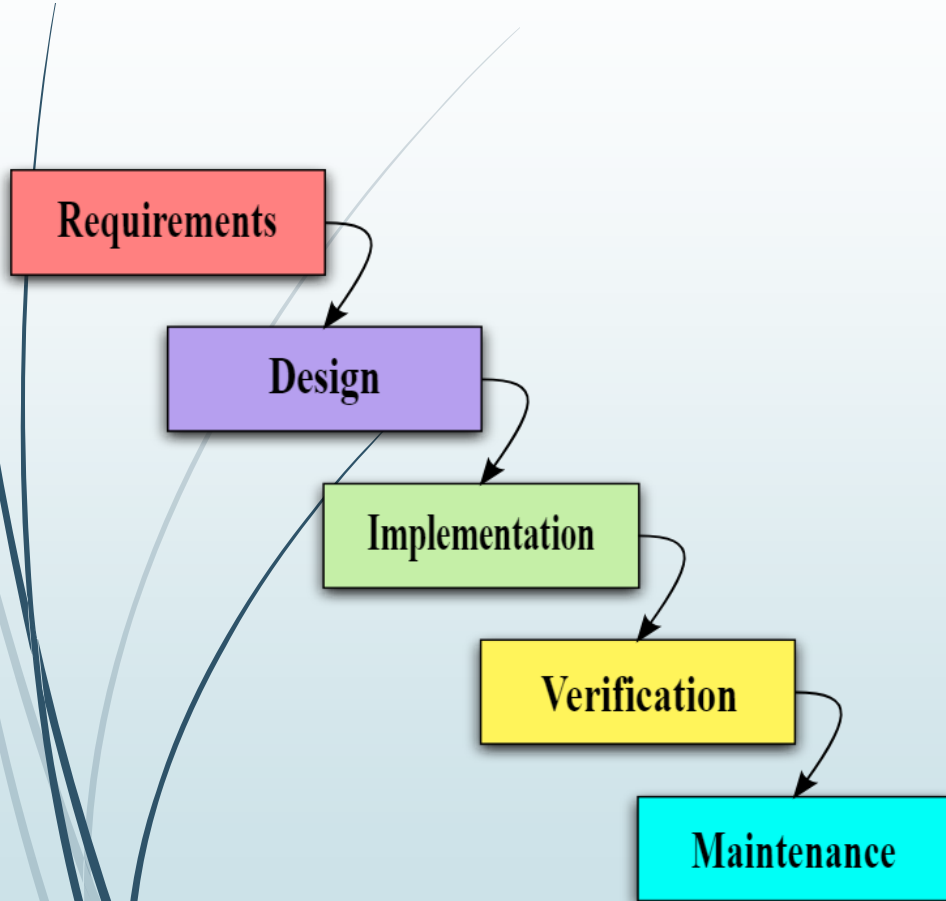
¿Qué es la Ingeniería de Requisitos?

- **Proceso** de **descubrir, analizar, documentar y verificar** los servicios proporcionados por el sistema y sus restricciones operativas [Sommerville].
- **Proceso** del estudio de las **necesidades** de los usuarios para llegar a una **definición** de los requisitos del sistema, de hardware o de software [IEEE 610]
- Conjunto de **procesos**, tareas y técnicas que permiten la **definición** y **gestión** de los requisitos de un producto de modo sistemático. [Pressman]

¿Qué es la Ingeniería de Requisitos?



¿Por qué es importante?



- Errores en la IR implican correcciones en fases posteriores.
- El coste de solventar estos errores puede ser: *[Haskins 2014]*
 - 3-8x en Diseño.
 - 7-16x en Implementación.
 - 21-78x en Pruebas.
 - 30-1500x en Mantenimiento.

Problemas típicos

- Los **requisitos no reflejan las necesidades** de los usuarios.

> Necesidades

- Bien porque los desarrolladores los ignoran...
- ... o los usuarios no se involucran suficientemente.

- **Requisitos excesivamente cambiantes.**

Van Cambiando

- **Requisitos ambiguos.** > No se sabe muy bien que hacen

- El **cliente** pide características de **escaso** valor.

> Inútiles

- El **desarrollador** añade **funcionalidades no solicitadas.**

> No me lo han pedido

- **Stakeholders** no tenidos en cuenta.

> Stakeholders Olvidados

- Solucionar estos problemas afecta a la **planificación.**

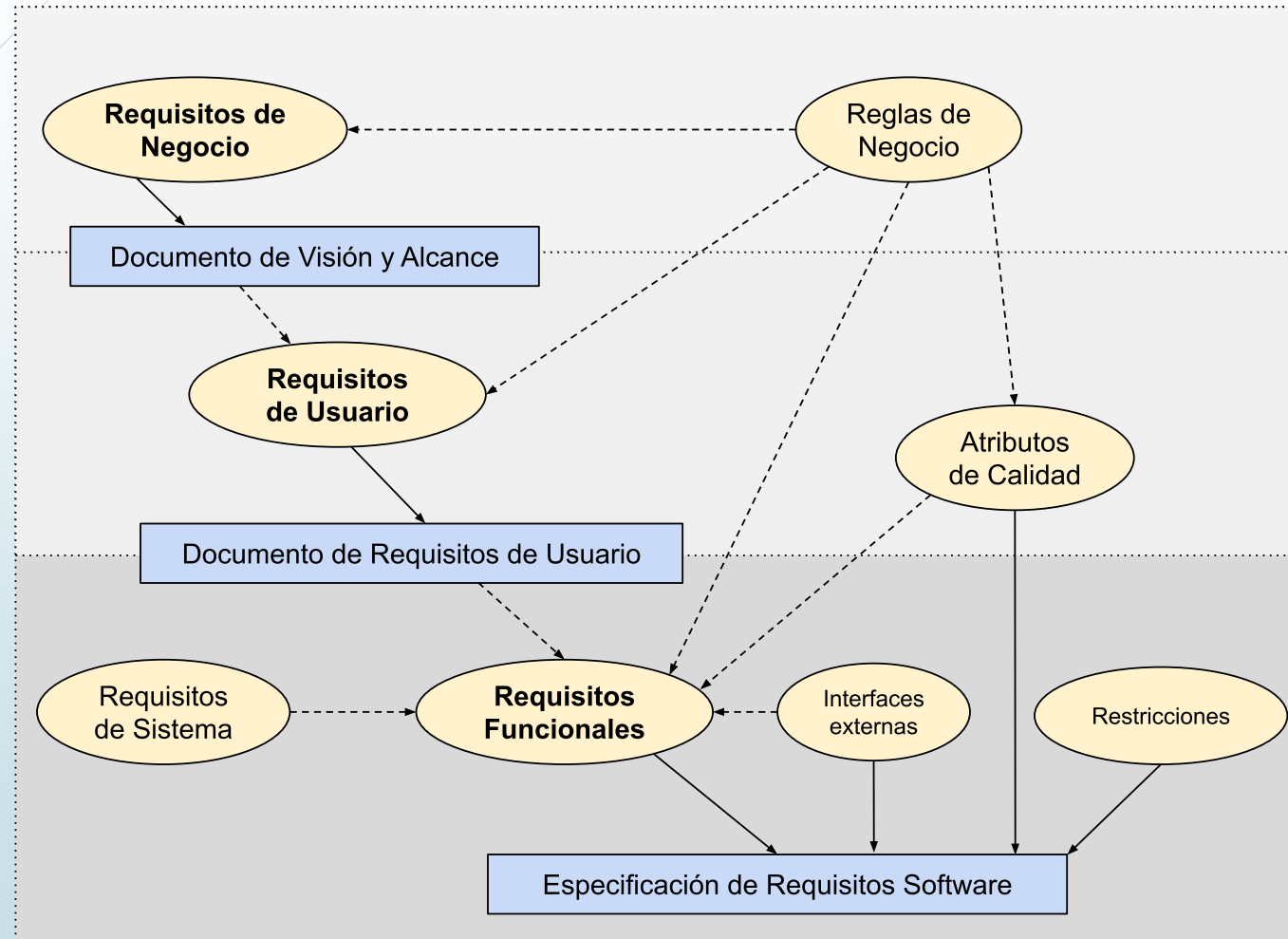
Niveles y tipos de requisitos

- **Requisitos de negocio.** Describen los **objetivos de negocio** que el cliente pretende conseguir.
- **Requisitos de usuario.** Objetivos o **tareas que** los **usuarios deben** ser capaces de **realizar** con el sistema. *Casos de uso, historias de usuario...*
- **Requisitos funcionales.** Describen **qué** deben **implementar** los **desarrolladores** para satisfacer los requisitos de usuario.
 - Se documentan **en una Especificación de Requisitos Software (SRS).**

Niveles y tipos de requisitos

- **Requisitos de sistema.** Describen un producto compuesto de varios subsistemas: sus roles y la compatibilidad entre ellos.
- **Reglas de negocio.** Legislación, políticas y estándares.
- **Atributos de calidad.** Describen características y propiedades del sistema en diversas dimensiones de importancia.
 - *Rendimiento, seguridad, disponibilidad...*
- **Interfaces externas.** Conexiones a otros sistemas y componentes.
- **Restricciones.** Sobre las opciones del desarrollador.

Niveles y tipos de requisitos



Requisitos Funcionales y No Funcionales

- Tradicionalmente, los requisitos se han clasificado de forma general como **Funcionales** y **No Funcionales**.
- Los Requisitos **Funcionales** describen el comportamiento del sistema bajo unas condiciones determinadas.
- Sin embargo, la definición de Requisitos **No Funcionales** es poco clara.
 - El término expresa lo que **no** son, pero no **qué** son.

Requisitos Funcionales y No Funcionales

- Los **RNF** suelen asociarse con los atributos de calidad.
 - ...pero no se limita a ellos.
- Como vimos en #8, hay más tipos de requisitos "no funcionales".
 - Restricciones, interfaces externas, reglas de negocio...
- Es posible que se **deriven** otros Requisitos Funcionales de ellos.

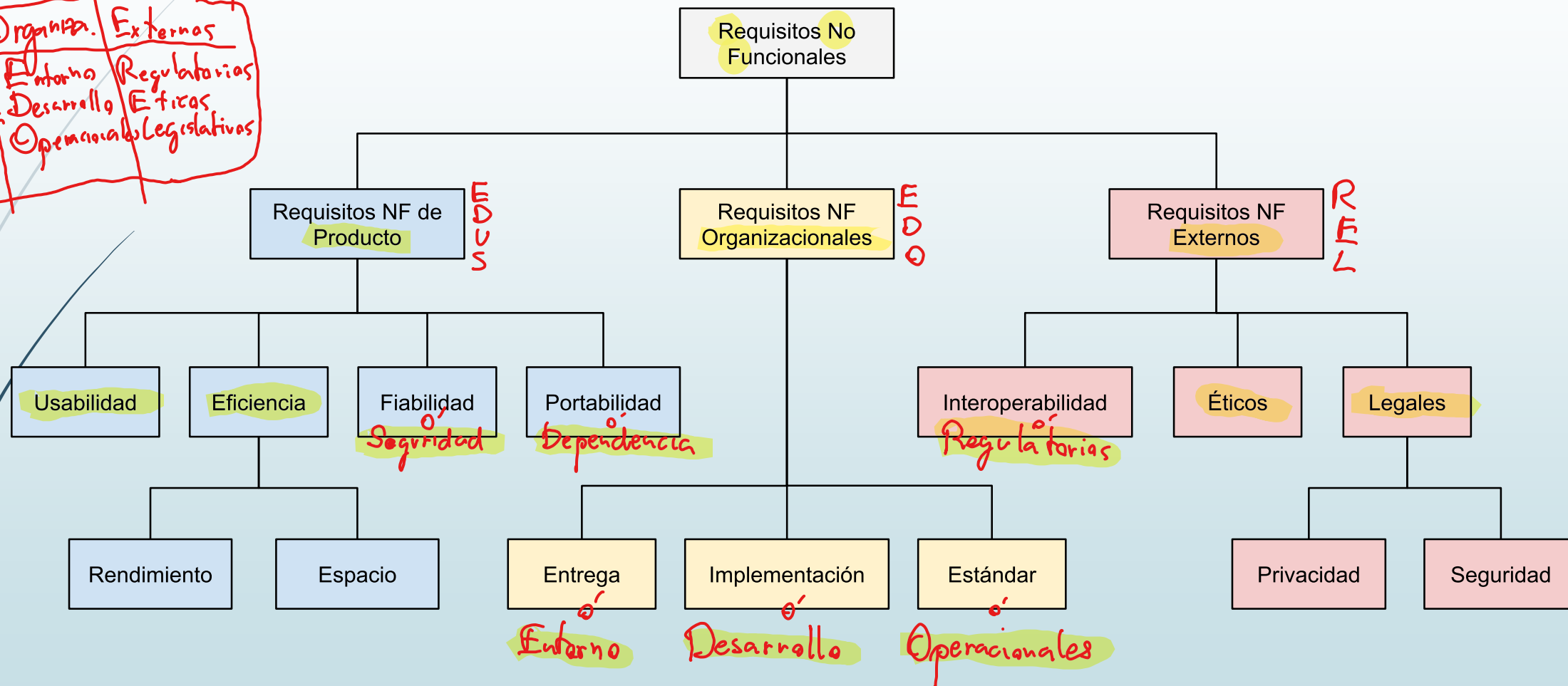
Requisitos según IEEE 830-1998

- El estándar **IEEE 830-1998** define el formato de un **SRS**.
- Clasifica los "Requisitos Específicos" según lo siguiente:
 1. Interfaces Externas.
 2. Funciones.
 3. Requisitos de Rendimiento.
 4. Restricciones de Diseño.
 5. Atributos del Sistema.
 1. Fiabilidad, disponibilidad, seguridad, mantenimiento, portabilidad.
 6. Otros Requisitos.

RNF según Kotonya y Sommerville

RNF (sumario)

Producto	Organiza.	Externas
Eficiencia	Entorno	Regulatorias
Dependencia	Desarrollo	Éticas
Usabilidad	Operacionales	Legislativas
Seguridad		



RNF según Kotonya y Sommerville



Usabilidad. La **tasa de error** de usuarios al introducir sus datos de pago debe ser **inferior** al 10%.

Fiabilidad. El servicio X del sistema debe tener un **tiempo** medio **entre fallos** de 28 horas.

Rendimiento. La página de inicio debe tener un **tiempo de respuesta** de 1 segundo o menos.

Espacio. La **BD** albergará un **máximo** de 50000 entradas.

Portabilidad. El sistema será **compatible** con Windows 10 y Windows 11.

Entrega. Los informes de **pruebas** deben entregarse **semanalmente**.

Implementación. El sistema **debe emplear** SOLID para gestionar los perfiles de usuario.

Estándar. El sitio web debe **cumplir** la **norma CSS** de nivel 3.

Interoperabilidad. El sistema **se comunicará** con el servicio externo X.

Privacidad. El sistema mostrará a los usuarios **información anonimizada** de los clientes.

Seguridad. La **pasarela** de procesamiento **de pagos** debe **cumplir** la **norma** PCI DSS.

Métricas RNF según Sommerville

Propiedad	Medida
Rapidez	Transacciones procesadas por segundo <i>Tx/seg</i> Tiempo de respuesta a usuarios/eventos <i>t.evento</i> Tiempo de refresco de la pantalla <i>t.refresco</i>
Tamaño	MBytes <i>MB</i>
Facilidad de uso	Tiempo de formación <i>→ "4 horas adaptación"</i>
Fiabilidad	Tiempo medio entre fallos <i>t/fallo</i> Probabilidad de no disponibilidad <i>% no disp</i> Índice de ocurrencia de fallos <i>"fallos > 0'05%"</i> Disponibilidad <i>"99'99% tiempo"</i>
Robustez	Tiempo de reinicio tras el fallo <i>"15 segundos reinicio"</i> % de eventos que causan fallos <i>fallos < 0'05%</i> Probabilidad de corrupción de datos tras el fallo <i>"< 0'005%"</i>
Portabilidad	Nº de sistemas objetivo

Otras clasificaciones de requisitos

- **Métrica V3:** *[Análisis del Sistema de Información 2: Establecimiento de Requisitos]*
 - Funcionales, Rendimiento, Seguridad, Implantación, Disponibilidad.
- **Pohl:** *[Requirements Engineering: An Overview (1996)]*
 - Requisitos Funcionales Abstractos (RF, Interfaces Ext.)
 - Descripciones de Datos
 - Requisitos No Funcionales Abstractos.
- **FURPS+:** *[Grady y Caswell]*
 - Funcionalidad, Usabilidad, Fiabilidad_(Reliability), Rendimiento_(Performance), Soportabilidad y otros.

Requisitos de producto y de proyecto

- Los requisitos que describen propiedades del sistema a desarrollar se denominan **requisitos de producto**.
- Los requisitos que no formen parte del sistema a desarrollar, pero sean necesarios para completar el proyecto se denominan **requisitos de proyecto**. Ejemplos:
 - Recursos físicos necesarios para el desarrollo.
 - Documentación para el usuario.
 - Pruebas y despliegue en el entorno operativo.
 - Migración del sistema previo.
 - Certificación del producto.
 - Adquisición de licencias software.
 - Protección legal de la propiedad intelectual del software desarrollado.

Propiedades de los requisitos

► Propiedades **comunes**:

- Identificador.

- Descripción.

- Versión.

- Fecha.

- Autor.

- Dependencias.

- Prioridad.

- Estado.

- Es posible **añadir** otros según las características del proyecto.

Bibliografía recomendada

- Wiegers, K. E., & Beatty, J. (2013). *Software requirements*. Microsoft Press.



Universidad de Oviedo

Tema 2

Proceso de la Ingeniería de Requisitos

1

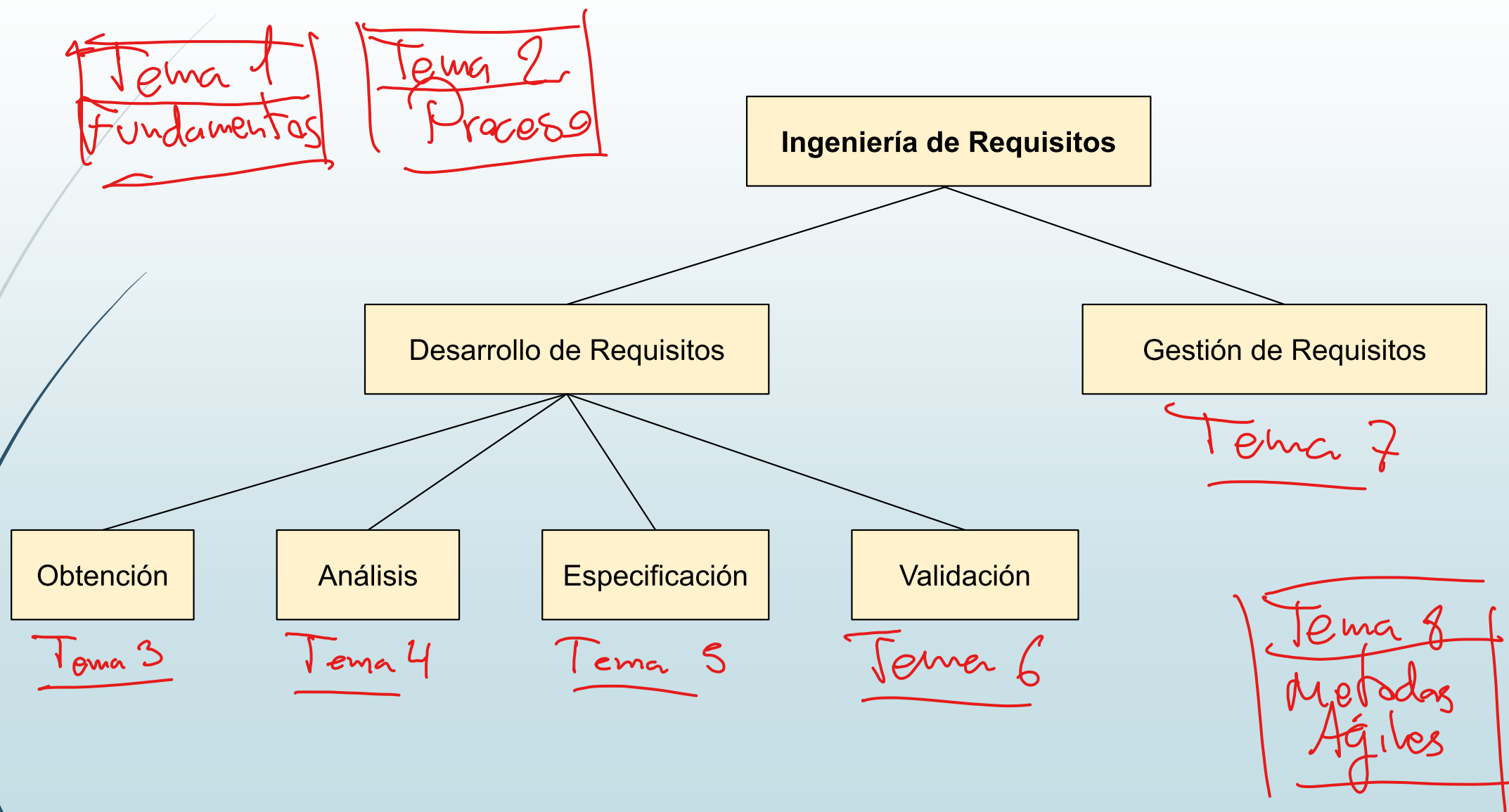
Jorge Álvarez Fidalgo

Introducción

Proceso IR: actividades para:
(Obtener, Validar, Mantener) → SRS

- **Proceso de IR:** conjunto estructurado de actividades que se siguen para obtener, validar y mantener un documento de requisitos del sistema. [Kotonya y Sommerville]
- Los procesos de IR **no** están **estandarizados**.
- **Objetivo:** elaborar y mantener un **SRS**.

Introducción



Obtención

Abarca las actividades relativas al **descubrimiento** de requisitos:

- **Identificar** los tipos de **usuarios** y otros **stakeholders**.
- **Entender** las **tareas** y **objetivos** de los **usuarios** y los objetivos de negocio relacionados.
- Comunicación con los usuarios tipo para **obtener sus necesidades** y **expectativas**.
- **Comprender el contexto** del producto.

Análisis

Abarca aquellas actividades que nos permiten conseguir un **entendimiento más detallado** y preciso de los requisitos:

- **Analizar** la información obtenida de los **usuarios**.
- **Detallar** adecuadamente **requisitos** de alto nivel.
- Estudio de los **atributos de calidad**.
- **Derivar** **requisitos** funcionales a partir de otros requisitos.
- Definir **prioridades**.
- Identificar requisitos **innecesarios** o **incompletos**.

Especificación

Consiste en **representar** y almacenar el **conocimiento** de requisitos de forma clara y organizada:

- Producir **requisitos escritos** de forma **completa**, **precisa** y **verificable**.
- Generar **diagramas** adecuados para el uso y comprensión por parte de los interesados.

Validación

Consiste en **comprobar** que los **requisitos** obtenidos son **adecuados** para construir un producto que satisfaga los requisitos de negocio:

- **Revisar** la documentación **en busca de errores:**
 - Los requisitos cumplen los atributos de calidad deseables.
 - Los requisitos reflejan las necesidades del cliente y los usuarios.

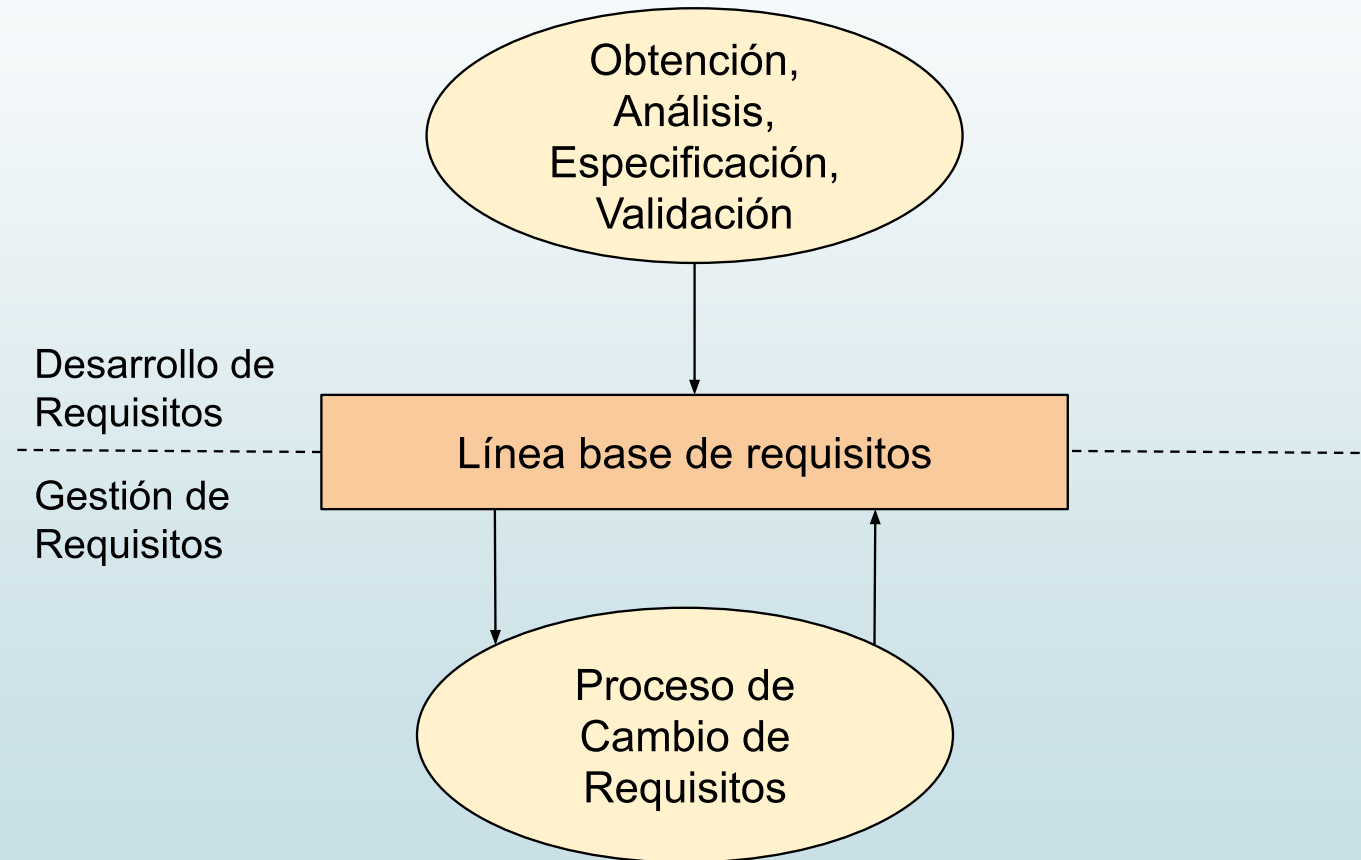
Los requisitos perfectos son una quimera. El objetivo es reunir un conocimiento suficiente de los requisitos que nos permita elaborar el producto con un nivel aceptable de riesgo.

Gestión

Consiste en anticipar y **gestionar** los cambios durante el ciclo de vida del proyecto para minimizar su impacto negativo.

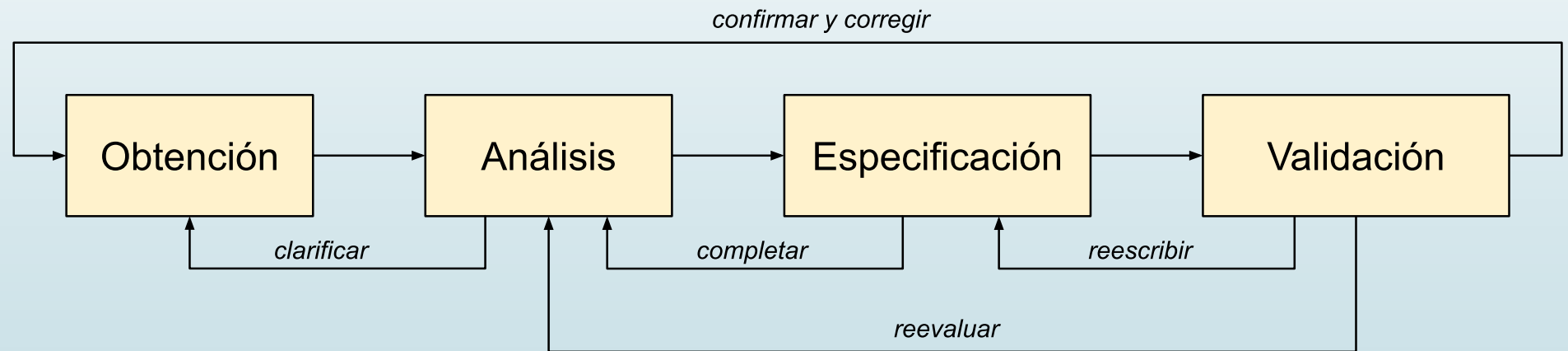
- **Definir la línea base** de requisitos.
 - Conjunto de requisitos acordados y revisados en un momento dado, de cara a un *release* o una iteración del desarrollo.
- **Evaluar cambios** propuestos.
- **Incorporar cambios** aprobados de forma controlada.
- **Trazar requisitos** individuales con los diseños, código y prueba derivados.
- Realizar **seguimiento** del estado de los requisitos y el historial de cambios a lo largo del proyecto.

Desarrollo y gestión de requisitos



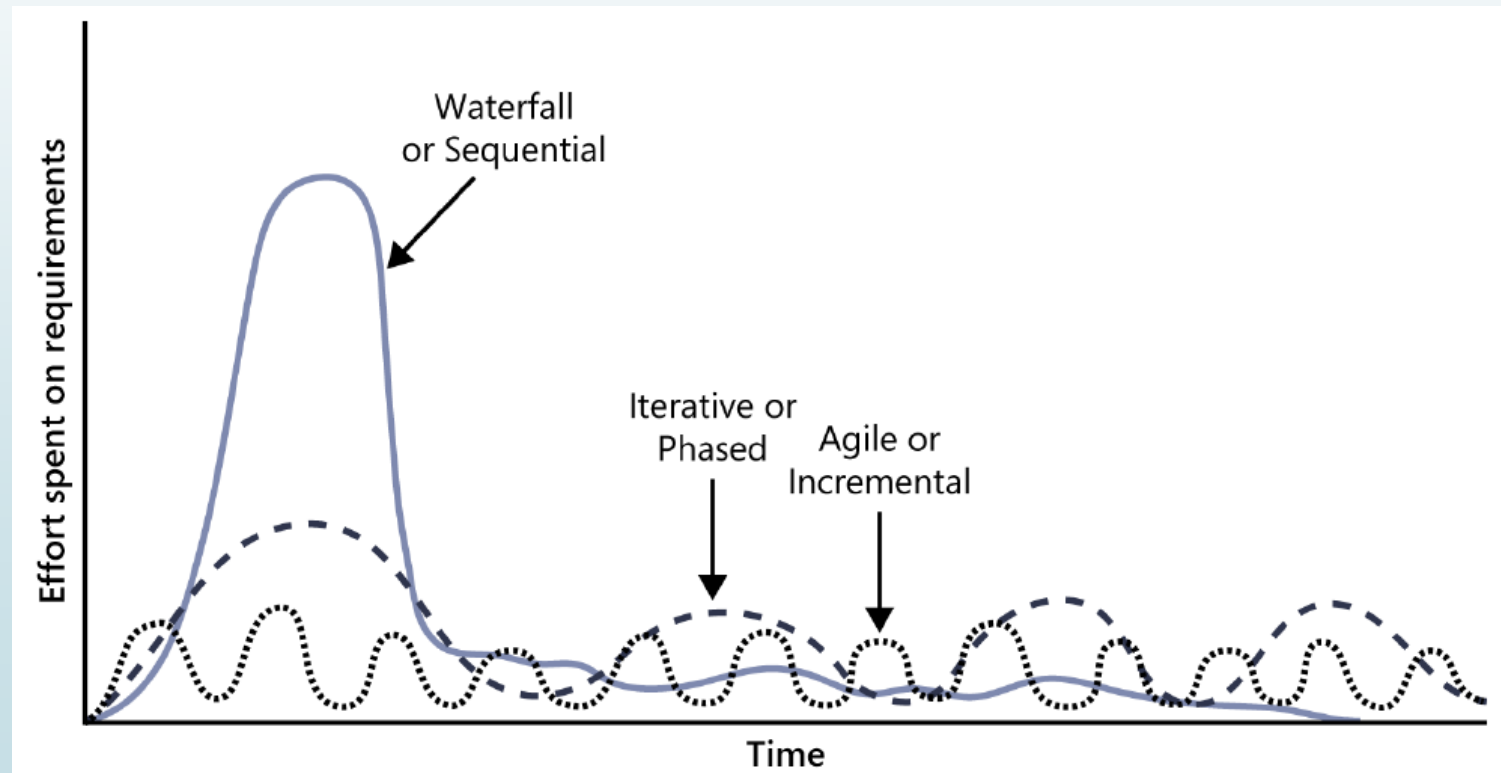
Flujo del proceso

- El proceso de IR **no** suele ser **linear**, sino **iterativo**.



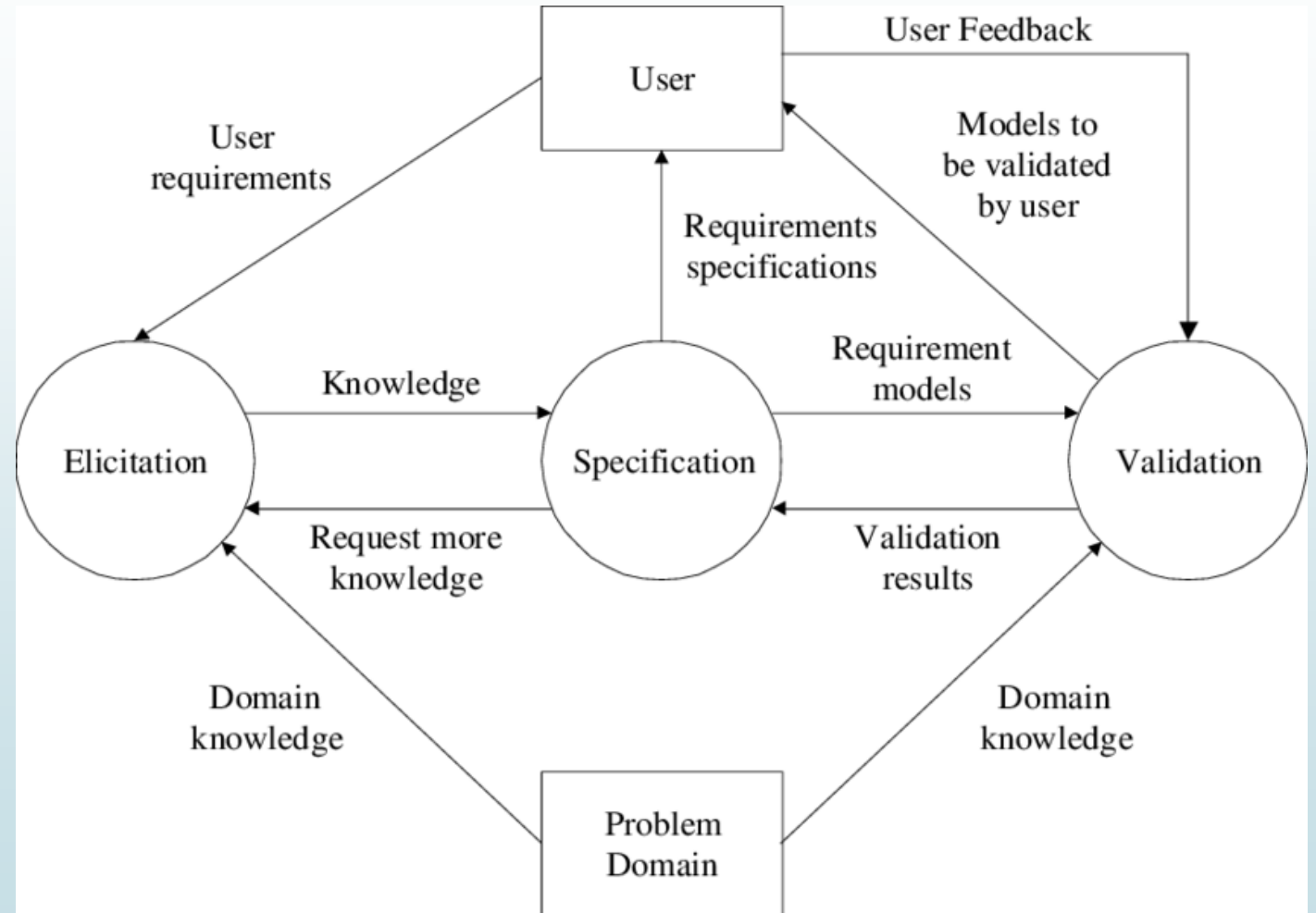
Flujo del proceso

- El esfuerzo empleado a lo largo del tiempo depende de la **metodología**.



Otros procesos de IR

► Loucopoulos y Karakostas (1995)





Otros procesos de IR

► Pohl (1996)

- Cliente
- Usuario
- Jefe Proyecto
- Ing. Reg.
- Ing. SW
- Prog
- Experto Market

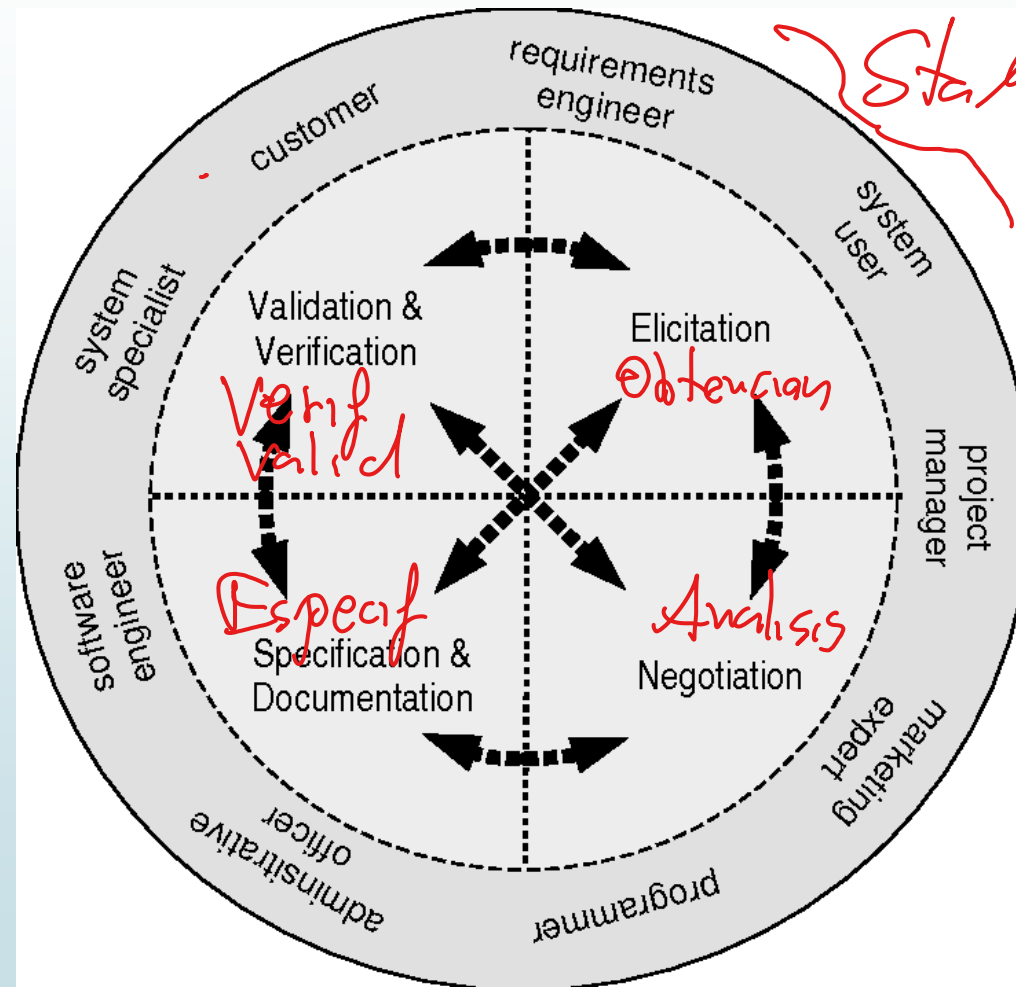


Fig. 3 Four tasks of the RE process.

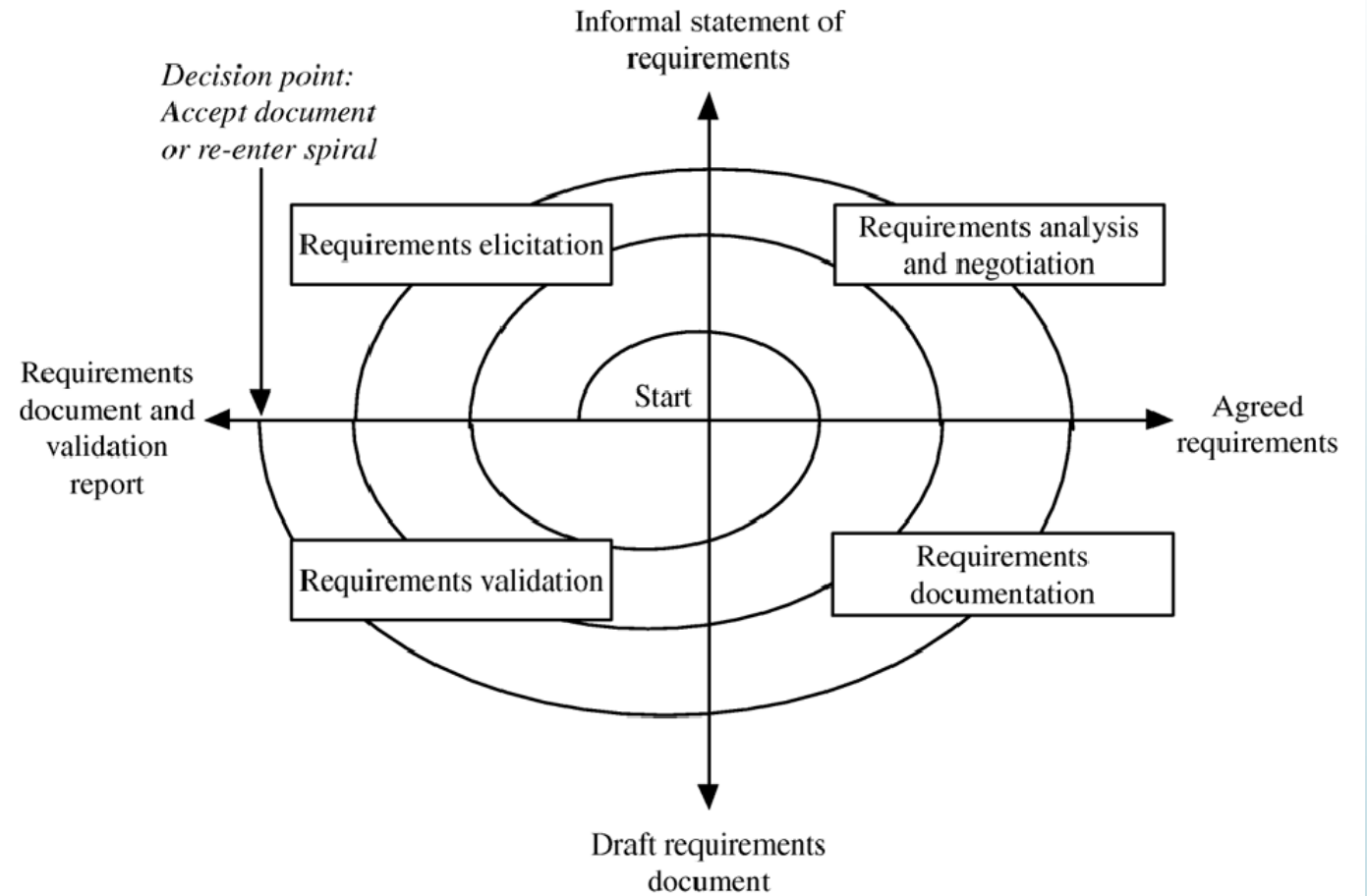
[Pohl, K. Requirements Engineering: An Overview.]



Otros procesos de IR

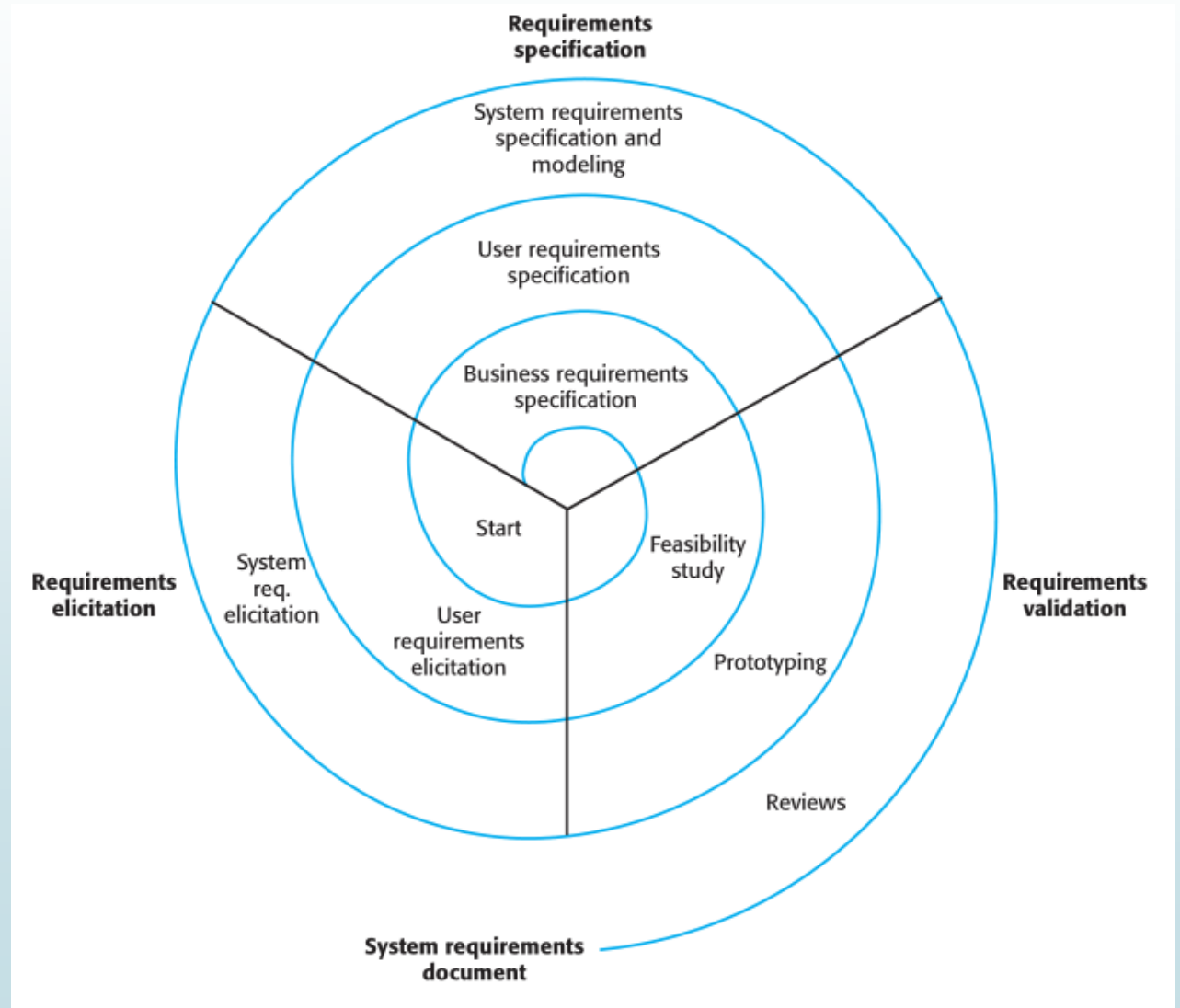
► Kotonya y

Sommerville (1997)



Otros procesos de IR

- Sommerville (2016)





Universidad de Oviedo

Tema 3

Técnicas de obtención de requisitos

1

Jorge Álvarez Fidalgo

Introducción

La **obtención** abarca las **actividades** relativas al **descubrimiento** de requisitos:

- **Identificar** los tipos de **usuarios** y otros **stakeholders**.
- **Entender las tareas y objetivos** de los **usuarios** y los objetivos de negocio relacionados.
- Comunicación con los usuarios tipo para **obtener sus necesidades** y **expectativas**.
- **Comprender el contexto** del producto.

Tipos de stakeholders

Podemos clasificar los **stakeholders** en 3 niveles:

- **Externos** a la organización desarrolladora.

- *Usuarios, cliente, contratista, equipo legal, agencias gub., consultores, proveedores...*

- **Organización** desarrolladora.

- *Director de desarrollo, márketing; equipos de ventas, producción, soporte, infraestructuras...*

- **Equipo** del proyecto.

- *Jefe de proyecto, analistas, director de producto, arquitectos, QA, diseñadores, desarrolladores, administradores de BD, dueño del producto (PO).*

Tipos de usuarios

Podemos clasificar de forma general a los **usuarios** en:

- **Usuarios deseados.** Aquellos mejor alineados con los objetivos de negocio.
- **Usuarios no deseados.** Aquellos que no deberían usar el producto por cuestiones legales o de seguridad.
 - Medidas a tomar: seguridad de accesos, roles de usuario, logs, antimalware.
- **Usuarios ignorados.** Pueden usar el producto, pero no son el objetivo.
- **Usuarios indirectos.** No usan el producto, pero acceden a sus datos o servicios a través de informes u otros medios.

(Sub)Tipos de usuarios

Debemos realizar una identificación más precisa de los tipos de usuarios.

- ¿Quién espera el **cliente** que use el sistema?
- Identificar todos los tipos de usuario posibles.
 - Acorde a las **tareas** que realizan.
 - A partir de **casos de uso** e **historias de usuario**.
 - A partir de **diagramas organizativos**.
 - A partir de los ya identificados.
 - *Nota: los usuarios pueden ser no humanos.*
- **Condensar grupos** de usuarios con **necesidades similares**.

[Gottesdiener 2002]

Técnicas de obtención de requisitos

Las técnicas que estudiaremos son las siguientes:

- Entrevistas.
- Talleres.
- Grupos focales.
- Observaciones.
- Cuestionarios.
- Análisis de interfaces.
- Análisis de documentación.

Entrevistas

- Técnica más **habitual**.
- **Individuales** o en pequeños **grupos**.
- Particularmente **útiles en dominios desconocidos**.
- Suele haber mayor **confianza** entre interlocutores.
- Pueden ser:
 - **Estructuradas**.
 - **Semi-estructuradas**.
 - **No estructuradas**.

Entrevistas estructuradas

- Siguen **preguntas preparadas de antemano** para obtener una **información concreta**.
- **Ventajas:**
 - Se obtiene **información precisa** y organizada acorde a unos temas.
 - **Fáciles de realizar** y repetir.
 - **Fiables:** suelen aportar resultados de interés.
- **Inconvenientes:**
 - **Evaluación limitada** de los resultados obtenidos.
 - Información precisa pero no particularmente detallada.
 - **Escasa flexibilidad.**
 - Requieren un **tiempo significativo**.

Entrevistas semi-estructuradas

- Se sigue una **estructura**, pero con **libertad** para que el entrevistador se desvíe si es necesario.
- **Ventajas:**
 - El entrevistador **puede expandir una pregunta** o explorar nuevas ideas.
 - **Flexibles pero guiadas.**
 - El formato de la pregunta depende del entrevistador.
 - Se consigue **información cualitativa**.
- **Inconvenientes:**
 - **Comparar respuestas a una misma pregunta puede ser complejo** si se desarrollan distintamente.

Entrevistas no estructuradas

- **Conversación improvisada** con el objeto de reunir información.
- **Ventajas:**
 - Permite construir una relación amistosa con el entrevistado.
 - Permite aclarar dudas y reformular preguntas.
 - Extremadamente flexible.
- **Inconvenientes:**
 - Fiabilidad cuestionable: irregulares.
 - Ceñirse al objetivo puede ser difícil.
 - El entrevistador debe tener en mente en todo momento el propósito de la entrevista, así como las habilidades e intereses del entrevistado.

Entrevistas: tipos de preguntas

- **Abiertas.** El entrevistado tiene **infinitas respuestas** posibles.
 - Proporcionan muchos detalles (relevantes... o no).
 - Mayor espontaneidad y comodidad del entrevistado.
 - Pueden arrojar luz sobre cuestiones inéditas.
 - Su abuso puede percibirse como ignorancia y/o inseguridad por parte del entrevistador.
 - *¿Qué opina acerca del sistema actual de la empresa?*
- **Cerradas.** El entrevistado tiene un nº de **respuestas finito**.
 - Rápidas y directas.
 - Facilitan la comparación entre entrevistados.
 - Resultados precisos, pero de escaso detalle.
 - No involucran al entrevistado.
 - *¿Cuál es el nº medio de transacciones que tienen lugar al día?*

Entrevistas: consejos

- Construir una **relación** con el entrevistado.
 - Detalles sutiles como replicar el lenguaje corporal o contacto visual frecuente son de ayuda.
- Mantenerse **fieles** al tema.
- Tener algunas **preguntas preparadas** de antemano.
- **Sugerir ideas** a los interlocutores.
 - Puede que tengan interés en posibilidades que desconocen o que creían inviables.
- **Escuchar** activamente.
 - Mostrar interés en las palabras del interlocutor y retroalimentarle de un modo neutral (p.e. parafrasear).

Talleres

- **Reunión estructurada** en la que un **grupo** selecto de **stakeholders** y expertos definen, crean, refinan y cierran entregables que representan requisitos. *[Gottesdiener 2002]*
- Permiten **obtener requisitos** de **múltiples** stakeholders a la vez.
 - Útiles para resolver desacuerdos entre stakeholders.
- También llamados Joint Application Design (JAD).

Talleres

- Poseen un rol especial denominado **facilitador**.
- **Planifica** el taller, escogiendo a sus integrantes y guiando el proceso.
- Esta labor suele recaer en el **analista de negocio**.
- Es complicado facilitar, anotar los resultados y participar en la discusión para una sola persona.
 - Se suele emplear un **escriba** que se encargue de recoger los resultados.
 - Si el analista es inexperto, es recomendable que sólo participe como stakeholder y se emplee un facilitador externo.

Talleres: consejos

- Definir reglas básicas.
- Definir roles. Escritura, gestión del tiempo y del alcance, moderadores...
- Elaborar planificación.
- Ceñirse al alcance del proyecto. Cuidar el nivel de abstracción correspondiente a la fase.
- Capturar cuestiones de relevancia. Atributos de calidad, restricciones u otros RNF.
- Asignar un tiempo a cada tema.
- Grupos pequeños son preferibles.
 - 6+ participantes puede resultar inmanejable.
 - Ceñirse a figuras clave: representante del producto, analista, desarrollador, usuarios clave, expertos en la materia...
- Procurar la participación de todos los integrantes.

Grupos focales

- **Grupo de usuarios representativos** que aporta información e *ideas* sobre los requisitos de un producto.
- Permiten conocer las necesidades, preferencias y expectativas de los usuarios.
- Variantes:
 - Múltiples grupos focales de usuarios del mismo tipo.
 - Grupo focal con usuarios de todos los tipos relevantes.
- Requieren **facilitación**.
 - La mayoría de los consejos aplicables a los talleres lo son también aquí.

Observaciones

- En ocasiones, las descripciones de los usuarios **no** son **suficientes** para comprender las tareas que realizan.
- La **observación** de los usuarios en su **entorno de trabajo** puede aportar información útil.
- Consumen mucho **tiempo**...
 - ... por lo que deberían emplearse en tareas importantes o de alto riesgo.
- Permiten identificar problemas a resolver, temas a profundizar y corroborar la información obtenida previamente.
- Pueden incluir **interacción** con el usuario.
- A tener en cuenta: efecto Hawthorne.

Cuestionarios

- Permiten **encuestar** a **grandes cantidades** de usuarios sin interacción directa.
- Equivalentes a una entrevista estructurada escrita.
- Pueden ser usados como entradas para técnicas más detalladas.

Cuestionarios: consejos

- Usar preguntas abiertas o cerradas según los **objetivos**.
- Las opciones de respuesta han de ser **exclusivas** y **exhaustivas**.
- **Escalas** consistentes.
- **Probar** el cuestionario antes de su uso.
- **Nº limitado** de preguntas.

Análisis de interfaces de sistema

- Estudio de los **sistemas externos** a los que deba **conectarse** nuestro producto.
- El objetivo es **identificar funcionalidades** en otros sistemas que impliquen **requisitos** en el sistema a construir.
 - Datos que se envían/reciben desde el sistema, formato y validación de aquellos...
- Los diagramas de **contexto** son un buen punto de partida.

Análisis de interfaces de usuario

- Estudio de **sistemas preexistentes** (similares en su defecto) con el objeto de descubrir requisitos.
- Permite conocer las **tareas habituales** de los usuarios.
- *Cuidado: el sistema a construir no necesariamente ha de incluir toda la funcionalidad presente en otros sistemas relevantes.*

Análisis de documentación

- Estudio de la **documentación** de sistemas preexistentes o similares.
 - Especificaciones de requisitos.
 - Procesos de negocio.
 - Manuales de usuario.
 - Reseñas comparativas.
 - Incidencias y retroalimentación de los usuarios.
- Pueden contener funcionalidad relevante (u obsoleta), restricciones de negocio, problemas frecuentes de los usuarios...

¿Cuándo emplear las técnicas?

	Entrevistas	Talleres	G. focales	Observaciones	Cuestionarios	An. de I. de Sistema	An. de I. de Usuario	An. de Documentación
Software de uso masivo	X		X		X			
Software corporativo interno	X	X	X	X		X		X
Sustitución de sistema existente	X	X		X		X	X	X
Mejora de sistema existente	X	X				X	X	X
Nueva aplicación	X	X	?			X		
Sistemas integrados	X	X				X		X
Stakeholders internacionales	X	X			X			

Recomendaciones: preguntas

- Evitar preguntas demasiado generalistas.
 - "*¿Qué necesita hacer?*" es preferible a "*¿Qué quieres?*"
- Cubrir todas las entradas y situaciones posibles.
 - *¿Qué podría pasar si...? ¿Podrías necesitar...? ¿Podría alguien intentar...?*
- Si existe un sistema previo, sondear al respecto.
 - *¿Cuáles son las tres cosas que más le molestan del sistema actual?*

Recomendaciones: preguntas

- Se pueden encadenar *¿Por qué?* a una pregunta para aclarar detalles.
- Al final de la sesión, se puede preguntar:
 - *¿Hay algo más que esperabas que te preguntase?*
- Bibliografía con sugerencias de preguntas:
 - *The Quest for Software Requirements*, Roxanne E. Miller (2009).
 - *More about Software Requirements*, Karl E. Wiegars (2006).

Recomendaciones: uso de modelos

- Las técnicas de obtención se pueden reforzar con el uso de modelos de hombre de paja (***straw man models***).
 - Borrador de un modelo planteado para generar **discusión** e inspirar mejores modelos.
 - *Ejemplos: casos de uso, flujos de proceso...*
- Se usan como **punto de inicio** para explorar la temática y obtener ideas.
 - Es más sencillo corregir un borrador que hacer uno desde cero.
- Fuentes: otras técnicas de obtención como análisis de interfaces o una entrevista con un experto en la materia.

Recomendaciones: obtención

- Reconsiderar el **alcance** del proyecto si es necesario.
- **Trato equitativo** a los stakeholders.
- Los requisitos no son totalmente incompatibles con el **diseño**.
 - Prototipado rápido.
- Especial cuidado con los requisitos **implícitos**.
 - *¿Qué estamos asumiendo?*

Final de la obtención

- No existe una métrica para marcar el final.
- Algunas referencias:
 - No se identifican más casos de uso e historias de usuario.
 - Los escenarios propuestos no se asocian a nuevos RF.
 - Se repiten cuestiones ya tratadas.
 - Nuevos requisitos se consideran fuera del alcance del proyecto.
 - Nuevos requisitos son de prioridad baja.
 - Se sugieren funcionalidades "para el futuro".



Universidad de Oviedo

Tema 4

Análisis de Requisitos

1

Jorge Álvarez Fidalgo

Introducción

Abarca aquellas actividades que nos **permiten conseguir un entendimiento más detallado** y preciso de los requisitos:

- **Analizar** la información obtenida de los **usuarios**.
- **Detallar** adecuadamente **requisitos** de alto nivel.
- Estudio de los atributos de **calidad**.
- **Derivar** requisitos funcionales a partir de otros requisitos.
- Definir **prioridades**.
- Identificar requisitos **innecesarios** o **incompletos**.



3

Refinado de Requisitos

Refinado de Requisitos

El objetivo consiste en **estudiar y detallar los requisitos de alto nivel** producidos en la Obtención para elaborar requisitos de **software**.

- Requisito de alto nivel:
 - El sistema generará una factura.
- Requisitos de software:
 - El sistema generará una factura por cada pedido realizado.
 - El sistema realizará el proceso de generación de facturas cada hora.
 - Las facturas generadas por el sistema contendrán la siguiente información: [...]
 - Por cada factura generada, el sistema cambiará el estado del pedido asociado a "Facturado".
 - Por cada factura generada, el sistema creará un registro de cobro.
 - El estado del nuevo registro de cobro será "Pendiente de pago".

Modelado visual

- Empleamos **modelos visuales** para:
 - Representar el **dominio** del problema.
 - Representar conceptualmente el nuevo **sistema**.
- Representan:
 - Datos y objetos del dominio.
 - Transacciones y transformaciones.
 - Cambios en el estado.
- Extrapolamos modelos a partir de los requisitos textuales...
 - ...o extraemos RF a partir de modelos de alto nivel.

Modelado visual

- El uso de modelos visuales **facilita la detección de errores** y omisiones.
 - Contrastar grandes listados textuales resulta inadecuado para las capacidades humanas.
- La **complejidad** de un sistema no es impedimento para modelar.
 - Si somos incapaces de modelar un sistema complejo, implementarlo será casi imposible.
- Los modelos generados en esta fase podrían usarse como modelos de hombre de paja en futuros proyectos.

Modelos visuales

- Diagramas de casos de uso.
- Diagramas de actividad.
- Diagramas de flujo de datos.
- Diagramas *swimlane*.
- Diagramas de estado.
- Mapas de diálogo.
- Árboles de decisión.
- Diagramas entidad-relación.
- Diagramas de clase.

Casos de uso

- **Caso de uso:** secuencia de interacciones entre un actor externo y un sistema que resulta en la obtención de un valor para dicho actor.
- Siempre siguen la estructura verbo + objeto.
 - *Comprar billete, elegir asientos, facturar equipaje...*
- A partir de un caso de uso, se pueden extrapolar requisitos funcionales y sus correspondientes pruebas.

Casos de uso: actores

- **No confundir** usuarios y actores.

- Un mismo usuario puede jugar el rol de distintos actores.
- *Recordad: un actor es una abstracción que define un participante en el caso de uso.*

- Para **identificar** actores, tened presente:

- ¿A **quién se notifica** un evento del sistema?
- ¿**Quién provee** servicios o información al sistema?
- ¿**Quién ayuda** al sistema a completar una tarea?

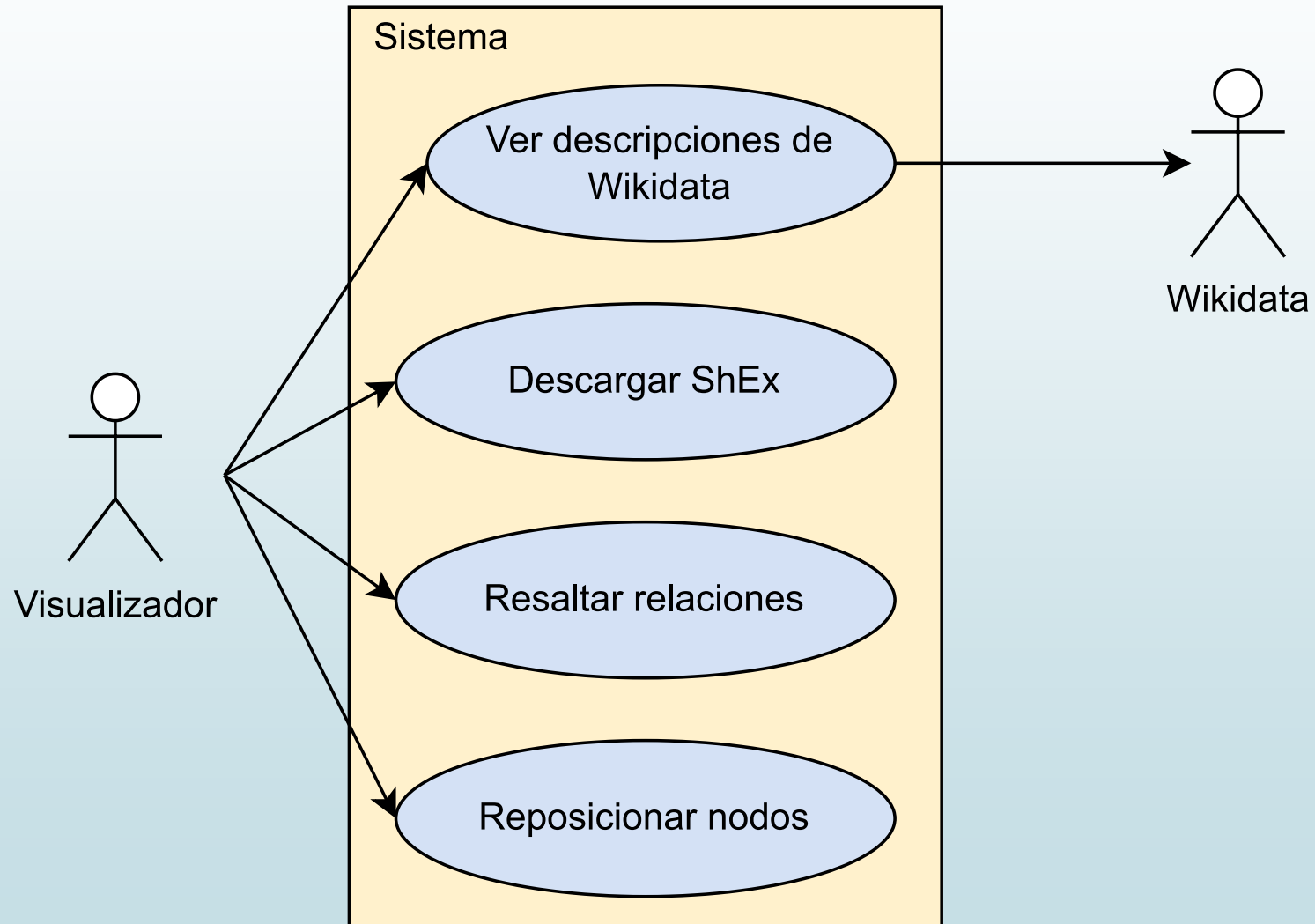
- Al igual que con los usuarios, no tienen por qué ser humanos.

Diagramas de casos de uso

- Se emplean **diagramas** para representar los casos de uso.
- Hacemos uso de una notación basada en **UML**. *
 - Una caja representa los límites del sistema.
 - Los casos de uso se representan mediante óvalos.
 - Las flechas entre actores y casos de uso representan las interacciones.
 - Actor -> caso de uso: actor primario del caso, lo inicia y obtiene valor.
 - Caso de uso -> actor: actor secundario, interviene de algún modo.

*[UML for IT Business Analyst: A practical guide, H. Podeswa]

Diagramas de casos de uso



Diagramas de casos de uso

- Los diagramas de casos de uso pueden incluir las siguientes relaciones:
 - **Extensión:** un caso de uso añade pasos a otro caso de uso de primer nivel.
 - **Inclusión:** un caso de uso requiere a otro caso de uso que no puede existir por sí mismo y está duplicado en varios casos de uso.

Ejemplo: *nuestro sistema es un cajero automático. Un caso de uso es "Retirar efectivo". El caso de uso "Calcular Comisión" extiende a "Retirar Efectivo" dado un "punto de extensión" (condición): la operación se realiza desde una entidad distinta a la del usuario.*

Por otra parte, "Retirar Efectivo" incluiría el caso de uso "Iniciar sesión", ya que es un prerequisite para ese y muchos otros casos de uso. Nótese que "Iniciar sesión" no puede existir por sí mismo ya que no aporta valor.

Casos de uso

- Un caso de uso ha de contener, como mínimo, la siguiente información:
 - **Identificador** único y **nombre** descriptivo,
 - **Descripción** textual.
 - Condición que **inicia** el caso de uso.
 - **Precondiciones** a satisfacer para empezar el caso de uso (0+).
 - **Postcondiciones** que describen el estado del sistema tras el caso de uso (1+).
 - **Flujo/s de eventos** que tiene lugar durante el caso de uso.
- Otras recomendaciones:
 - Fecha, autor, actores, prioridades, referencias, supuestos...

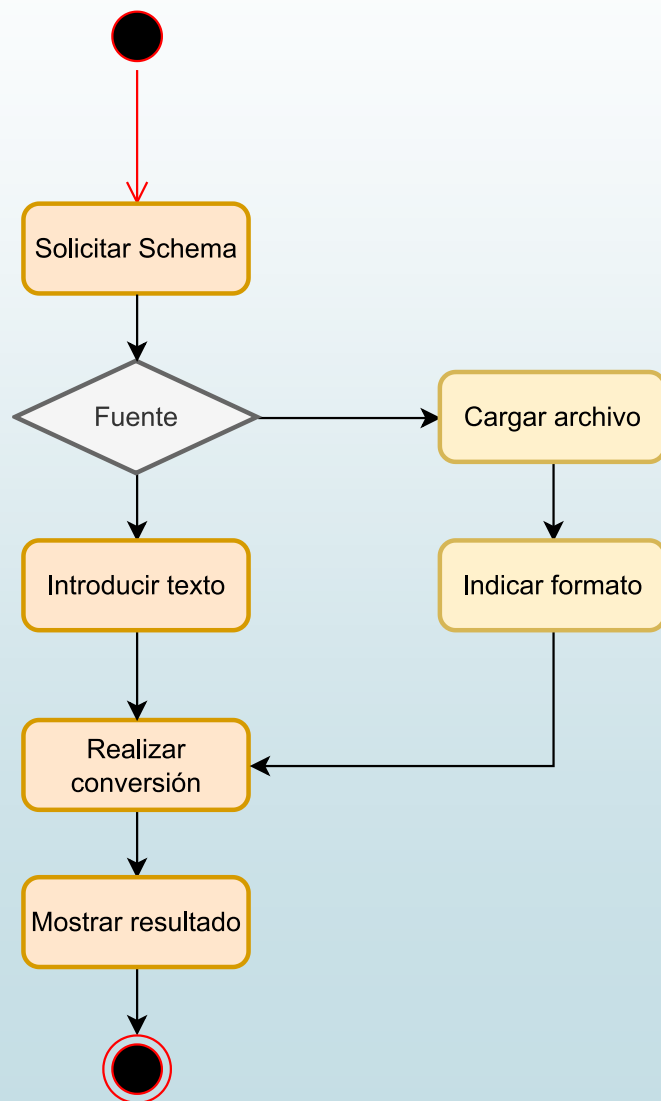
Casos de uso: condiciones

- **Precondiciones.** Requisitos a cumplir por el sistema previa ejecución del caso de uso.
 - La condición de inicio (*trigger*) no es una precondición.
 - *El cajero automático debe disponer de efectivo para realizar una retirada de efectivo.*
- **Postcondiciones.** Estado del sistema tras la ejecución del caso de uso.
 - Cambios observables por el usuario.
 - Salidas físicas.
 - Cambios internos en el sistema.
 - *El cajero automático debe actualizar su registro de efectivo tras una retirada.*
- Es posible que las postcondiciones de un caso de uso sean las precondiciones de otro.
 - Combinando casos de uso para formar una *macro*.

Casos de uso: flujos

- **Normales.** Representan la secuencia de acciones más habitual que produce un resultado satisfactorio. (*Escenario primario*)
- **Alternativos.** Representan secuencias de acciones menos comunes o relevantes que también producen un resultado satisfactorio. (*Esc. secundario*)
- **Excepciones.** Representan condiciones que pueden impedir un resultado satisfactorio.
 - Pueden ser recuperables o no.
 - Si no se especifican en esta fase:
 - El manejo de errores puede ser inconsistente.
 - El sistema puede fallar en determinadas condiciones.
 - Excepciones comunes (errores de conexión, de BD...) deberían extraerse a sus propios RF.

Casos de uso: d. de actividad



Casos de uso: escenarios

- Un **escenario** es una descripción de una **instancia** concreta de **uso** del sistema.
- Un caso de uso consta de **múltiples** escenarios relacionados...
 - ...y un escenario es una instancia específica de un caso de uso.
- De cara al análisis de requisitos, podemos:
 - Extrapolar escenarios específicos a partir de un caso de uso general.
 - Generalizar un escenario concreto para obtener un caso de uso general.

Casos de uso

ID y Nombre:	UC-33 Solicitar una miniatura
Creado por:	Seth
Fecha de creación:	06/09/2020
Actor Primario:	Solicitante
Actores Secundarios:	Impresor, Almacén
Descripción:	El Solicitante indica la miniatura deseada introduciendo su nombre, identificador numérico o seleccionándola del catálogo. El sistema ofrece al Solicitante obtenerla del Almacén si hay stock o permite al Solicitante imprimirla en 3D.
Condición inicio:	Solicitante indica que quiere solicitar una miniatura.
Precondiciones:	PRE-1. La identidad del usuario ha sido autenticada. PRE-2. La BD del inventario del almacén está en línea.
Postcondiciones:	POST-1. La solicitud es almacenada en el sistema. POST-2. La solicitud es enviada al Almacén o al Impresor.
Flujo normal:	33.0 Solicitar una miniatura del Almacén. 1. El Solicitante especifica la miniatura deseada. 2. El sistema lista las réplicas de la miniatura deseada que se encuentran en el Almacén. 3. El Solicitante selecciona una miniatura específica o solicita una orden de impresión (ver 4.1). 4. El Solicitante introduce otra información necesaria para completar la solicitud. 5. El sistema almacena la solicitud y notifica al Almacén.
Flujos alternativos:	33.1 Solicitar una miniatura al Impresor. 1. El Solicitante busca en los catálogos del Impresor la miniatura (ver 4.1.E1). 2. El sistema lista los tamaños y materiales disponibles para la impresión. 3. El Solicitante selecciona un tamaño, material y nº de miniaturas. 4. El Solicitante introduce otra información necesaria para completar la solicitud. 5. El sistema almacena la solicitud y notifica al Impresor.
Excepciones:	33.1.E1 Miniatura no disponible. 1. El sistema informa que no existe una miniatura de esas características en los catálogos. 2. El sistema pregunta al Solicitante si desea solicitar otra miniatura (3a) o salir (4a). 3a. El Solicitante pide solicitar otra miniatura. 3b. El sistema empieza de nuevo el flujo normal. 4a. El Solicitante pide salir. 4b. El sistema termina el caso de uso.
Prioridad:	Alta
Frecuencia de Uso:	Aproximadamente 30 veces a la semana.
Reglas de negocio:	BR-88
Otra información:	N/A
Supuestos:	Las miniaturas a imprimir están libres de derechos de autor.

Casos de uso: ejemplo

- **ID:** UC-33.
- **Nombre:** Solicitar una miniatura.
- **Creado por:** Seth.
- **Fecha de creación:** 06/09/2020.
- **Actor primario:** Solicitante.
- **Actores secundarios:** Impresor, Almacén.
- **Descripción:** El Solicitante indica la miniatura deseada introduciendo su nombre, identificador numérico o seleccionándola del catálogo. El sistema ofrece al Solicitante obtenerla del Almacén si hay stock o permite al Solicitante imprimirla en 3D.

Nombre
Precond
Postcond
Disparadores
Condición Error
Estado después Error
Situación Estándar
Situación Específica

Casos de uso: ejemplo

- **Condición de inicio:** Solicitante indica que quiere solicitar una miniatura.
- **Precondiciones:**
 - PRE-1. La identidad del usuario ha sido autenticada.
 - PRE-2. La BD del inventario del Almacén está en línea.
- **Postcondiciones:**
 - POST-1. La solicitud es almacenada en el sistema.
 - POST-2. La solicitud es enviada al Almacén o al Impresor.

Casos de uso: ejemplo

► **Flujo normal:** 33.0 Solicitar una miniatura del Almacén.

1. El Solicitante especifica la miniatura deseada.
2. El sistema lista las réplicas de la miniatura deseada que se encuentran en el Almacén.
3. El Solicitante selecciona una miniatura específica o solicita una orden de impresión (ver 4.1).
4. El Solicitante introduce otra información necesaria para completar la solicitud.
5. El sistema almacena la solicitud y notifica al Almacén.

Casos de uso: ejemplo

► **Flujos alternativos:** 33.1 Solicitar una miniatura al Impresor.

1. El Solicitante busca en los catálogos del Impresor la miniatura (ver 4.1.E1).
2. El sistema lista los tamaños y materiales disponibles para la impresión.
3. El Solicitante selecciona un tamaño, material y nº de miniaturas.
4. El Solicitante introduce otra información necesaria para completar la solicitud.
5. El sistema almacena la solicitud y notifica al Impresor.

Casos de uso: ejemplo

► **Excepciones:** 33.1.E1 Miniatura no disponible.

1. El sistema informa que no existe una miniatura de esas características en los catálogos.
2. El sistema pregunta al Solicitante si desea solicitar otra miniatura (3a) o salir (4a).
 - 3a. El Solicitante pide solicitar otra miniatura.
 - 3b. El sistema empieza de nuevo el flujo normal.
 - 4a. El Solicitante pide salir.
 - 4b. El sistema termina el caso de uso.

Casos de uso: ejemplo

- **Prioridad:** Alta.
- **Frecuencia de Uso:** Aproximadamente 30 veces a la semana.
- **Reglas de negocio:** BR-88.
- **Otra información:** N/A
- **Supuestos:** Las miniaturas a imprimir están libres de derechos de autor.

Casos de uso: nivel de detalle

- El nivel de detalle depende de la **relevancia** de cada caso.
- Podemos **limitarnos** a una descripción del objetivo del usuario y sus interacciones con el sistema, o sólo mencionar los flujos.
- Una especificación **completa** puede ser apropiada si:
 - Se trata de un sistema complejo cuyos fallos ocasionan gran riesgo.
 - Los casos de uso representan requisitos novedosos para los desarrolladores.
 - Los casos de uso son los requisitos más detallados que los desarrolladores dispondrán.
 - Existirá escasa interacción entre los usuarios y el equipo de desarrollo.
 - Se pretende elaborar casos de prueba basados en los requisitos de usuario.

Casos de uso: identificación

- Existen distintos enfoques:
 - 1) Identificar los actores, 2) listar los procesos de negocio, 3) buscar **interacciones** entre ambos para definir casos de uso.
 - Describir un **escenario** concreto para cada proceso de negocio, y generalizar casos de uso a partir de éstos.
 - Estudiar qué **tareas** debe realizar el sistema para completar un proceso de negocio; pueden ser casos de uso.
 - Identificar los **eventos** externos a los que debe responder el sistema y relacionarlos con los actores.
 - Identificar datos que requieran un **CRUD**.
 - Estudiar el rol de los **sistemas externos** que deben interaccionar con el sistema.

Casos de uso: elaboración

Pasos propuestos para la elaboración de casos de uso:

1. Identificar el **actor primario**.
2. Elaborar la **descripción**.
3. Definir **precondiciones** y **postcondiciones**. *Límites del caso.*
4. Definir el **flujo normal** de pasos.
5. Definir el **resto** de flujos.

Requisitos funcionales y casos de uso

- **Requisitos funcionales != casos de uso.**
 - Los casos de uso cubren únicamente el comportamiento "**externo**" del sistema.
- Debemos **especificar** los requisitos funcionales necesarios para implementar cada caso de uso.
 - *P.e.: ¿Qué hacer si una precondición no se satisface?*
- Mantener **trazabilidad** entre casos de uso y requisitos funcionales.

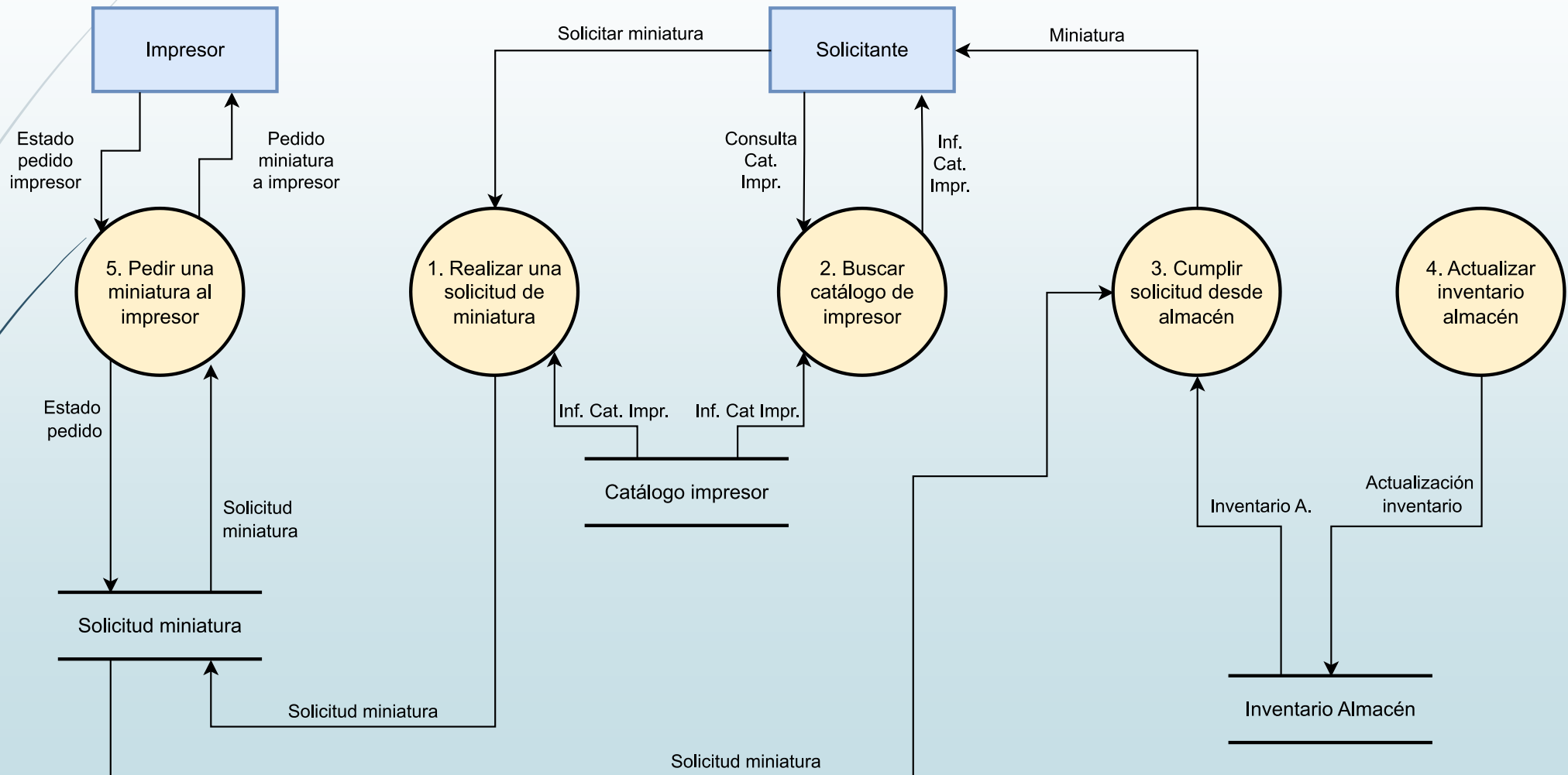
Casos de uso: problemas

- **Demasiados casos de uso.**
 - No es necesario cubrir todos los escenarios posibles.
 - Detallar los más relevantes.
- Casos de uso **excesivamente complejos.**
 - Si un flujo excede 15 pasos, es posible que incluya más de un escenario.
- **Dependencia del diseño.**
 - "El sistema muestra un *drop-down*" -> "El sistema presenta las opciones".
- Casos de usuario que los **usuarios no comprenden.**
 - Si un usuario no relaciona un caso de uso con sus objetivos, estamos perdiendo la perspectiva del usuario.

Diagramas de flujo de datos

- Adecuados para sistemas de procesamiento de **transacciones**.
- Describen el **movimiento de datos** en un sistema.
 - Comunicaciones, transformaciones, colecciones...
 - *Ejemplo: ¿cuántos datos requiere la creación de un pedido?*
- Útiles para extraer **requisitos de datos**.
- A menudo usados en entrevistas.

Diagramas de flujo de datos



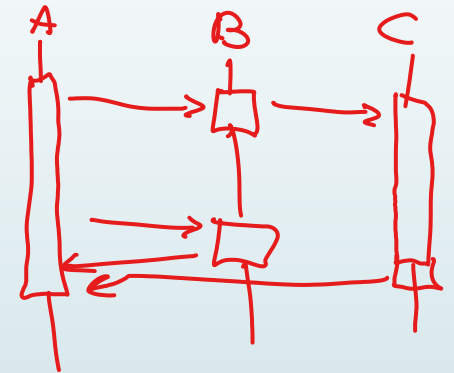
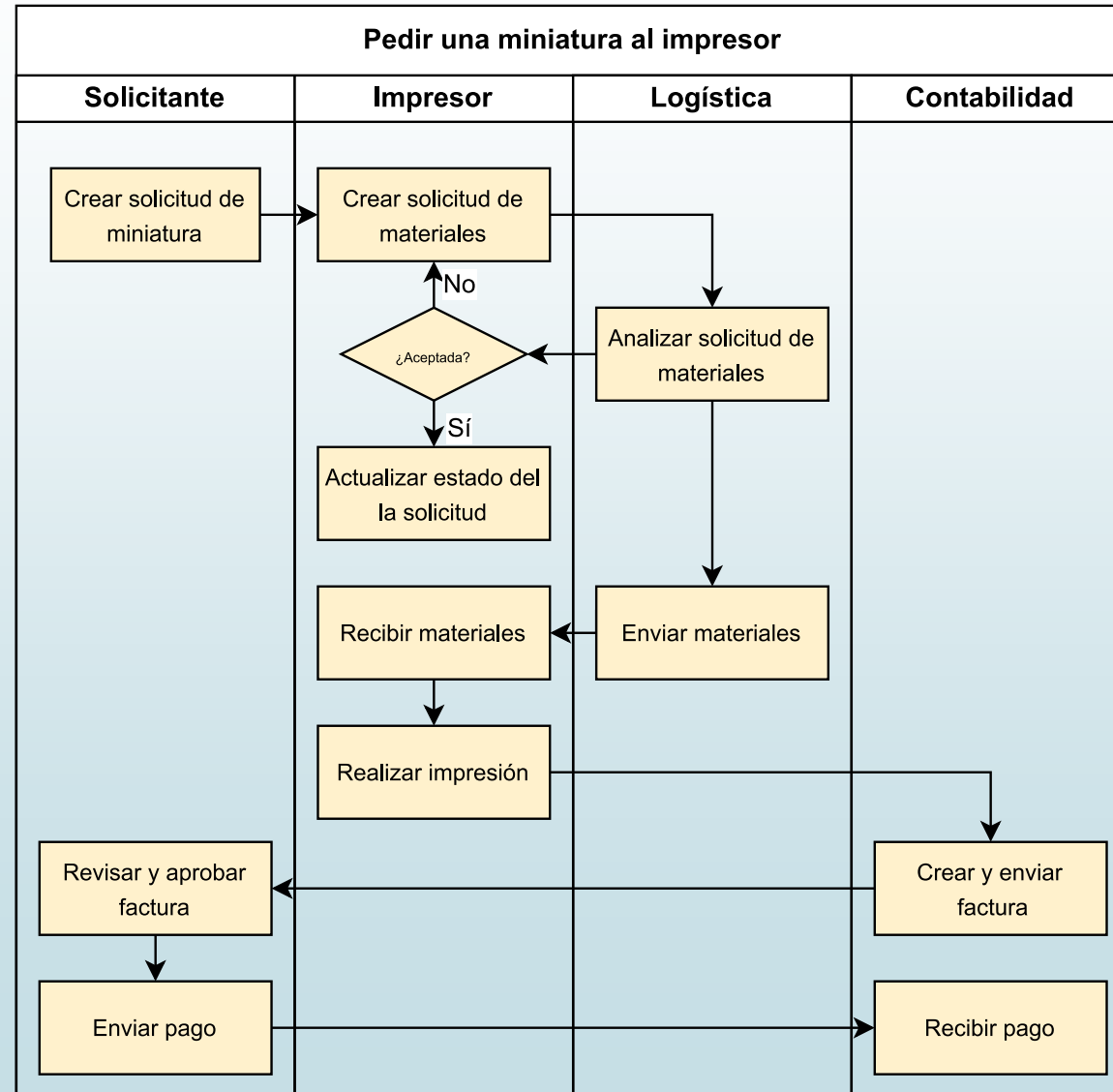
Diagramas de flujo de datos

- Algunas **convenciones** a tener en cuenta:
 - Los datos no pueden comunicarse directamente entre entidades del mismo tipo.
 - El DFD no debería tratar de representar la secuencia del proceso.
 - Los procesos se identifican de manera única y dependiendo del nivel de detalle.
 - Los procesos se nombran de forma concisa.
 - Evitar 10+ procesos; abstraer procesos de mayor nivel si es necesario.
 - Los procesos deberían tener tanto entradas como salidas de datos.

Diagramas *swimlane*

- Representan las **operaciones** a realizar para ejecutar un proceso del sistema.
- Similares a los diagramas de actividad, pero subdivididos en **secciones** que representan a los **actores** que realizan una serie de pasos.

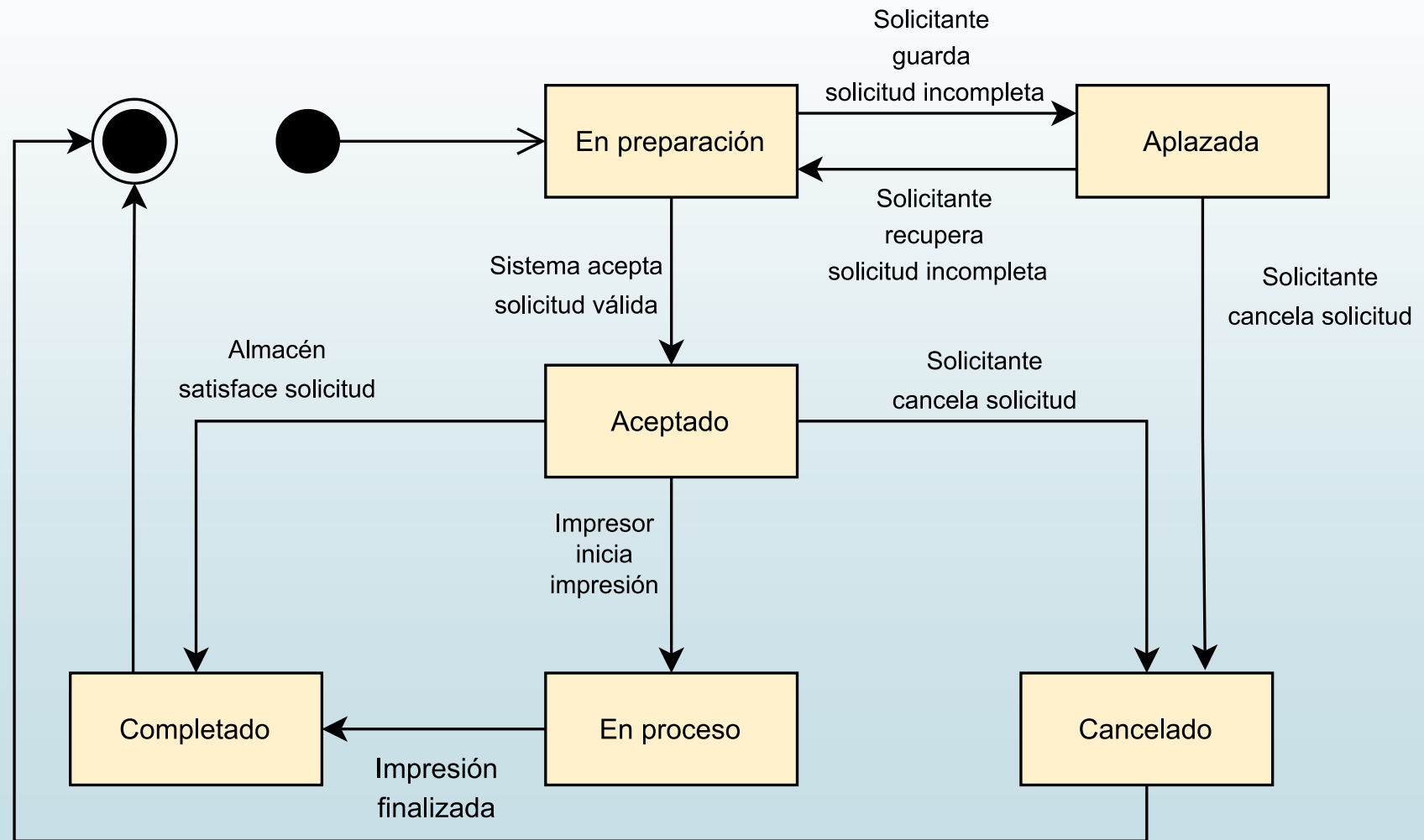
Diagramas swimlane



Diagramas de estado

- Describen los **cambios de estado** de un sistema bajo unas condiciones determinadas.
- Uso típico en sistemas de tiempo real y en la gestión de pedidos, inventarios y similares.
- Los cambios de estado se representan mediante flechas entre estados; los eventos que ocasionan dichos cambios se indican mediante etiquetas.
- Se suele representar alternativamente mediante una **tabla de estados**.

Diagramas de estado

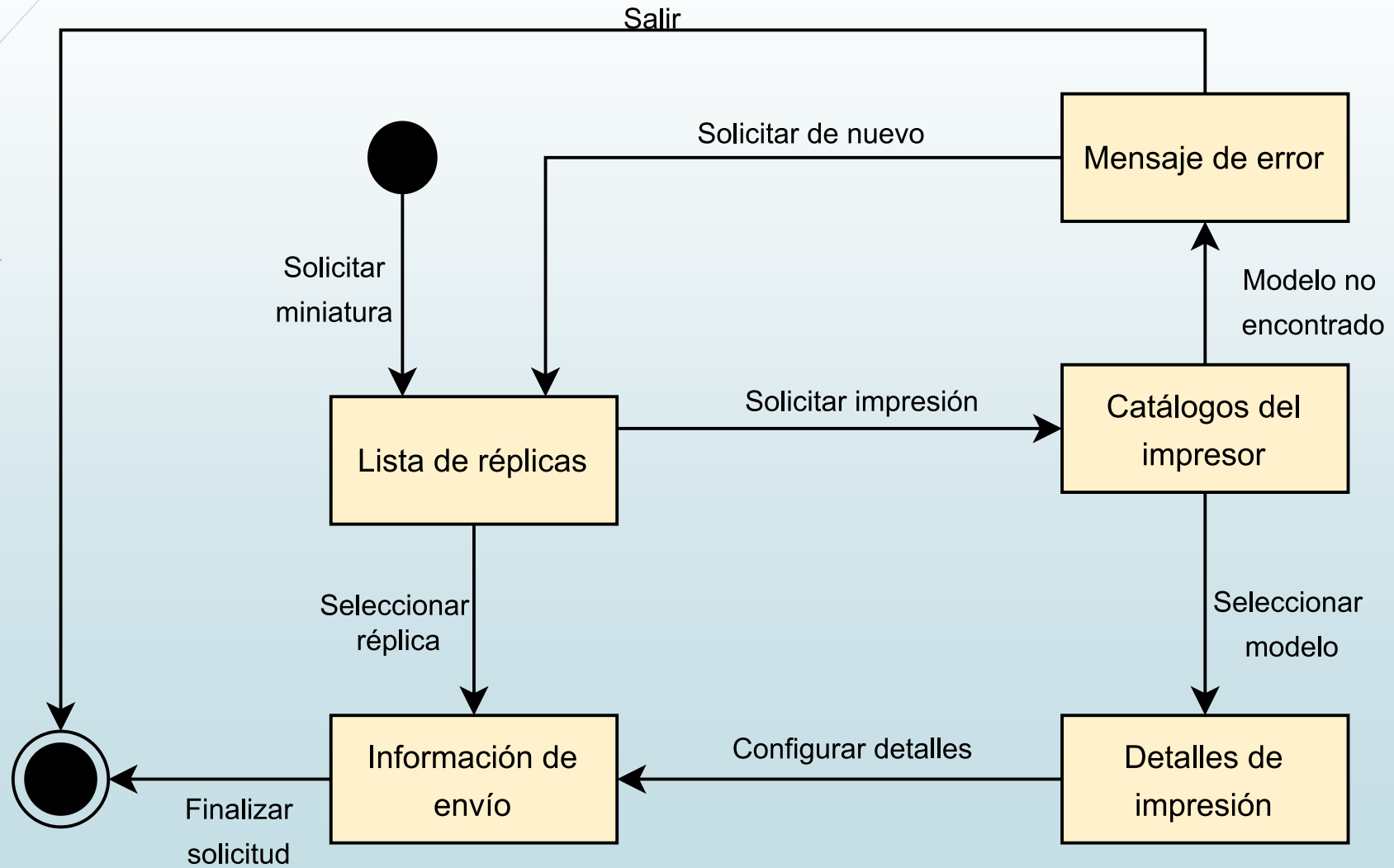


Mapas de diálogo

- Describen la **interfaz de usuario** a un nivel de **abstracción alto**.
 - Diálogos y sus relaciones, sin entrar en cuestiones de diseño.
 - Puede considerarse un **diagrama de estados** de la interfaz de usuario.
- Permite **explorar el flujo de interacciones** entre usuarios y sistema **sin** entrar en **detalles** de los procesos o cuestiones de diseño de IU.
 - Son un buen complemento para los casos de uso.
- Las transiciones entre diálogos pueden deberse a:
 - Acciones del usuario.
 - Valores de datos.
 - Estados del sistema.



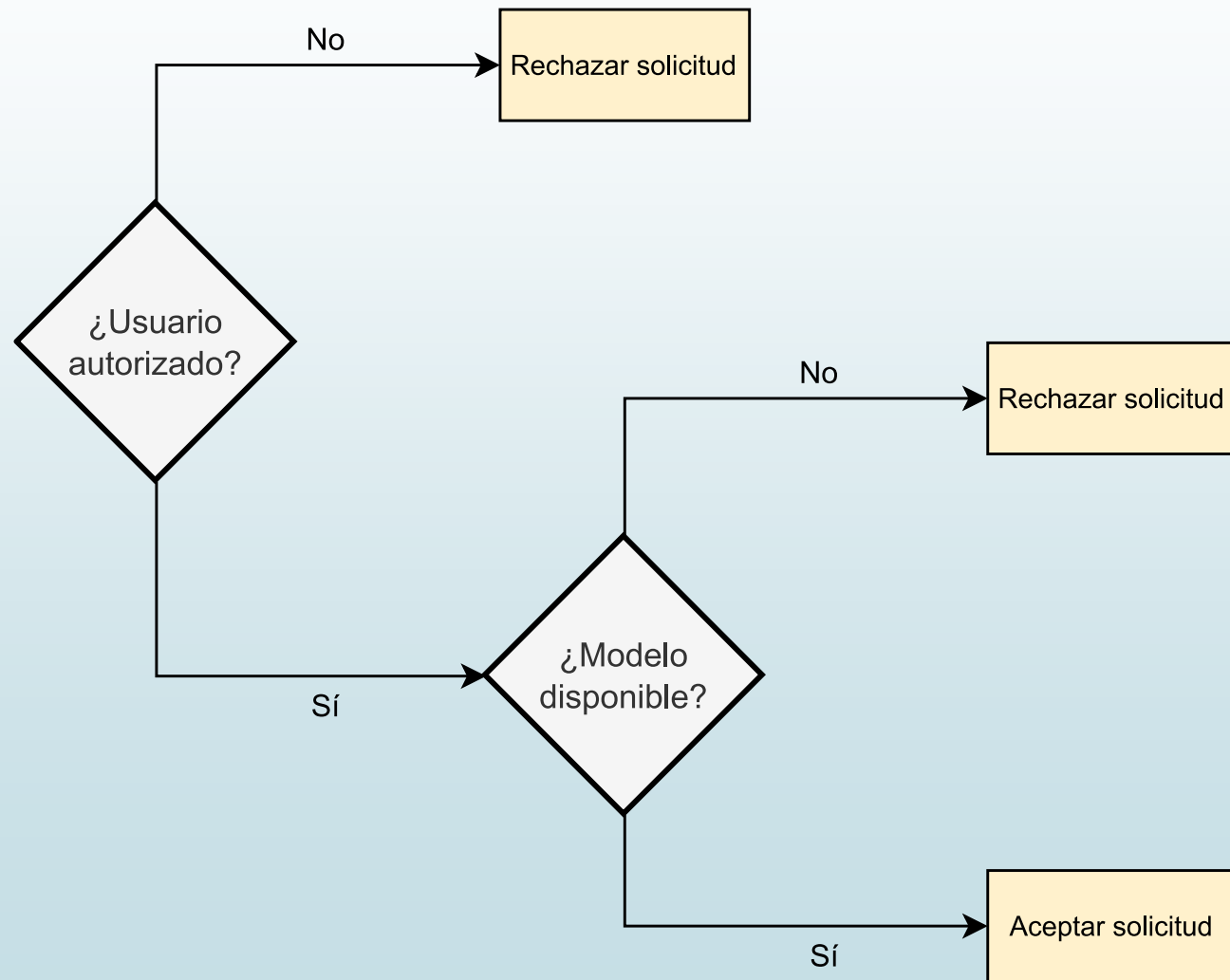
Mapas de diálogo



Árboles de decisión

- Representan la **secuencia lógica** de acciones a tomar por el sistema bajo unas **condiciones** determinadas.
 - *Toma de decisiones.*
- Adecuados para documentar **restricciones** de negocio.

Árboles de decisión



Diagramas E-R

- Un diagrama E-R **representa entidades y las relaciones** entre ellas.
- En el Análisis de Requisitos, se emplean para representar entidades del **dominio** y sus interrelaciones.
- Se emplean en el Diseño de un proyecto **o para definir BD...**
 - ... pero su uso en el Análisis **no** implica la existencia de una BD. (Mayor nivel de abstracción).
 - No obstante, puede ser un buen punto de partida.

Diagramas E-R

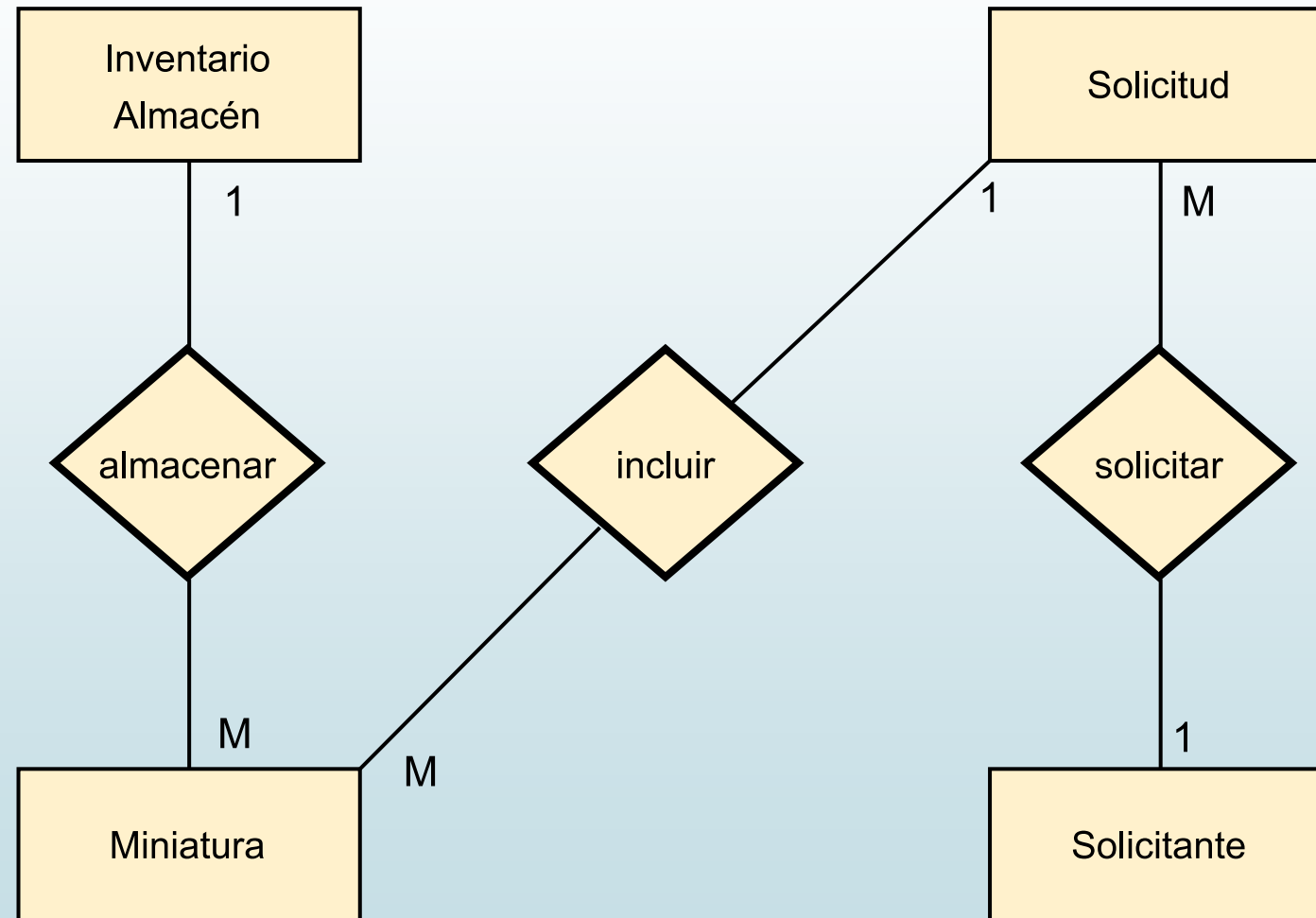
► **Entidades.** Representan elementos **materiales** o conjuntos de **datos** relevantes para el negocio/sistema.

- Tomando como referencia el DFD, tanto los actores como los almacenes de datos se convierten en entidades en el E-R.
- Se definen en base a una serie de atributos que deberían especificarse aparte.
- En el diseño de BD, se suelen convertir en tablas.

► **Relaciones.** Conexiones lógicas entre **pares** de entidades.

- Recomendable etiquetarlos con un infinitivo.
- Es posible expresar la cardinalidad (1-1, 1-M, M-M).

Diagramas E-R

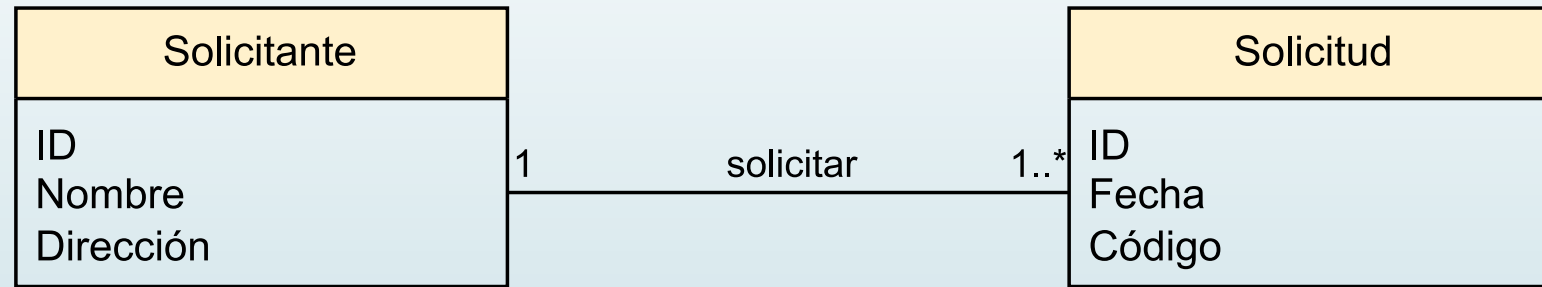


Este diagrama representa parcialmente el caso de uso previo (he ignorado las impresiones, por sencillez)

Diagramas de clase

- **Alternativa** a los diagramas E-R habitual en los desarrollos orientados a objetos.
- Las clases se corresponden con las entidades. Relaciones y cardinalidades se expresan con líneas simples.
- Los **atributos** que en un E-R se obviaban aquí se especifican.
 - No así las operaciones, ya que son cuestiones de implementación.

Diagramas de clase



Refinado de Requisitos II

Otras técnicas de interés

Matrices CRUD

- Técnica para el análisis de requisitos de **datos**.
- Consiste en relacionar acciones del sistema y datos para estudiar cuando éstos se crean, leen, actualizan o eliminan (**CRUD**).
- Algunas combinaciones posibles:
 - Datos y eventos.
 - Datos y casos de uso.
 - Clases y casos de uso.

Matrices CRUD

Create, Read, Update, Delete

	Solicitud	Miniatura	Solicitante
Crear solicitud	C	R	R
Cambiar solicitud	U, D		R
Ver solicitudes	R		R
Gestionar inventario almacén		C, U, D	
Registro			C
Gestionar cuenta			U



NO QUIERO MORIR

¿Se realizan todas las operaciones para una entidad? ¿Por qué?

Atributos de calidad

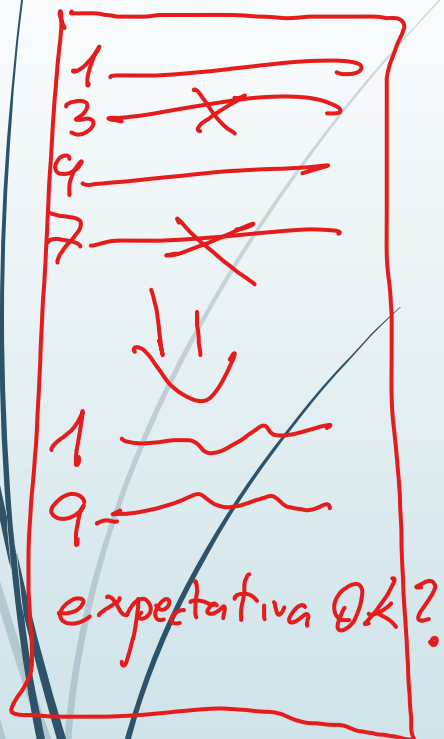
Análisis de atributos de calidad

Según la clase de sistema, algunos atributos son más importantes:

- **Aplicaciones web y corporativas:** disponibilidad, escalabilidad, integridad (de datos), interoperabilidad, rendimiento, seguridad, usabilidad.
- API ➤ **Sistemas integrados:** eficiencia, fiabilidad, rendimiento, robustez, seguridad, usabilidad?
- **Aplicaciones de escritorio y móviles:** rendimiento, seguridad, usabilidad.

Esto no significa que se limite a ellos; depende de las características del sistema y del dominio. (Requisitos emocionales de Callele, Neufeld y Schneider)

Análisis de atributos de calidad



El análisis de atributos de calidad consta de:

1. Partir de un **listado completo** de atributos.
2. **Eliminar** atributos según la información extraída en la Obtención.
3. **Priorizar** atributos.
4. **Concretar** expectativas y necesidades.

Lista de atributos de calidad

Una posible **clasificación** de los atributos de calidad es la que sigue:

► **Externos.** Características observadas durante la ejecución del sistema, que influyen al **usuario y** su experiencia.

► *Disponibilidad, instalabilidad, integridad, interoperabilidad, rendimiento, fiabilidad, robustez, seguridad, usabilidad...*

► **Internos.** Propiedades **percibidas** por **el equipo técnico** durante su interacción con el sistema, que pueden afectar indirectamente a los usuarios.

► *Eficiencia, modificabilidad, portabilidad, reusabilidad, escalabilidad, verificabilidad...*

A continuación, se definen y detallan los más relevantes.

Disponibilidad

- Medida del **tiempo** durante el que el sistema es **completamente operacional**.
 - $\text{Tiempo disponible} / \text{Tiempo total}$.
 - $\text{Tiempo medio entre fallos} / \text{Tiempo medio entre fallos} + \text{Tiempo medio de reparación}$.
- Relevante en aplicaciones web y, en general, sistemas de uso global.
- Estrechamente relacionada con la fiabilidad y la mantenibilidad.

Disponibilidad

- Se debe examinar si los **objetivos de negocio** exigen la disponibilidad del sistema, o parte de éste, durante determinados períodos de tiempo.
- Recomendable acordar cuándo se considera un sistema **disponible**.
 - ¿Qué ocurre con el mantenimiento rutinario? ¿Se considera disponible si es accesible en un modo limitado?
- Una disponibilidad extremadamente alta implica **costes** muy altos (redundancia del hardware, arquitectura compleja).
 - Sólo deberíamos aproximarnos al 100% en sistemas particularmente críticos.

Disponibilidad

- A considerar: [Miller 2009]
 - ¿Qué partes del sistema requieren de una mayor disponibilidad?
 - ¿Qué consecuencias tiene para los objetivos de negocio la no disponibilidad?
 - ¿Debe realizarse un mantenimiento periódico? ¿Cómo afecta a la disponibilidad?
 - ¿Cómo se notifica al usuario la no disponibilidad del sistema?
 - ¿Existen dependencias de disponibilidad entre funcionalidades?

El sistema estará disponible un 99% del tiempo entre las 6.00 y las 24:00.

Fiabilidad

- Probabilidad de que el sistema se ejecute **sin fallos** durante un período de tiempo.
- **Métricas** habituales:
 - % de operaciones completadas sin fallo.
 - Tiempo medio entre fallos (MTBF).
- Como es habitual con los atributos de calidad, es difícil asegurar si un requisito de fiabilidad se cumple...
 - ... pero sí podemos detectar si no se cumple.

Fiabilidad

- Aspectos a considerar:
 - ¿Qué ocurre si tiene lugar un fallo al realizar una operación concreta?
 - ¿Cuáles son las consecuencias de un fallo para los objetivos del sistema?
 - No todos los fallos son iguales: distinguir entre fallos críticos y simples molestias.
 - ¿Existen elementos del sistema cuya fiabilidad es de absoluta importancia?

El tiempo medio entre fallos del subsistema X será de 42 días.

Instalabilidad

- Medida de la facilidad de realización de las operaciones relacionadas con la **instalación** del sistema.
 - Instalación, recuperación, actualización, regresión, desinstalación...
- Una métrica típica es el **tiempo medio de instalación**.
- Posibles requisitos: pasos de la instalación, requisitos mínimos, variaciones por plataforma y región, niveles de acceso, configuraciones varias...

La instalación del sistema requerirá de privilegios de administrador.

Integridad

- Preservación de la **corrección** y la **completitud** de los **datos** del sistema.
- El objetivo es evitar:
 - Pérdida o corrupción de los datos.
 - Borrado o sobreescritura de los datos por parte de los usuarios.
 - Robo o corrupción intencionado de los datos.
 - Imprecisiones, inconsistencias o errores en el formato.
 - Véase ["Redondeo de los Misiles Patriot"](#).

Integridad

► **Recomendaciones** acerca de la integridad del sistema: *[Withall 2007]*

- Los cambios en los datos deben realizarse completamente o deshacerse si el proceso no se finaliza.
- Los cambios en los datos deben ser persistentes.
- Coordinar cambios realizados en distintos almacenes de datos.
- Realizar y recuperar copias de seguridad.
 - Hay mucho que sopesar: frecuencia, objetivos, automatización, cifrado...
- Archivado de datos.
 - Cuáles y durante cuánto tiempo...

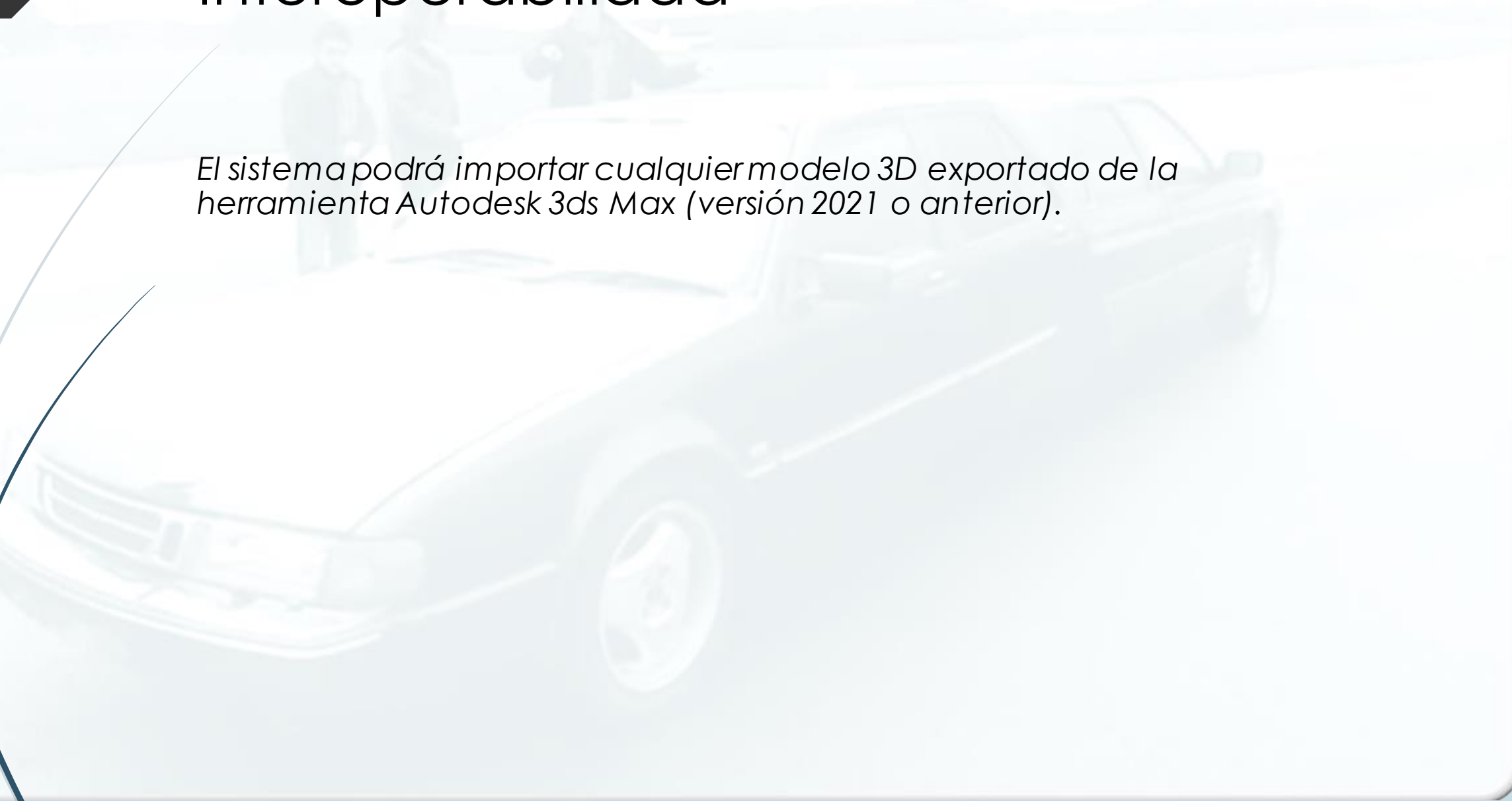
El sistema no redondeará el tiempo transcurrido desde el arranque del sistema al registrar el valor medido.

Interoperabilidad

- Facilidad de **intercambio** de datos y servicios con **otros sistemas**, así como de integración con dispositivos externos.
- Típicamente, consiste en definir **estándares** a emplear para el intercambio de datos entre sistemas.
- A considerar:
 - ¿Con qué sistemas hemos de conectarnos y qué datos se intercambian?
 - ¿Qué estándares de datos se emplean en estos sistemas?
 - ¿Qué mensajes recibe el sistema de otros sistemas o dispositivos?
 - ¿Existen restricciones externas acerca de la interoperabilidad?

Interoperabilidad

El sistema podrá importar cualquier modelo 3D exportado de la herramienta Autodesk 3ds Max (versión 2021 o anterior).



Rendimiento

- Medida de la **velocidad** o **éxito** con el que el **sistema** actúa.
 - Abarca múltiples aspectos: tiempo de respuesta, transacciones por segundo, nº de usuarios concurrentes, latencia, capacidad de la BD...
- Afecta notablemente a la **percepción** del usuario del sistema...
 - ... y también a otros atributos, especialmente en sistemas críticos.
- Estrechamente relacionado con la eficiencia.

El sistema situará la placa giratoria en la posición indicada por el usuario en un máximo de TIEMPO_MAXIMO_GIRO segundos.

Robustez

- Capacidad del sistema para soportar **condiciones** operativas **no ideales**: defectos de los sistemas externos, errores de los usuarios o ataques maliciosos.
- Reflexionar sobre **posibles errores** y cómo el sistema debería **recuperarse** sin afectar negativamente al usuario.

Si el editor de imágenes se cierra antes de que el usuario guarde el archivo, la próxima vez que el usuario inicie el editor el sistema recuperará los contenidos del archivo de, como máximo, TIEMPO_MAXIMO_RECUPERACIÓN antes del fallo.

Seguridad (*safety*)

- **Prevención** de **daños** personales o materiales por parte del sistema.
- Típicamente, se expresan como acciones o estados que el sistema **no** debe permitir.
 - Según el dominio, habrá restricciones externas más o menos detalladas.
 - Habituales en sistemas que controlan dispositivos físicos (Therac-25).
- A considerar:
 - ¿Cómo podría este sistema dañar a una persona? ¿Cómo puede el sistema detectar estos casos y cómo debería responder?
 - ¿Qué acciones del usuario podrían inadvertidamente causar daños?
 - ¿Existen modos de operación que suponen riesgos específicos?

Seguridad (safety)

El sistema finalizará inmediatamente el proceso de radiación ionizante si detecta niveles de radiación anómalos.



Seguridad (*security*)

- **Protección** de ataques y accesos no autorizados a las capacidades e información del sistema.
- Aspectos a tratar:
 - Autenticación de los usuarios y niveles de acceso.
 - Privacidad de los datos.
 - Protección contra malware.
 - Encriptado de datos.
 - Logs de operaciones y accesos.

Seguridad (*security*)

- En **SSI**, se describe cómo elaborar requisitos de seguridad (gestión de datos, autenticación y otros) y se proveen algunos ejemplos:
 - [Tema 6: Introducción a la seguridad de aplicaciones.](#)
 - [Seminario 5 \(ejemplos\).](#)

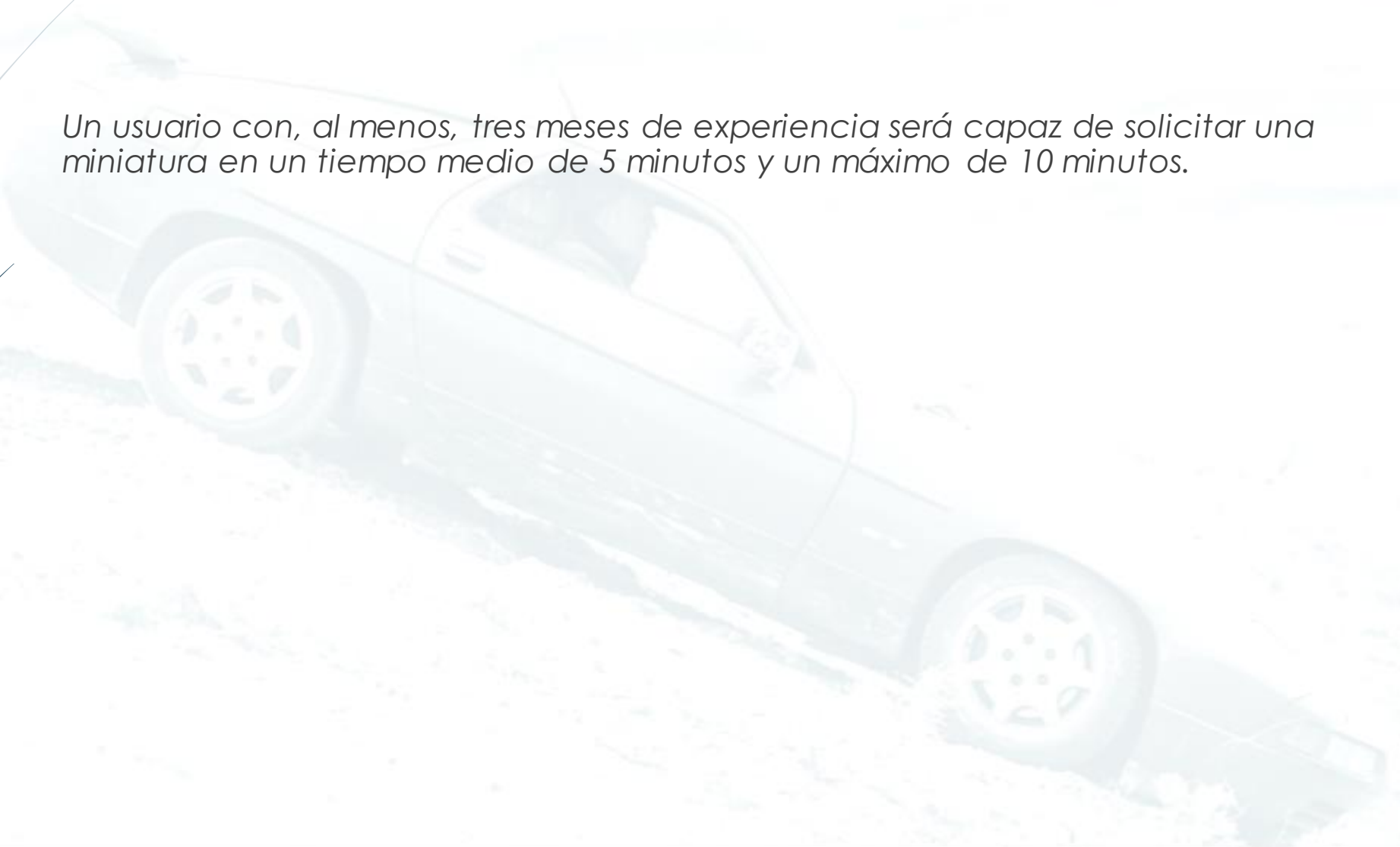
El sistema no permitirá a los usuarios reutilizar contraseñas que haya empleado previamente.

Usabilidad

- **Esfuerzo** necesario por parte del **usuario** para operar con el sistema e interpretar sus salidas.
- Abarca muchos temas, en ocasiones **contradictorios**:
 - Facilidad de uso, facilidad de aprendizaje, accesibilidad, gestión de errores...
- **Métricas**:
 - Tiempo medio para realizar una tarea.
 - % de tareas realizadas sin ayuda/errores.
 - N° de errores durante una tarea.
 - N° de interacciones requerido para realizar una tarea.

Usabilidad

Un usuario con, al menos, tres meses de experiencia será capaz de solicitar una miniatura en un tiempo medio de 5 minutos y un máximo de 10 minutos.



Eficiencia

- **Aprovechamiento** del sistema de los **recursos** de procesamiento.
 - Directamente relacionada con el rendimiento.
- A considerar:
 - Definir requisitos mínimos de hardware.
 - Definir márgenes para condiciones inesperadas y futuras.
 - Lo anterior se basará en el n° de operaciones y usuarios concurrentes previstos para el sistema.

El 33% de la memoria RAM disponible para el sistema se dejará libre durante los períodos de máxima carga previstos.

Escalabilidad

- Capacidad del sistema para **acomodar más** servicios o usuarios **sin comprometer** otros atributos.
- Puede implicar ampliaciones de hardware u optimizaciones de software.
- A considerar:
 - ¿Cómo podría evolucionar la demanda del sistema en los próximos años?
 - ¿Cuál es el rendimiento mínimo aceptable independientemente del n° de usuarios?

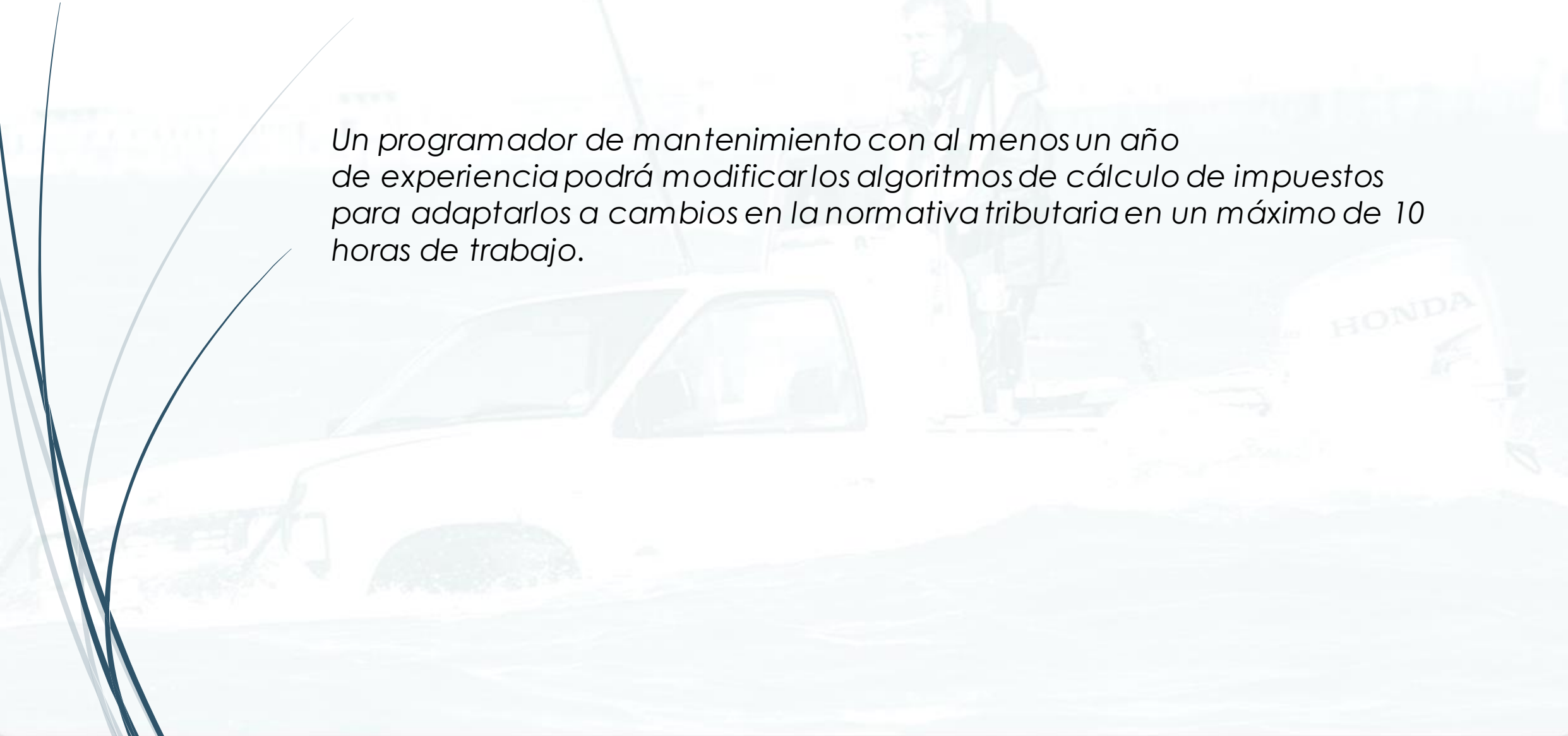
La capacidad del sistema debe ser capaz de incrementarse de 420 transacciones al día a 1500 transacciones al día en un máximo de 24 horas.

Modificabilidad

- Facilidad para **comprender**, **cambiar** y **extender** el diseño y código del sistema.
- Varias dimensiones dependientes del tipo de mantenimiento:
 - Si **corregimos** defectos, nos centramos en mantenibilidad y comprensibilidad.
 - Si **mejoramos** y extendemos la funcionalidad: extensibilidad, flexibilidad.
 - Si debemos **adaptarnos** a nuevos entornos: mantenibilidad.
- **Métricas:**
 - Tiempo medio para añadir nueva característica/solucionar un defecto.
 - % de modificaciones realizadas exitosamente.

Modificabilidad

Un programador de mantenimiento con al menos un año de experiencia podrá modificar los algoritmos de cálculo de impuestos para adaptarlos a cambios en la normativa tributaria en un máximo de 10 horas de trabajo.



Portabilidad

- Esfuerzo necesario para **migrar** el sistema de un **entorno** a otro.
 - Algunos consideran la internacionalización como parte de la portabilidad.
- Es común que una aplicación deba funcionar en distintos sistemas operativos, navegadores o dispositivos.
- A considerar:
 - ¿En qué plataformas debe funcionar este sistema?
 - ¿Qué elementos del sistema necesitan ser portables?
 - ¿Se comprometen otros atributos de calidad al incrementar la portabilidad?

El sistema permitirá importar las características del perfil entre iOS y Android.

Reusabilidad

- Esfuerzo requerido para **emplear parte** del sistema en otros sistemas.
- Algunos **prerrequisitos** de la reusabilidad:
 - Modularidad, independencia del entorno, documentación precisa, cierta genericidad.
- A considerar:
 - ¿Qué requisitos o datos podrían ser reusados en el futuro?
 - ¿Qué funcionalidades de este sistema podrían emplearse en otros?

Los algoritmos de cálculo de impuestos serán reusables en futuras aplicaciones de compra-venta.

Verificabilidad

- Capacidad de **evaluación** de las funcionalidades del sistema.
- Imprescindible en sistemas críticos o sometidos a numerosos cambios.
- A considerar:
 - ¿Cómo comprobamos que las operaciones producen los resultados deseados?
 - ¿Qué salidas del sistema podemos examinar para comprobar su validez?
 - ¿Existen funciones del sistema indeterministas?

La máxima complejidad ciclomática de un módulo del sistema será de 21.

Priorización de atributos de calidad

- Una técnica útil para la priorización es la **comparación por pares**.
- Tras contrastar todas las combinaciones, se **ordenan** en función de las comparaciones ganadas.
- La priorización nos permite **resolver conflictos** en fases posteriores.

Concreción de atributos de calidad

- Por sí solos, los atributos de calidad no nos proporcionan suficiente información.
 - *"El rendimiento es importante para nuestro sistema".*
- Debemos concretar las **expectativas** y **necesidades** de los stakeholders relativas a un atributo de calidad.
 - ¿Cuál es un tiempo de respuesta aceptable para el usuario?
 - ¿Cuál es un tiempo de respuesta inaceptable para el usuario?
 - ¿Cuál sería el nº de usuarios concurrentes de media?
 - ¿Cuál sería el máximo nº de usuarios concurrentes en un momento dado?
 - *Ver las preguntas listadas en "A considerar" en los atributos anteriores.*

Concreción de atributos de calidad

- Resulta útil **definir** los **límites** de lo aceptable.
 - ¿Cuál es un tiempo de respuesta inaceptable para el usuario?
- Nos permiten definir **pruebas** de cumplimiento.
 - Si no es posible forzar al sistema a una situación definida como inaceptable, probablemente hemos cumplido con los objetivos de calidad.
 - Los usuarios no autorizados no podrán eliminar archivos.



Universidad de Oviedo

Tema 5

Especificación de Requisitos Software

1

Jorge Álvarez Fidalgo

Introducción

Consiste en **representar y almacenar el conocimiento** de requisitos de forma clara y organizada:

- Producir **requisitos escritos** de forma completa, precisa y verificable.
- Generar **diagramas** adecuados para el uso y comprensión por parte de los interesados.

Documentación de Requisitos

- El acto de **documentar** requisitos nos **permite**:
 - Disponer de un **acuerdo escrito** con los stakeholders.
 - **Proporcionar la información** necesaria a los desarrolladores.
 - **Revisar** y concretar **requisitos**.
 - Formalizar por escrito un requisito puede revelar aspectos que la discusión verbal no contempla.
- Recordad: los requisitos perfectos no existen.
 - Se aspira a una aproximación suficientemente detallada de los requisitos relevantes para afrontar la siguiente iteración del proceso.

Documentación de Requisitos

- Se pueden documentar requisitos de los siguientes **modos**:
 - **Lenguaje natural** preciso y estructurado.
 - **Modelos visuales** y otras técnicas de representación.
 - **Especificaciones formales.**
- Las especificaciones formales son las más rigurosas...
 - ...pero también las más difíciles de elaborar y comprender.
 - Se emplean en sistemas particularmente críticos.
- En la **mayoría** de proyectos, lo más práctico es emplear lenguaje natural estructurado acompañado de representaciones visuales.

Especificación de Requisitos Software

Especificación de Requisitos Software

- **Documentación de los requisitos** esenciales del software (funciones, rendimiento, restricciones de diseño y atributos) y sus interfaces externas. [IEEE Std 610.12-1990]
- Describe el **comportamiento del sistema** bajo diversas condiciones, sus **atributos** deseados y las **restricciones** a respetar.
- Base para las posteriores fases del proyecto...
 - ... pero no debe incluir cuestiones de diseño, implementación, etc. (excepto aquellas impuestas por las restricciones).

Especificación de Requisitos Software

- El SRS **no** tiene por qué ser un documento tradicional.
 - Aunque, por sencillez, emplearemos dicho término.
- Representar requisitos en un documento escrito tiene sus **inconvenientes**:
 - El control de versiones y la gestión de cambios son imprácticos.
- Existen varias **alternativas**:
 - Base de datos.
 - Wiki.
 - Hoja de cálculo.
 - Herramientas de gestión de requisitos.

Estándar IEEE 830

- El estándar **IEEE 830** define un posible formato para un SRS.
 - Se emplea en las prácticas de la asignatura.
- Sustituido más tarde por **ISO/IEC/IEEE 29148:2011** y éste siendo revisado por **IEEE/ISO/IEC 29148-2018**.

Estándar IEEE 830

1. Introducción.
 1. Propósito.
 2. Ámbito del Sistema.
 3. Definiciones, Acrónimos y Abreviaturas.
 4. Referencias.
 5. Visión General del Documento.
2. Descripción General.
 1. Perspectiva del Producto.
 2. Funciones del Producto.
 3. Características de los Usuarios.
 4. Restricciones.
 5. Suposiciones y Dependencias.
3. Requisitos Específicos.
 1. Interfaces Externas.
 2. Funciones.
 3. Requisitos de Rendimiento.
 4. Requisitos lógicos de BD.
 5. Restricciones de Diseño.
 6. Atributos del Sistema.
 7. Otros Requisitos.
4. Apéndices.

Estándar IEEE 830

1. Introducción.

1. **Propósito.** Define el propósito y la audiencia objetivo del SRS.
2. **Ámbito del Sistema.** Identificación del sistema y su alcance: objetivos, tareas y beneficios.
3. **Definiciones, Acrónimos y Abreviaturas.** Glosario del vocabulario empleado.
4. **Referencias.** Listado de los documentos referenciados.
5. **Visión General del Documento.** Descripción del contenido y la organización del SRS.

Estándar IEEE 830

2. Descripción General.

1. **Perspectiva del Producto.** Relaciones del sistema con otros sistemas; identificación de interfaces entre ellos.
2. **Funciones del Producto.** Resumen organizado de las funcionalidades principales del sistema.
3. **Características de los Usuarios.** Descripción de los atributos y habilidades de los usuarios (nivel educativo, experiencia laboral y técnica...)
4. **Restricciones.** Limitaciones impuestas sobre el producto: políticas de empresa, limitaciones de hardware, interfaces externas, funciones de auditoría y control, protocolos de comunicación, lenguajes de programación...
5. **Suposiciones y Dependencias.** Factores que, si cambian, pueden afectar a los requisitos (p.e. estructura organizativa de la empresa).
6. **Requisitos Futuros.** Planteamiento de futuras mejoras para el sistema.

Estándar IEEE 830

3. Requisitos Específicos.

1. **Interfaces Externas.** Requisitos que afectan a la interfaz de usuario, interfaces hardware/software con otros sistemas e interfaces de comunicaciones.
2. **Funciones.** Requisitos funcionales organizados por objetivos, usuarios, objetos, estímulos, jerarquía funcional...
3. **Requisitos de Rendimiento.** Requisitos que afectan a la carga esperada por el sistema: nº de usuarios concurrentes, transacciones por segundo...
4. **Requisitos lógicos de BD.**
5. **Restricciones de Diseño.** Impuestas por estándares, limitaciones del hardware...
6. **Atributos del Sistema.** Atributos de calidad.
7. **Otros Requisitos.** Todos aquellos que no se categoricen en lo anterior.

4. **Apéndices.** Información de relevancia que no corresponde a alguna de las categorías previas.

Estándar IEEE 830

► El estándar también define unas **cualidades** deseables para el SRS:

- **Corrección.** Todos los requisitos se **llevarán a cabo**.
- **No ambigüedad.** Los requisitos tienen una **única interpretación**.
- **Compleitud.** Todos los requisitos **relevantes** se encuentran en el SRS.
- **Consistencia.** No existen **conflictos** entre los propios requisitos o con otros docs.
- **Clasificación.** Según la **importancia** y/o estabilidad de los requisitos. *Prioriz*
- **Verificabilidad.** Existe un **proceso** finito y eficiente para **comprobar** que el software cumple los requisitos.
- **Modificabilidad.** Los requisitos **se pueden cambiar** de un modo fácil, completo y sin incurrir en inconsistencias.
- **Trazabilidad.** Los requisitos **se pueden referenciar** con facilidad.

Comp
Corr
Clas

No Amb
Verif
Traz

Cons
Modif

Comp
Corr
Cons

No Amb
Org
Traz

Estándar ISO/IEC/IEEE 29148

1. Introducción.
 1. Propósito.
 2. Ámbito del Sistema.
 3. Visión General del Producto.
 1. Perspectiva del Producto.
 2. Funciones del Producto.
 3. Características de los Usuarios.
 4. Limitaciones.
 4. Definiciones.
2. Referencias.
3. Requisitos Específicos.
 1. Interfaces Externas.
 2. Funciones.
 3. Requisitos de Usabilidad.
 4. Requisitos de Rendimiento.
 5. Requisitos lógicos de BD.
 6. Restricciones de Diseño.
 7. Atributos del Sistema.
 8. Información complementaria.
4. Verificación.
5. Apéndices.

Escritura de Requisitos

Identificadores

- Podemos emplear los siguientes tipos de identificadores:
 - **Numeración secuencial.** Asignar un nº único a cada requisito.
 - Se emplea un prefijo que indica el tipo de requisito: RF-33.
 - Por sí solo no provee orden ni jerarquía alguna.
 - No se reusan los números.
 - **Numeración jerárquica.** La más habitual, empleando el formato x.[x.]* (2.8.8.)
 - El nº de niveles puede volverse excesivo.
 - Cambios en los requisitos pueden alterar la numeración de forma significativa.
 - Combinarlo con los números secuenciales suele ser práctico. (RF-SOL-2.8.8)
 - **Jerarquía textual.** Cada requisito recibe una etiqueta textual; se identifica como la combinación de su etiqueta y la de sus antecesores.
 - Producto.
 - Carrito.
 - Error. => Este requisito se identifica como Producto.Carrito.Error.

Cualidades deseables de los requisitos

- Según BABOK (IIBA):
 - **Atómicos.** Autocontenidos e independientes.
 - **Completos.** Cada requisito contiene toda la información necesaria para su implementación y comprensión.
 - **Comprensibles.**
 - **Consistentes.**
 - **Factibles.** Implementables dentro de las características del sistema y las restricciones del proyecto.
 - **No ambiguos.**
 - **Priorizados.**
 - **Verificables.**

Cualidades deseables de los requisitos

- Según Hull et al. [*Requirements Engineering 2nd Ed*]:
 - **Abstractos.** Independientes del diseño.
 - **Atómicos.**
 - **Claros.**
 - **Factibles.**
 - **Legales.**
 - **Precisos.**
 - **Únicos.**
 - **Verificables.**

Cualidades deseables de los requisitos

➤ Según Wieggers & Beatty:

➤ **Completos.**

➤ **Correctos.** Describen con precisión capacidades necesarias para los stakeholders.

➤ **Consistentes.**

➤ **Factibles.**

➤ **Necesarios.** Aportan valor al producto.

➤ **No ambiguos.**

➤ **Priorizados.**

➤ **Verificables.**

Estructura de un requisito

- Se pueden escribir requisitos desde la **perspectiva** del sistema o del usuario (voz activa).
 - Emplear fórmulas consistentes: el sistema/usuario deberá/podrá...
- Mavin et al. proponen la siguiente plantilla para la perspectiva del **sistema**:
 - [Precondición opcional] [trigger opcional] el sistema [Verbo][respuesta sistema]
 - *Si la miniatura buscada se encuentra en el catálogo, el sistema creará una solicitud de impresión.*
- Alexander y Stevens proponen lo correspondiente para el **usuario**:
 - El [clase de usuario/actor] podrá [hacer algo][a algo][condiciones exigidas, tiempo de respuesta o declaración de calidad].
 - *El Impresor podrá reimprimir cualquier miniatura impresa en el pasado mediante la consulta de un historial de impresiones.*

El Verbo

- En inglés, el verbo principal a emplear en la construcción de requisitos es **shall**.
 - *The system shall display a list of...*
 - Si bien similares, existen diferencias sutiles en la cultura anglosajona entre *shall* y *will*; este último puede ser interpretado como jurídicamente vinculante, así que es preferible evitarlo.
- Una traducción directa en **español** sería "deberá", pero podríamos limitarnos a emplear un **futuro simple** de indicativo.
 - *El sistema mostrará una lista de...*
- Se debe evitar a toda costa el uso de **condicionales** (debería, podría).
 - Sustituid "debería" por "probablemente no". El requisito resultante probablemente no sea aceptable.

Completitud

- Comprobar la **simetría** de las operaciones.
 - En otras palabras: ¿una funcionalidad implica la existencia de otras complementarias?
 - *El usuario podrá guardar la solicitud en cualquier momento durante su creación.*
- Si existe una lógica compleja en algún paso, deben cubrirse todas las **combinaciones** posibles.
 - Si es la primera solicitud del usuario y hay más de una unidad en el stock de la miniatura deseada, se incluirá una unidad más de regalo.
- Recordad considerar todas las posibles entradas y errores para cubrir las respuestas del sistema a las **excepciones**.

Compleitud

- El nivel de **detalle** necesario para considerar un requisito **completo** varía.
- Según Wiegers, es necesario ser más específicos si:
 - El cliente es externo.
 - La implementación se externaliza.
 - Se emplearán pruebas basadas en requisitos (RBT).
 - Se requieren estimaciones de costes precisas.
- El nivel de detalle puede ser menor si:
 - Los desarrolladores son expertos en el dominio.
 - Existen precedentes directos.

Concisión

- Si un requisito incluye las siguientes **palabras**, probablemente no sea conciso:
 - Y, o, además, también... (**conjunciones**, en general)
 - Excepto, a menos que, pero... (no sólo no son concisos, sino que tal vez falte información)
- *El sistema cargará el pago a la tarjeta de crédito registrada por el cliente, excepto si no está activa.*
 - ¿Y qué ocurre si no lo está?
- En estos casos probablemente se pueda **dividir** en dos o más requisitos.

No ambigüedad

- Evitar el uso de construcciones **X/Y**.
 - ...enviará información al Departamento de Ventas/Pedidos.
 - ¿Es el nombre completo? ¿Son sinónimos? ¿Son dos departamentos? ¿Se puede enviar a cualquiera de los dos, o debe ser a ambos?
- Los **rangos** numéricos deben ser claros:
 - El usuario podrá añadir hasta 3 archivos.
 - ¿El máximo son 2 o 3?
 - Añadir "inclusive/exclusive" o emplear términos más exactos, como:
 - El usuario podrá añadir 3 archivos o menos.

No ambigüedad

- Evitar **vocabulario** ambiguo, como los que se muestran a continuación y muchos de sus sinónimos.
 - Aceptable.
 - Al menos/no más de.
 - Dar soporte a.
 - Eficiente, rápido.
 - Entre X e Y, de X a Y.
 - Flexible.
 - Maximizar, mejorar, optimizar.
 - Normalmente, generalmente, idealmente.
 - Por ejemplo, tal como.
 - Opcionalmente, si es posible.
 - Robusto.
 - Suficiente.
 - Transparente.
 - Usable, *user-friendly*.
 - Varios, múltiples.
 - Y/o.

No ambigüedad

- Evitar requisitos **negativos**.
 - *El sistema impedirá al usuario solicitar una miniatura que no esté disponible.*
 - *Los usuarios que reciban el servicio sin estar autenticados no deberían generar registros contables.*
 - *Los registros no deberían fallar la validación si los atributos no están presentes.*
- Estas construcciones con dobles o más negaciones resultan confusas.
 - Reformular positivamente.

Ejemplos

- El sistema funcionará al máximo de su capacidad en todo momento, excepto en situaciones de emergencia, en las que podrá proporcionar hasta un 125% de su capacidad, a menos que la situación de emergencia se prolongue durante más de 15 minutos, en cuyo caso la capacidad se reducirá al 105%, pero en caso de que sólo se pueda alcanzar el 95%, el sistema activará una excepción de capacidad reducida y mantendrá la capacidad dentro del 10% de los valores indicados durante un mínimo de 30 minutos.

Ejemplos reales citados en Requirements Engineering 2nd Ed. de Hull et al.

Ejemplos

- El sistema proporcionará funcionalidades generales de procesamiento de textos que serán fáciles de usar por personal sin formación y funcionará en una red de área local Ethernet delgada conectada a los conductos aéreos con tarjetas de interfaz integradas alojadas en cada sistema junto con memoria adicional si fuera necesario.

Ejemplos reales citados en Requirements Engineering 2nd Ed. de Hull et al.

Ejemplos

- El sistema proveerá mensajes de estado en intervalos regulares no inferiores a 60 segundos.

Ejemplos reales citados en Software Requirements Engineering 3rd Ed.

Ejemplos

- El sistema debe comprobar si existen incoherencias en los datos de las cuentas entre el registro de cuentas activas y el archivo del Administrador de cuentas. La lógica que se utiliza para generar estas comparaciones debería basarse en la lógica de la herramienta de comprobación de coherencia existente. En otras palabras, el nuevo código no necesita desarrollarse desde cero. Los desarrolladores deberían utilizar el código actual del verificador de consistencia como base. Sin embargo, debe añadirse lógica adicional para identificar qué base de datos es la fuente autorizada. La nueva funcionalidad incluirá la escritura de datos en tablas de retención para indicar cómo/dónde resolver inconsistencias. Además, el código también debería comprobar los escenarios de excepción con la base de datos de herramientas de seguridad. Se enviarán alertas automáticas por correo electrónico al equipo de cumplimiento de la seguridad siempre que se detecten discrepancias.

Ejemplos reales citados en Software Requirements Engineering 3rd Ed.

Especificaciones formales

Especificación formal

- Lenguajes de base **matemática** que pueden utilizarse tanto **para describir** un sistema como para analizar su **comportamiento**, posiblemente verificando propiedades clave de interés. *[Hierons et al.]*
- Sus **ventajas** residen en:
 - **Definición no ambigua** del sistema, con mayor facilidad de detección de omisiones y errores.
 - **Facilita la implementación** (podría emplearse generación de código) y las pruebas.
- Su uso se suele **limitar a sistemas particularmente críticos**, ya que:
 - Son **costosos** de definir y difíciles de comprender.
 - Tienen un **alcance limitado**; p.e. no son apropiados para describir sistemas interactivos.

Especificación formal

- Las principales **técnicas** son las siguientes:
 - **Basado en modelos.** Se construye un modelo del sistema en el que se describen sus posibles estados y las operaciones que los cambian. (Z, VDM, B)
 - Existen variantes que limitan el n° de estados (basados en estados finitos) como FSM.
 - **Lenguajes algebraicos.** Describen el comportamiento de un sistema en función de axiomas que caracterizan sus propiedades deseadas. (OBJ, CASL)
 - **Híbridos.** Combinan características de ambos, como los lenguajes basados en estados y álgebra de procesos (CCS, LOTOS).

Nota: es posible probar la corrección de un sistema respecto a una especificación formal (verificación formal), pero naturalmente, no podemos garantizar la corrección de la propia especificación...

Especificación formal

Basado en modelos (Z)	Algebraico (CASL)
<i>Pop</i> $\Delta Stack$ $x! : Object$ $items \neq ()$ $Items' = tail\ items$ $x! = head\ items$	spec $Bank = Nat$ then sort $account$ ops $empty : account$ $credit : Nat * account \rightarrow account;$ $debit : Nat * account \rightarrow account;$ $balance : account \rightarrow Nat$ vars $n : Nat; acc : account$ axioms $balance(empty) = 0;$ $balance(credit(n, acc)) = balance(acc) +$ $n;$ $balance(debit(n, acc)) =$ $balance(acc) - n;$ $balance(acc) < n \Rightarrow balance(debit(n, acc)) = 0$ end

Herramientas de Gestión de Requisitos

HGR

- En el mercado existe una variedad de **herramientas de gestión de requisitos.**
 - *Jama Connect, ReqSuite...*
- **Ventajas** frente al uso de documentos tradicionales:
 - **Gestión integrada de cambios**, versiones y estados.
 - Los requisitos constan de atributos adicionales (p.e. fecha de creación).
 - Definición de relaciones entre requisitos y otros requisitos, sistemas, componentes...
 - Facilita sopesar el impacto de los cambios en los requisitos.
 - Facilita la identificación de requisitos omitidos.
 - **Facilita el reuso de requisitos.**
 - Mayores capacidades para gestionar e informar a los stakeholders.

HGR

- Estas herramientas **facilitan** la elaboración de un SRS...
 - ... pero **no** resuelven deficiencias en el proceso de IR.
- Esto es, resultan inútiles si nuestros problemas residen en:
 - La obtención de requisitos.
 - El análisis de requisitos.
 - La escritura de requisitos que cumplan las características deseables.



Universidad de Oviedo

Tema 6

Validación de Requisitos

1

Jorge Álvarez Fidalgo

Validación

Consiste en **comprobar** que los **requisitos** obtenidos **son adecuados** para construir un producto que satisfaga los requisitos de negocio:

- **Revisar** la documentación en busca de errores:
 - Los requisitos cumplen los atributos de calidad deseables.
 - Los requisitos reflejan las necesidades del cliente y los usuarios.

Los requisitos perfectos son una quimera. El objetivo es reunir un conocimiento suficiente de los requisitos que nos permita elaborar el producto con un nivel aceptable de riesgo.

Validación & verificación

- Valid $\Rightarrow$ necesidades
- Verif $\Rightarrow$ calidad

- Conceptos inseparables pero **distintos**.
 - Por sencillez, se suelen emplear como un **conjunto**.
- **Validación**: determinar si los requisitos satisfacen las **necesidades** de los stakeholders.
 - Requiere conocimiento del dominio y del problema.
- **Verificación**: determinar si los requisitos satisfacen los **criterios de calidad**.
 - Requiere conocimiento de IR, del lenguaje y la lógica.
- Sigue un orden teórico: primero la verificación, después la validación.
 - A la hora de la verdad, ambos conceptos se entrelazan.
- "Hacer lo correcto vs hacerlo correctamente".

Criteria de calidad

- Los criterios de calidad/características deseables para los requisitos fueron descritos en el Tema 5.
 - Puesto que ya deberían considerarse durante la especificación de los requisitos.
- No todos los requisitos habrán de satisfacer los criterios de calidad al mismo nivel.
- Su aplicación variará según:
 - El contexto del proyecto.
 - La experiencia en el dominio.
 - Pragmatismo.
- *El sistema solicitará obligatoriamente un NIF o correo electrónico.*

Criterios de calidad

Por ejemplo, Katasonov y Sakkinen proponen el siguiente listado como condensación de distintas publicaciones:

Cada requisito **individual** debería ser:

- Completo.
- Conciso.
- Correcto.
- No ambiguo.
- Factible.
- Necesario.
- Priorizado.
- Verificable.

El **SRS** en su conjunto debería ser:

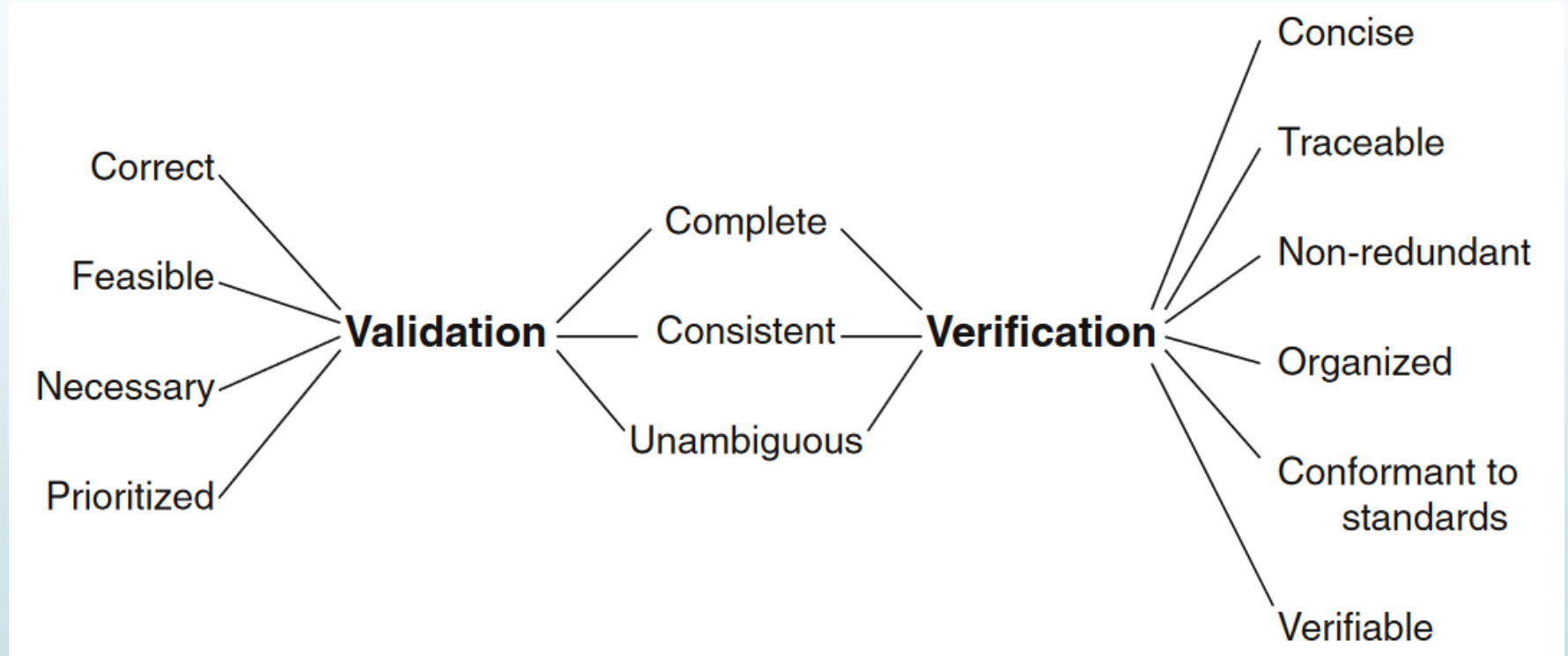
- Completo.
- Consistente.
- Conforme a los estándares.
- No redundante.
- Organizado.
- Trazable.

CIP
CCN
CFV

CAN
CCO
CCT



Criteria de calidad



[Katasonov y Sakkinen, 2006]

Técnicas de V&V

- Las **técnicas** aquí definidas **suelen estar asociadas a la validación**, si bien pueden ayudar a los objetivos de verificación:
 - **Revisiones**. *De particular importancia para la verificación.*
 - **Generación de casos de prueba.**
 - **Prototipado.**
 - **Criterios de aceptación.**

Revisiones

- **Inspección** del SRS con el objeto de encontrar defectos.

- Pueden ser:

- **Formales.** Revisión estructurada por parte de un equipo preestablecido.

- **Informales.** Revisión rápida por parte de uno o varias personas (*peers*).



- *Nota: es recomendable realizar una pre-revisión rápida que detecte errores evidentes.*

Revisiones formales

- Se forma un **equipo de revisores** que aúne **distintas perspectivas**.
 - **Autor/es del SRS** y otros ingenieros de requisitos.
 - Aunque no hayan participado en el documento, otros IR sabrán qué errores buscar.
 - **Fuentes de información**.
 - Tales como **representantes del cliente** o de los usuarios finales, o autores de una especificación previa.
 - **Equipo de desarrollo** que se basará en el SRS.
 - Desarrolladores, jefe de proyecto, testers...
- Como ocurría en la obtención, el tamaño del equipo debe ser limitado (7-).

Revisiones formales

- Consta de las siguientes **fases**:
 - **Planificación**. **Creación del equipo** y planificación temporal. El tiempo dedicado afectará a la tasa de defectos encontrados. introducción al contexto, distribución del SRS...
 - **Preparación**. Introducción al contexto y **distribución** de materiales. Los revisores (excepto autores) examinan el documento individualmente para determinar defectos y problemas.
 - **Reunión de inspección**. El equipo se reúne para **analizar los resultados** del examen individual y acordar acciones a tomar. Se decide si se acepta el documento, se acepta con revisiones menores, o debe recibir una gran revisión.
 - **Reelaboración**. **Arreglo de defectos**.
 - **Seguimiento**. **Notificación/inspección de las correcciones** realizadas por parte de un individuo designado.

Revisiones: técnicas

- Aplicables tanto a revisiones formales como informales.
- Son las siguientes:
 - **Ad-hoc.** Sin guía alguna; se basa en el conocimiento y experiencia del revisor.
 - **Checklists.** Lista de preguntas a resolver que formulan cuestiones de relevancia.
 - **Basado en defectos.** Procedimientos diseñados para encontrar defectos comunes.
 - **Basado en perspectivas.** Procedimientos similares al anterior, pero desde la perspectiva de los stakeholders.
 - **Basado en escenarios.** Se definen situaciones de interés para el sistema y se examina la calidad de los requisitos que cubren dichas situaciones.

Revisiones: **checklists**

- Lo **más habitual** es emplear **checklists**.
 - Tanto para **la validación como la verificación**.
- Existen numerosas checklists que podemos emplear como referencia.
 - Algunas referencias de interés son Wiegers, Kotonya & Sommerville o Pressman.
 - **Enfocadas, sobre todo, a la verificación**; reformulan los criterios de calidad de los requisitos en una o varias preguntas para su comprobación.

Revisiones: checklists

Inventar

- ¿Hay algún requisito que entre en conflicto con otros o los duplique?
- ¿Está redactado cada requisito en un lenguaje claro, conciso y sin ambigüedades?
- ¿Se puede verificar cada requisito mediante pruebas, demostraciones, revisiones o análisis?
- ¿Cada requisito está dentro del alcance del proyecto?
- ¿No contiene errores gramaticales ni de contenido?
- ¿Falta información necesaria en algún requisito? Si es así, ¿se identifica como TBD?
- ¿Pueden aplicarse todos los requisitos dentro de las limitaciones conocidas?
- ¿Los mensajes de error especificados son únicos y tienen sentido?
- ¿Está identificado el origen del requisito (por ejemplo, persona, norma, documento)? ¿El planteamiento final del requisito ha sido contrastado con la fuente original?

Revisiones

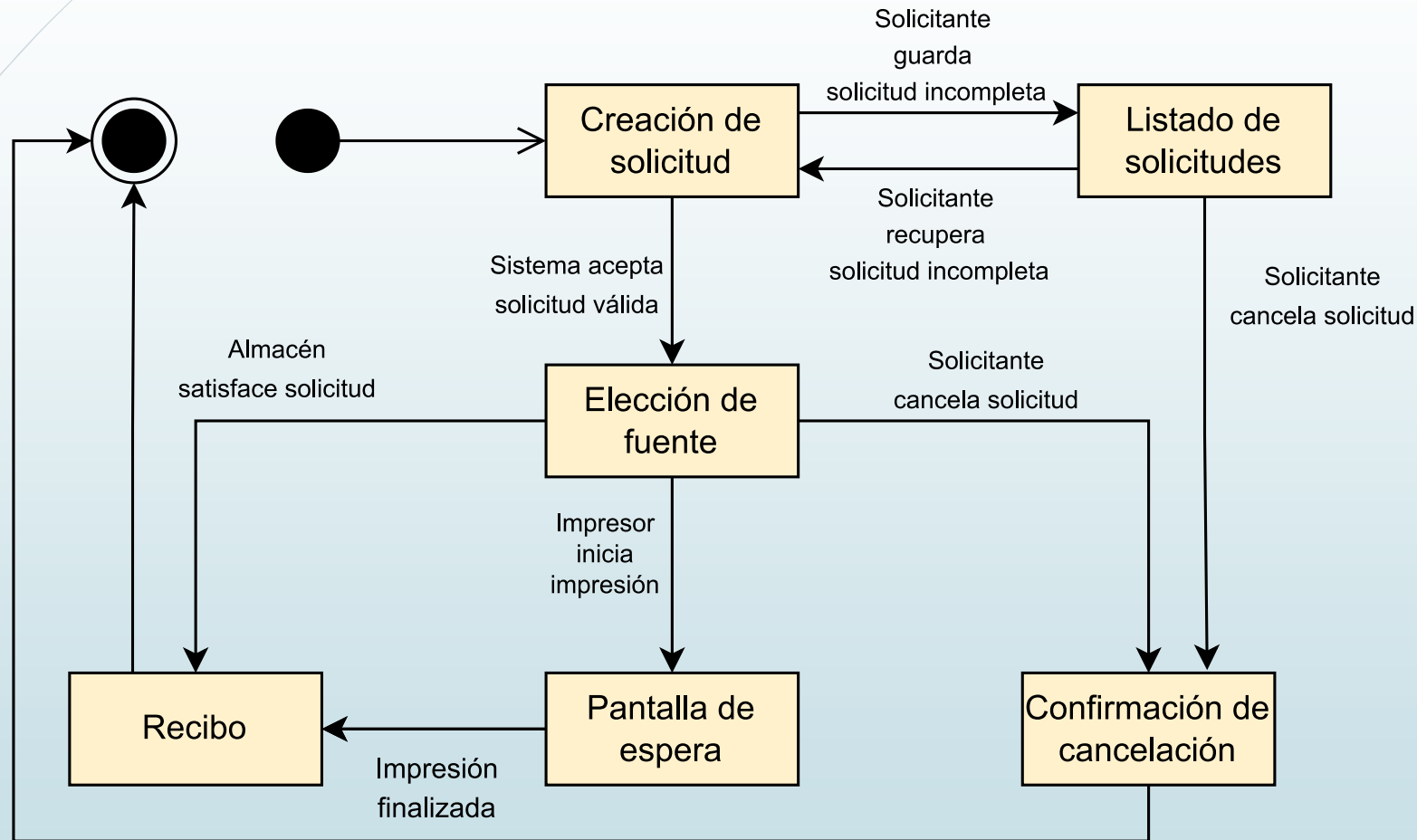
- El producto de las inspecciones será un **listado de defectos y acciones a tomar**.
- Un posible **formato** es el que sigue:

Requisito	Defecto	Acciones recomendadas
RF-SG-1.04.85.96	Incompleto, Ambiguo	Especificar las interfaces necesarias para conectarse con un IBM-5100.

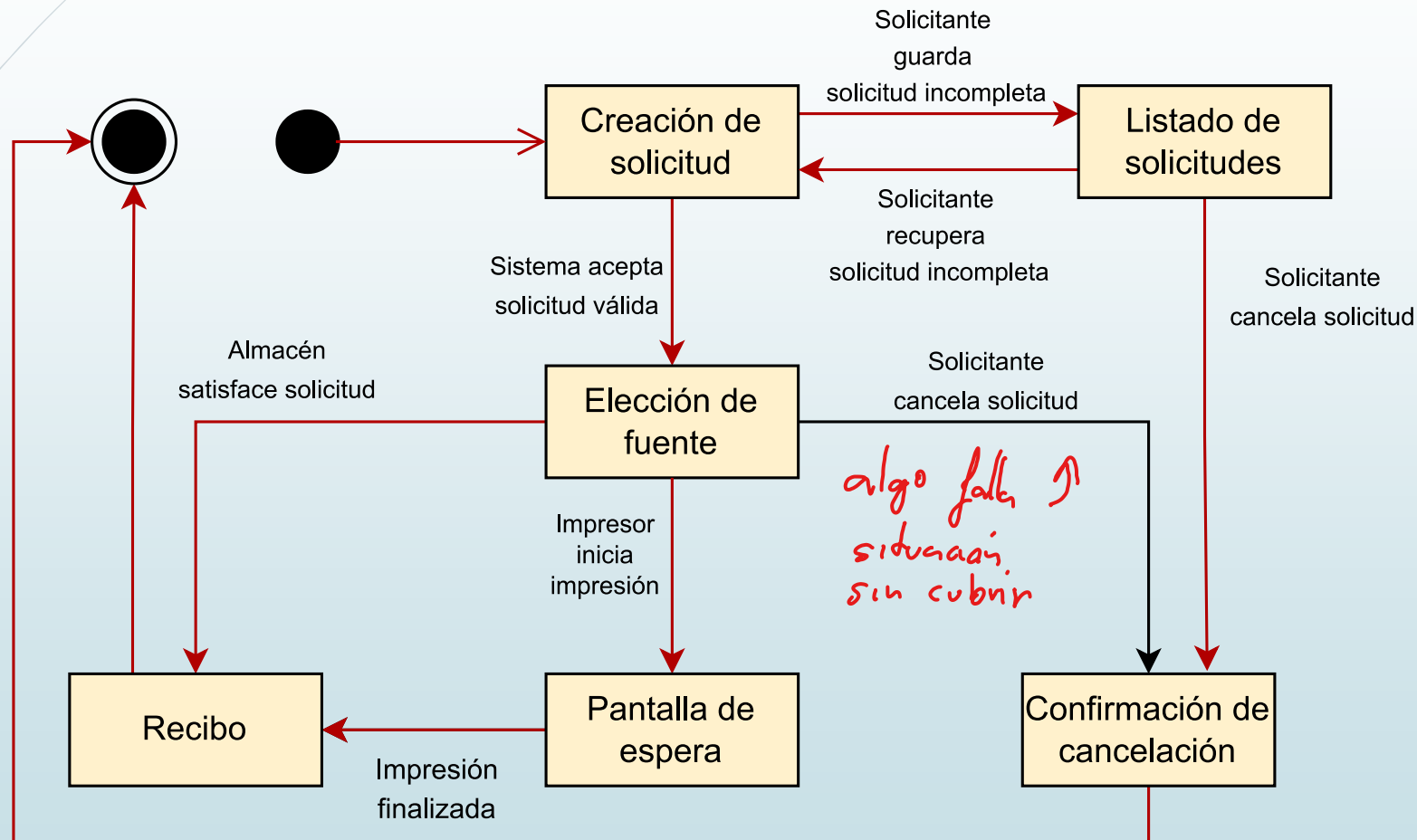
Generación de casos de prueba

- Definición de **casos de prueba conceptuales** con el objeto de evaluar requisitos funcionales y modelos.
 - Deberían cubrir todos los flujos identificados durante el análisis.
- La detección de **errores** u **omisiones** vendrá dada por:
 - **Requisitos** para los que **no podemos definir** pruebas (no verificables).
 - **Tras definir pruebas** para todos los requisitos, en los modelos elaborados durante el análisis aparecen **situaciones no probadas**.

Generación de casos de prueba



Generación de casos de prueba



Si, para cada prueba relacionada con la realización de una solicitud, trazamos sobre el diagrama cada camino recorrido, ¿cubrimos todas las posibilidades? ¿Qué puede estar fallando?

Generación de casos de prueba

- En un caso como el anterior, caben las siguientes posibilidades:
 - La situación no cubierta no es un comportamiento deseable del sistema. El **modelo es incorrecto** y debe ser corregido.
 - La situación es legítima, pero **no** se ha **definido** un caso de prueba para ello o se ha **omitido** el requisito que describe dicha situación.
 - El caso de prueba es **erróneo**.
- Se han detectado errores u omisiones en la especificación sin necesidad de implementar las pruebas.
 - Puede ser una base para las futuras implementaciones.

Prototipado

- Creación de una **versión simple** del software para identificar problemas y limitaciones.
- **Facilita la comunicación con los stakeholders no técnicos al hacer los requisitos tangibles.**
 - Estos detectan más fácilmente posibles errores y omisiones.
- El prototipo debe:
 - Ser simple.
 - Contener las principales funcionalidades e interacciones con el sistema.
 - Tener una IU similar a la definitiva.

Prototipado

- Podemos distinguir los siguientes tipos:
 - **Horizontales.** Representan la interfaz de usuario (opciones disponibles, estructura de navegación) pero sin funcionalidad alguna.
 - Si se muestran datos, deberían ser reales.
 - Cuestiones de diseño de IU no son relevantes (posición, fuentes, colores...)
 - **Verticales.** Implementan una funcionalidad concreta a todos los niveles, simulando el entorno de producción.
 - Útiles en casos concretos como verificar la viabilidad de una arquitectura/BD o probar tiempos de respuesta críticos.

Prototipado

- Otras clasificaciones de interés:

- **Desechables** (throwaway).

- Exploratorios, no formarán parte del producto final.
 - Favorecen la implementación rápida y la modificabilidad.
 - Útiles cuando resulta complicado visualizar el sistema a partir de los requisitos.

- **Evolutivos.**

- Base sólida para construir incrementalmente el producto según los requisitos evolucionan.

- Los apropiados para la validación son los prototipos desechables.

Prototipado

- Aún más clasificaciones de interés:

- **De baja fidelidad.**

- **Esbozos** en papel; rápidos y baratos.
 - Para explorar funcionalidad **y** flujos del proceso (**storyboard**).
 - Una persona puede simular la funcionalidad (mago de Oz).

- **De alta fidelidad.**

- Modelos **electrónicos**.
 - Para una visión más exacta del sistema y sus interfaces.
 - De nuevo, centrarse en la funcionalidad y las interacciones del usuario con el sistema, y no en cuestiones exactas de diseño.

Prototipado

Prototipos	Desechables
Horizontales	• Identificar errores y omisiones en los requisitos funcionales.
Verticales	• Comprobar viabilidad técnica. • Evaluar rendimiento.

Criterios de aceptación

- Evalúan si un producto **es** considerado **adecuado** para su uso en el entorno operativo por parte de sus **stakeholders**.
- Estableciendo paralelismos con fases previas:
 - Obtención: *¿qué necesita hacer el usuario con el sistema?*
 - Aceptación: *¿cómo juzgaría el usuario si el sistema satisface sus necesidades?*
- Complementariamente, podríamos definir "criterios de rechazo".

Criterios de aceptación

- Los criterios de aceptación **han de cubrir** aspectos como:
 - La **especificación** de funcionalidad de alta prioridad.
 - **Requisitos no** funcionales que deban estar presentes.
 - Restricciones **externas** a satisfacer.
- **Las pruebas de aceptación** permiten determinar si un sistema satisface los **criterios de aceptación**.
 - *Deberían cubrir los escenarios más comunes y relevantes.*
- Al igual que ocurría con la generación de casos de prueba, su definición temprana puede arrojar luz sobre cuestiones problemáticas en la especificación de requisitos.

1 Requisitos a validar

La especificación presentada a continuación ha sido generada automáticamente bajo orden expresa de respetar los atributos deseables de los requisitos (no lo hace). El sistema a representar es un videojuego; para ser exactos, un RPG post-apocalíptico de mundo abierto.

Este documento se encuentra plagado de problemas, unos omnipresentes y otros más dignos de mención, que he marcado con un asterisco.

1.1 Jugabilidad

***RF-JUG-1.1:** El juego permitirá al jugador controlar el movimiento del personaje principal utilizando las teclas WASD en el teclado o el joystick en un control de juego.

RF-JUG-1.2: El personaje se moverá a una velocidad realista que se ajustará según las estadísticas del personaje y el terreno en el juego.

***RF-JUG-1.3:** El movimiento será fluido y no presentará retardos notables en la respuesta a las acciones del jugador.

RF-JUG-2.1: El juego ofrecerá una perspectiva en tercera persona que permitirá al jugador observar el personaje desde diferentes ángulos mediante un control de la cámara.

RF-JUG-2.2: El juego permitirá a los jugadores alternar entre la perspectiva en tercera persona y la primera persona de manera suave y sin problemas.

RF-JUG-2.3: La cámara no debe causar mareos ni distorsiones incómodas en la vista del jugador.

RF-JUG-3.1: El juego permitirá al jugador interactuar con objetos y NPCs a través de una tecla de acción designada, que resaltará los elementos interactivos en el entorno.

***RF-JUG-3.2:** La interacción con objetos, como abrir puertas, recoger objetos y saquear contenedores, debe ser intuitiva y de respuesta rápida.

RF-JUG-3.3: Las acciones de interacción, como hablar con NPCs, deben presentar opciones de diálogo claras y permitir al jugador avanzar o retroceder en la conversación según sus elecciones.

RF-JUG-4.1: El juego ofrecerá un sistema de combate en tiempo real que permitirá al jugador disparar, golpear y utilizar habilidades especiales durante los enfrentamientos.

***RF-JUG-4.2:** El sistema de combate en tiempo real permitirá al jugador apuntar a partes específicas de los enemigos para infligir daño adicional.

***RF-JUG-4.3:** El juego también proporcionará un sistema de combate por turnos con una interfaz que permitirá al jugador seleccionar y ejecutar acciones en su turno.

RF-JUG-5.1: El sistema de combate permitirá a los jugadores utilizar cobertura y elementos del entorno para protegerse de los ataques enemigos.

RF-JUG-5.2: Los jugadores podrán activar habilidades especiales durante el combate que se basen en las estadísticas y habilidades de su personaje.

***RF-JUG-5.3:** Los enemigos responderán de manera coherente a las tácticas de combate del jugador, como buscar cobertura o flanquear al jugador cuando sea necesario.

RF-JUG-6.1: Los enemigos se comportarán de acuerdo con su naturaleza y la lógica del juego, lo que incluirá patrullas, ataques basados en la detección del jugador y reacciones realistas a las acciones del jugador.

RF-JUG-6.2: Los enemigos tendrán estadísticas y atributos individuales que influirán en su fuerza, resistencia y habilidades de combate.

***RF-JUG-7.1:** La cámara seguirá de cerca al personaje del jugador durante el combate en tiempo real, manteniendo al enemigo en el centro de la pantalla.

***RF-JUG-7.2:** La cámara permitirá una visión completa del área de combate, lo que facilitará la planificación de tácticas y estrategias.

1.2 Narrativa

RF-NAR-1.1: El juego presentará una trama principal con múltiples ramificaciones y finales que se basarán en las decisiones y acciones del jugador.

RF-NAR-1.2: Las decisiones tomadas por el jugador afectarán la dirección y el desenlace de la trama principal, lo que permitirá al jugador influir significativamente en la historia del juego.

RF-NAR-2.1: Debe haber una variedad de misiones secundarias y actividades opcionales disponibles en el juego, que enriquecerán la experiencia del jugador.

RF-NAR-2.2: Estas misiones secundarias y actividades opcionales se activarán a través de interacciones con NPCs y objetos en el mundo del juego.

RF-NAR-2.3: Las misiones secundarias deben contribuir al desarrollo del mundo y los personajes, brindando oportunidades para explorar historias adicionales y desafíos.

RF-NAR-3.1: El juego ofrecerá diálogos con personajes no jugables (NPCs) que estarán completamente doblados y subtitrados.

RF-NAR-3.2: Debe haber opciones de conversación que permitan al jugador seleccionar respuestas y tomar decisiones que influirán en la trama y las relaciones con otros personajes.

***RF-NAR-3.3:** Los diálogos y las opciones de conversación deben ser claros y no ambiguos, lo que garantizará que las elecciones del jugador tengan un impacto predecible en la historia.

RF-NAR-4.1: La historia y los eventos del juego deben ser coherentes con el mundo post-apocalíptico de X y la línea de tiempo establecida en la serie.

RF-NAR-4.2: Las acciones de los NPCs y los resultados de las misiones deben reflejar la lógica y el contexto del mundo del juego.

RF-NAR-5.1: El juego incluirá escenas de corte y secuencias de video que contribuirán a la narrativa y proporcionarán información adicional sobre la historia y los personajes.

RF-NAR-5.2: Estas escenas de corte deben mantener un alto nivel de calidad visual y estar sincronizadas con el audio para una experiencia de juego inmersiva.

***RF-NAR-6.1:** Los guiones y diálogos del juego deben ser escritos de manera profesional y coherente con la calidad esperada en una producción de videojuegos AAA.

***RF-NAR-6.2:** Los diálogos con voz deben estar en consonancia con las características y las personalidades de los personajes, lo que garantizará una representación auténtica.

RF-NAR-7.1: El juego permitirá que los NPCs respondan de manera contextual a las acciones y elecciones del jugador, lo que enriquecerá la inmersión y la coherencia narrativa.

RF-NAR-7.2: Las respuestas de los NPCs deben reflejar de manera realista las consecuencias de las decisiones del jugador en el mundo del juego.

1.3 Mundo y entorno

RF-MUN-1.1: El mundo del juego estará compuesto por áreas diferentes y reconocibles, como desiertos, ciudades en ruinas, áreas subterráneas y paisajes únicos que reflejen la estética post-apocalíptica de la serie X.

RF-MUN-2.1: El juego incluirá ciclos realistas de día y noche con transiciones suaves que afectarán la visibilidad y el comportamiento de los NPCs.

RF-MUN-2.2: Los cambios climáticos, como lluvia, tormentas de arena y neblina, serán parte del mundo del juego, agregando variedad y desafíos visuales y tácticos.

RF-MUN-3.1: Deben ocurrir eventos aleatorios en el mundo del juego, como encuentros con criaturas hostiles, bandidos, o situaciones de conflicto entre facciones.

RF-MUN-3.2: Estos eventos aleatorios deben ser generados de manera lógica y contextual, lo que garantizará que encajen en el mundo del juego y sean coherentes con la experiencia del jugador.

RF-MUN-4.1: El mundo debe estar lleno de detalles y elementos que refuercen la ambientación post-apocalíptica, como ruinas de edificios, vehículos abandonados y objetos que cuenten historias del pasado.

RF-MUN-4.2: La calidad gráfica y los detalles del entorno deben ser lo suficientemente altos para sumergir al jugador en un mundo creíble y vívido.

RF-MUN-5.1: Los jugadores deberán poder explorar libremente el mundo sin restricciones artificiales, lo que fomentará la sensación de libertad y descubrimiento.

RF-MUN-5.2: Se proporcionará un mapa detallado del mundo y un sistema de navegación que ayudará a los jugadores a orientarse y planificar sus viajes.

RF-MUN-6.1: El mundo del juego responderá a las acciones del jugador, incluyendo la reacción de los NPCs a sus decisiones y la posibilidad de cambiar el estado del mundo mediante acciones importantes.

RF-MUN-6.2: Las decisiones del jugador tendrán un impacto medible en las facciones, las relaciones con los NPCs y la disponibilidad de misiones.

RF-MUN-7.1: Debe haber sonidos ambientales realistas que reflejen la atmósfera del mundo post-apocalíptico, como el viento silbante, el crujir de la arena y los sonidos de la vida silvestre.

RF-MUN-7.2: Los efectos de sonido, como disparos de armas de fuego y explosiones, deben ser precisos y proporcionar una experiencia auditiva inmersiva.

1.4 Interacción con personajes

RF-INT-1.1: El juego debe presentar NPCs con una variedad de personalidades que incluyan héroes, villanos, comerciantes, científicos, y otros arquetipos coherentes con el mundo post-apocalíptico de X.

RF-INT-1.2: Cada NPC debe comportarse de acuerdo con su personalidad, lo que incluirá diálogos, actitudes y acciones apropiadas.

RF-INT-2.1: El juego ofrecerá diálogos interactivos con NPCs que permitirán a los jugadores seleccionar respuestas y tomar decisiones.

RF-INT-2.2: Las respuestas y elecciones del jugador afectarán la relación con los NPCs y las facciones, lo que influirá en la trama y las misiones disponibles.

RF-INT-3.1: Los NPCs deben reaccionar de manera contextual a las acciones y elecciones del jugador, lo que incluirá emociones, cambios en la actitud y cambios en la disponibilidad de misiones.

RF-INT-3.2: Las reacciones de los NPCs deben ser consistentes con la lógica del mundo del juego y la personalidad de cada personaje.

RF-INT-4.1: Los NPCs deben realizar una variedad de acciones y comportamientos en el mundo del juego, como trabajar, descansar, patrullar, comerciar, y participar en actividades específicas de su facción.

RF-INT-4.2: Los comportamientos de los NPCs deben ser creíbles y contribuir a la inmersión en el mundo del juego.

RF-INT-5.1: Los NPCs deben pertenecer a diferentes facciones dentro del juego, cada una con sus objetivos, creencias y relaciones con otras facciones.

RF-INT-5.2: Las interacciones del jugador con los NPCs y las facciones deben influir en su reputación y posición en el mundo del juego.

RF-INT-6.1: Los diálogos con los NPCs deben estar completamente doblados y subtitrados en el idioma del juego.

***RF-INT-6.2:** La calidad del doblaje y las actuaciones de voz deben ser profesionales y estar en consonancia con las personalidades de los personajes.

RF-INT-7.1: El juego debe mantener un registro de las decisiones tomadas por el jugador en las interacciones con NPCs, lo que permitirá un seguimiento de las consecuencias a lo largo de la partida.

RF-INT-7.2: Debe proporcionarse una interfaz que permita a los jugadores consultar su reputación con facciones y personajes específicos en cualquier momento.

1.5 Personalización

RF-PER-1.1: El juego permitirá a los jugadores crear su personaje al inicio del juego, incluyendo la elección del género, nombre, apariencia facial, color de cabello, y rasgos faciales.

RF-PER-1.2: Debe existir una opción para personalizar el pasado y la historia de fondo del personaje, lo que influirá en las estadísticas iniciales y las opciones de diálogo.

RF-PER-2.1: Los jugadores podrán asignar puntos de habilidad a estadísticas como fuerza, inteligencia, destreza, percepción, carisma y resistencia en el momento de la creación del personaje.

***RF-PER-2.2:** La asignación de puntos de habilidad afectará las capacidades iniciales del personaje, como su salud, capacidad de carga, capacidad de persuasión y precisión en el combate.

RF-PER-3.1: El juego permitirá a los jugadores seleccionar una habilidad especial en el momento de la creación del personaje, como "Puntería Mortal" o "Cazador Nato."

RF-PER-3.2: Las habilidades especiales proporcionarán ventajas y desventajas específicas para el personaje, lo que influirá en su estilo de juego y en las opciones estratégicas.

RF-PER-4.1: Los jugadores podrán equipar a su personaje con una variedad de prendas y armas que encontrarán en el mundo del juego.

RF-PER-4.2: Deben existir opciones para personalizar la ropa y el equipo del personaje, lo que permitirá una apariencia única.

***RF-PER-5.1:** El juego debe permitir a los jugadores cambiar la apariencia de su personaje en cualquier momento, lo que incluye opciones para peinados, barbas, tatuajes y más.

RF-PER-5.2: Los cambios de apariencia deben ser visibles en el modelo de personaje en el juego.

RF-PER-6.1: El juego mantendrá registros de las elecciones de personalización realizadas por el jugador, lo que permitirá realizar cambios futuros en el personaje de manera coherente.

RF-PER-6.2: Los registros de personalización también afectarán la forma en que los NPCs y las facciones perciben al personaje del jugador.

1.6 Combate

RF-COM-1.1: El juego proporcionará una variedad de armas, que incluirá pistolas, rifles, armas cuerpo a cuerpo y armas de energía, que los jugadores podrán adquirir y equipar.

RF-COM-1.2: Cada arma tendrá estadísticas específicas, como daño, precisión y velocidad de recarga, que afectarán su eficacia en el combate.

RF-COM-2.1: Debe existir un sistema de inventario que permita a los jugadores gestionar su equipo, incluyendo la visualización de objetos, transferencia de objetos entre el inventario y contenedores, y la gestión de peso y capacidad de carga.

RF-COM-2.2: Los jugadores podrán organizar su inventario mediante categorías y etiquetas para facilitar la búsqueda de objetos.

RF-COM-3.1: El juego permitirá a los jugadores comerciar con NPCs y adquirir objetos a través de la compra y venta de bienes.

RF-COM-3.2: Los jugadores podrán saquear objetos y equipo de enemigos derrotados y contenedores en el mundo del juego.

***RF-COM-4.1:** El combate debe ser estratégico, permitiendo a los jugadores apuntar a partes específicas de los enemigos, como cabeza o extremidades, para infligir daño adicional.

RF-COM-4.2: Los jugadores podrán utilizar cobertura y objetos del entorno para protegerse de los ataques enemigos durante los enfrentamientos.

RF-COM-5.1: Los jugadores podrán activar habilidades especiales durante el combate, como el uso de poderes de armadura, técnicas de sigilo y ataques cuerpo a cuerpo mejorados.

RF-COM-5.2: Cada habilidad especial tendrá un tiempo de recarga y requisitos de recursos específicos.

RF-COM-6.1: Los jugadores deberán administrar su munición y consumibles, como comida, agua y medicamentos, para mantenerse preparados durante el combate y la exploración.

RF-COM-6.2: El juego proporcionará una interfaz que permitirá a los jugadores ver el inventario de munición y consumibles disponibles.

RF-COM-7.1: El juego incluirá un sistema de recarga para armas de fuego que requerirá una acción del jugador para cargar el arma después de agotar la munición.

RF-COM-7.2: La recarga debe ser una acción realista y tomar en cuenta el tipo de arma y la munición.

1.7 Experiencia y progresión

RF-EXP-1.1: El juego implementará un sistema de experiencia que permitirá a los jugadores ganar puntos de experiencia (EXP) al completar misiones, derrotar enemigos y realizar acciones significativas en el juego.

RF-EXP-1.2: Los puntos de experiencia acumulados se utilizarán para avanzar de nivel, y cada nivel alcanzado proporcionará puntos de habilidad adicionales y ventajas específicas.

RF-EXP-2.1: Los puntos de habilidad permitirán a los jugadores mejorar estadísticas específicas, como fuerza, inteligencia, destreza, percepción, carisma y resistencia.

RF-EXP-2.2: La asignación de puntos de habilidad al subir de nivel se realizará mediante una interfaz de selección y deberá ser coherente con el sistema de experiencia ganado.

***RF-EXP-3.1:** El juego permitirá a los jugadores seleccionar una habilidad especial (Perks) al subir de nivel.

RF-EXP-3.2: Cada habilidad especial proporcionará ventajas y desventajas únicas, lo que influirá en el estilo de juego y las decisiones tácticas.

RF-EXP-4.1: El juego incluirá un sistema de reputación y karma que rastreará las acciones y decisiones morales del jugador en el mundo del juego.

RF-EXP-4.2: Las acciones del jugador afectarán la reputación con facciones y la percepción de otros personajes en el juego, lo que influirá en las interacciones y misiones disponibles.

RF-EXP-5.1: Los jugadores podrán administrar su inventario y equipo de manera eficiente, lo que incluirá la gestión de armas, armaduras, consumibles y objetos.

RF-EXP-5.2: La capacidad de carga estará relacionada con las estadísticas del personaje y se verá afectada por el peso del equipo y objetos transportados.

RF-EXP-6.1: Los jugadores podrán mejorar sus habilidades a medida que avanzan de nivel, lo que incrementará la eficacia en áreas como el combate, la persuasión y la supervivencia.

RF-EXP-6.2: La mejora de habilidades estará ligada a las estadísticas y habilidades iniciales del personaje.

RF-EXP-7.1: La progresión de la historia se verá influenciada por las acciones y elecciones del jugador, lo que generará múltiples ramificaciones y finales posibles.

RF-EXP-7.2: Cada elección y consecuencia debe ser verificable a lo largo de la partida, y el juego proporcionará retroalimentación sobre las implicaciones de las decisiones tomadas.

1.8 Supervivencia

***RF-SUP-1.1:** El juego incluirá un sistema de hambre y sed que requerirá que los jugadores administren su ingesta de alimentos y agua para mantenerse saludables.

RF-SUP-1.2: El sistema de hambre y sed afectará las estadísticas del personaje, como la salud y la resistencia, y tendrá consecuencias en el juego si no se atienden las necesidades básicas.

***RF-SUP-2.1:** Los jugadores deberán satisfacer las necesidades básicas de su personaje, como dormir para descansar, mantenerse limpios y evitar la exposición a condiciones extremas.

RF-SUP-2.2: La negligencia en la atención de las necesidades básicas del personaje tendrá efectos negativos, como fatiga, enfermedad y disminución del rendimiento.

RF-SUP-3.1: El juego incluirá un sistema de enfermedades y radiación que hará que los jugadores sean susceptibles a infecciones y condiciones médicas si no toman precauciones.

RF-SUP-3.2: La radiación del entorno afectará la salud del personaje, y la exposición prolongada podría tener consecuencias graves.

RF-SUP-4.1: Los jugadores podrán administrar medicamentos y tratamientos, como antirradiación, antibióticos y vendajes, para mantener o restaurar la salud del personaje.

RF-SUP-4.2: Debe existir un sistema de inventario que permita a los jugadores acceder a medicamentos y aplicarlos según sea necesario.

RF-SUP-5.1: Los jugadores deberán lidiar con las condiciones climáticas extremas y los entornos hostiles, como tormentas de arena y temperaturas extremadamente altas o bajas.

RF-SUP-5.2: La supervivencia en estos entornos requerirá la adquisición de equipo específico, como trajes protectores y equipo de supervivencia.

RF-SUP-6.1: El juego proporcionará retroalimentación clara al jugador sobre las necesidades básicas, el estado de salud y la exposición a la radiación.

RF-SUP-6.2: Las notificaciones visuales y de audio informarán al jugador sobre el estado de su personaje y la necesidad de tomar medidas.

RF-SUP-7.1: Debe haber opciones de dificultad que permitan a los jugadores personalizar la intensidad de las mecánicas de supervivencia.

RF-SUP-7.2: Las opciones de dificultad afectarán la frecuencia y severidad de las necesidades básicas, enfermedades y radiación.

1.9 Misiones

RF-MIS-1.1: El juego proporcionará una variedad de misiones principales y secundarias que los jugadores podrán aceptar y completar.

RF-MIS-1.2: Las misiones estarán disponibles a través de NPCs, terminales de computadora y otros elementos en el mundo del juego.

RF-MIS-2.1: Cada misión deberá tener objetivos claros y específicos que se describan de manera concisa en el registro de misiones del jugador.

***RF-MIS-2.2:** Los objetivos deben ser comprensibles y no ambiguos, lo que permitirá a los jugadores entender qué deben lograr.

RF-MIS-3.1: El juego ofrecerá una función de seguimiento de misiones que permitirá a los jugadores establecer misiones activas y ver marcadores en el mapa que indiquen la ubicación de los objetivos.

RF-MIS-3.2: Los jugadores podrán cambiar entre misiones activas y ver un resumen de los objetivos en curso en cualquier momento.

RF-MIS-4.1: El juego incluirá misiones con múltiples caminos y soluciones basadas en las decisiones y acciones del jugador.

RF-MIS-4.2: Las decisiones del jugador afectarán el resultado y las recompensas de la misión, así como las relaciones con facciones y personajes.

RF-MIS-5.1: Debe haber misiones opcionales que los jugadores pueden elegir aceptar o ignorar según su preferencia.

RF-MIS-5.2: Las misiones opcionales enriquecerán la experiencia del juego y proporcionarán recompensas adicionales.

RF-MIS-6.1: Las misiones y objetivos en el juego deben ser coherentes con la historia y el mundo post-apocalíptico de X.

RF-MIS-6.2: Las acciones de los NPCs y las consecuencias de las misiones deben reflejar la lógica y la coherencia narrativa.

RF-MIS-7.1: El juego proporcionará retroalimentación al jugador a medida que progresen en las misiones, lo que incluye actualizaciones de objetivos, recompensas y consecuencias de decisiones.

RF-MIS-7.2: La retroalimentación ayudará a los jugadores a comprender el impacto de sus acciones en la historia y el mundo del juego.

1.10 Interfaz de usuario

RF-IU-1.1: El juego presentará una interfaz de usuario principal que incluirá elementos como el marcador de salud, el indicador de munición, el indicador de hambre y sed, y una brújula que muestra la dirección del objetivo de la misión activa.

***RF-IU-1.2:** Los elementos de la interfaz de usuario principal serán fácilmente legibles y no obstruirán la vista del juego.

RF-IU-2.1: El juego proporcionará un menú de inventario que permitirá a los jugadores gestionar su equipo, objetos y estadísticas del personaje.

RF-IU-2.2: El menú de inventario debe ser intuitivo y permitir la visualización y la gestión eficiente de objetos.

RF-IU-3.1: Los menús de diálogo permitirán a los jugadores seleccionar respuestas y opciones durante las conversaciones con NPCs.

RF-IU-3.2: Los menús de diálogo deben mostrar claramente las opciones de respuesta y proporcionar una representación coherente de las respuestas del personaje.

RF-IU-4.1: El juego mostrará marcadores en el mapa y la brújula para indicar la ubicación de misiones activas y objetivos.

RF-IU-4.2: Los marcadores serán precisos y se actualizarán en tiempo real a medida que el jugador se mueva por el mundo del juego.

RF-IU-5.1: Debe existir una opción en el menú de configuración que permita a los jugadores personalizar los controles del juego, incluyendo asignación de teclas y sensibilidad del mouse o joystick.

RF-IU-5.2: Los cambios en la configuración de controles deben aplicarse de manera coherente en todo el juego.

RF-IU-6.1: La interfaz de usuario proporcionará retroalimentación visual y de audio que informe al jugador sobre su estado de salud, necesidades básicas y eventos importantes en el juego.

RF-IU-6.2: La retroalimentación visual y de audio será clara y no obstruirá la experiencia de juego.

RF-IU-7.1: El juego ofrecerá tutoriales y consejos en la interfaz de usuario para ayudar a los jugadores a comprender las mecánicas del juego, como el combate, el uso del inventario y las conversaciones.

RF-IU-7.2: Los tutoriales podrán ser habilitados o deshabilitados según la preferencia del jugador.

1.11 Gráficos y sonido*

RF-GRA-1.1: El juego presentará gráficos de alta calidad que reflejen un entorno post-apocalíptico detallado y realista.

RF-GRA-1.2: Los gráficos deben ser nítidos y proporcionar un alto nivel de inmersión en el mundo del juego.

RF-GRA-2.1: El juego será compatible con una variedad de resoluciones de pantalla para adaptarse a las configuraciones de hardware de los jugadores.

RF-GRA-2.2: Debe existir una opción de calidad gráfica que permita a los jugadores ajustar la configuración para un rendimiento óptimo en sus sistemas.

RF-GRA-3.1: El juego implementará efectos de iluminación realistas, incluyendo ciclos de día y noche, sombras dinámicas y efectos atmosféricos.

RF-GRA-3.2: La iluminación y las sombras deben contribuir a la atmósfera y el realismo del entorno.

RF-GRA-4.1: Los personajes y NPCs en el juego deberán estar bien diseñados y presentar detalles faciales y animaciones realistas.

RF-GRA-4.2: La variedad en el diseño de personajes y NPCs contribuirá a la diversidad del mundo del juego.

RF-GRA-5.1: El juego incluirá efectos visuales especiales, como partículas, explosiones, efectos de fuego y efectos climáticos, que mejorarán la experiencia visual.

RF-GRA-5.2: Los efectos visuales serán coherentes con la ambientación post-apocalíptica del juego.

RF-GRA-6.1: El juego ofrecerá sonido ambiente realista que refleje el entorno post-apocalíptico, incluyendo viento, fauna, y sonidos de la vida cotidiana en el yermo.

RF-GRA-6.2: El sonido ambiente contribuirá a la inmersión en el mundo del juego.

RF-GRA-7.1: La banda sonora del juego contará con una variedad de música que se ajustará a la atmósfera y el tono de las diferentes áreas y situaciones.

RF-GRA-7.2: Los efectos de sonido serán realistas y reflejarán las acciones de los personajes y los eventos en el juego.

***RF-GRA-8.1:** El juego incluirá actuación de voz profesional para los personajes principales y NPCs, con diálogos completamente doblados en el idioma del juego.

RF-GRA-8.2: Las actuaciones de voz serán de alta calidad y coincidirán con la personalidad y las emociones de los personajes.



Universidad de Oviedo

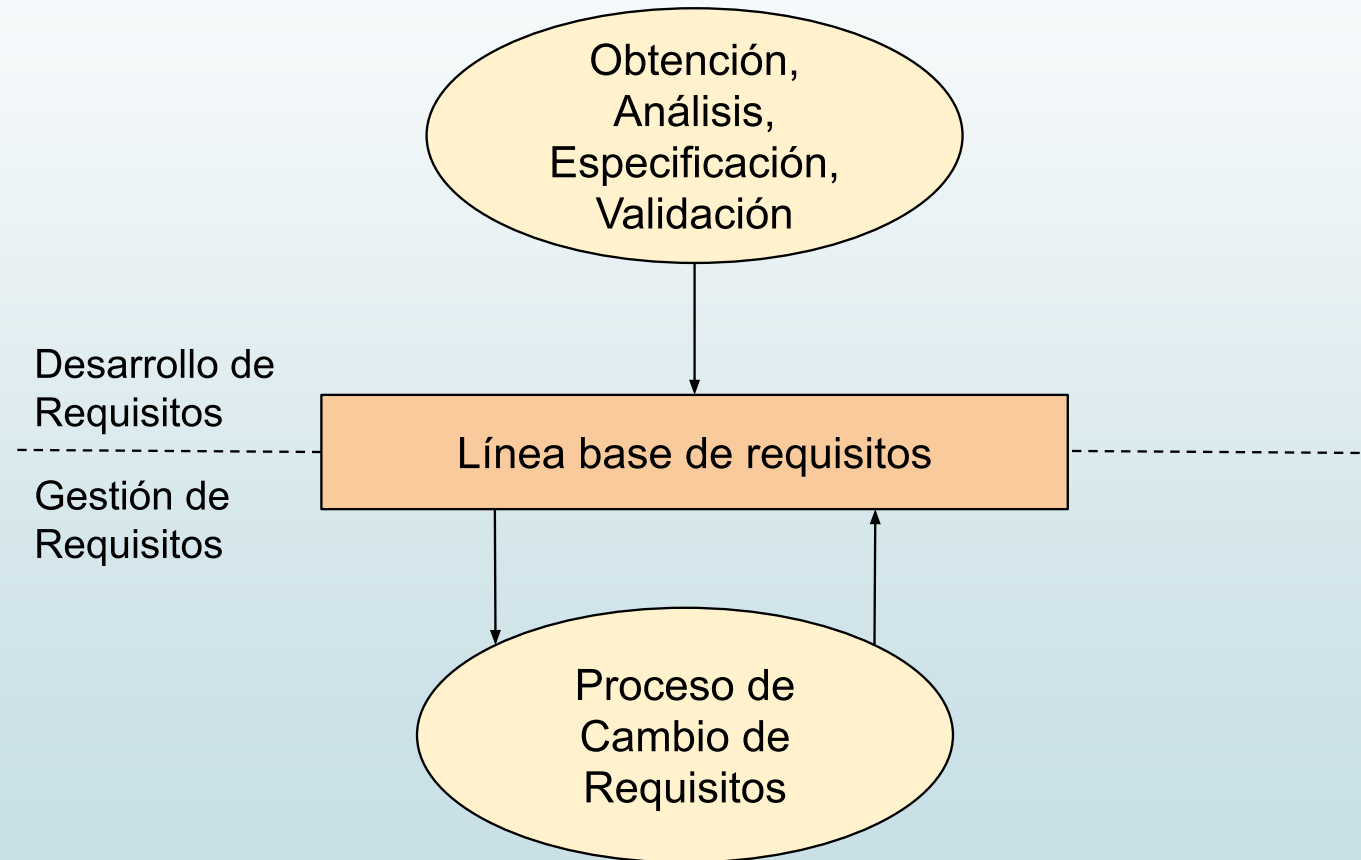
Tema 7

Gestión de Requisitos

1

Jorge Álvarez Fidalgo

Desarrollo y gestión de requisitos



Gestión

Consiste en anticipar y **gestionar** los cambios durante el ciclo de vida del proyecto para minimizar su impacto negativo.

- Definir la **línea base** de requisitos.
- **Evaluar cambios** propuestos.
- **Incorporar cambios aprobados** de forma controlada.
- **Trazar requisitos** individuales con los diseños, código y prueba derivados.
- Realizar **seguimiento del estado** de los requisitos y el historial de cambios a lo largo del proyecto.

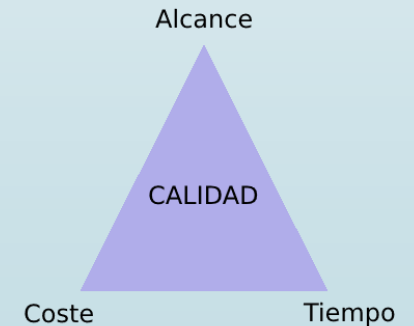
Línea base de requisitos

- Conjunto de **requisitos acordados** y revisados en un momento dado, **de cara a un *release*** o una iteración del **desarrollo**.
 - Según lo descrito en el Tema 2.
- Una vez un conjunto de requisitos se somete a una línea base, entran en la jurisdicción de la **gestión**.
 - Los potenciales cambios deben ser negociados y han de someterse a unos procedimientos determinados.

Línea base de requisitos

- Los **cambios** o añadidos a la línea base **pueden afectar a** la **planificación** y compromisos de **calidad** del equipo de desarrollo.
- En estos casos, hay varios planteamientos posibles:
 - **Aplazar** requisitos de baja prioridad a futuras iteraciones.
 - Ampliando el **personal** o externalizando el trabajo.
 - **Retrasar** la entrega.
 - Sacrificar la **calidad**.
- Salvo el primero, el resto refleja el **Triángulo de Hierro**.

El Bonja
haciendo de
Tensión de
con las manos



Gestión

El **proceso de gestión** de requisitos consta de 4 **categorías** principales:

- **Control de versiones.**
- **Control de cambios.**
- **Seguimiento del estado** de los requisitos.
- **Trazado** de los requisitos

Control de versiones



Control de versiones

El control de versiones aplica a:

- Requisitos **individuales**.
- **Conjuntos** de requisitos (**SRS**).

Implica lo siguiente:

- **Identificación única** de cada versión.
- Acceso global, por defecto, a la **versión actual**.
- **Historial** de versiones.
 - Cambios realizados, incluyendo el autor, fecha y justificaciones.
 - Debería permitir el revertir a una versión previa.

Control de versiones

Hay varias **opciones** para realizar el control de versiones:

- Emplear **herramientas de gestión de requisitos.**
- Almacenar los documentos en **herramientas de control de versiones.**
 - Git, CVS...
- Alternativa simple: **etiquetar** distintamente **cada** nueva **versión** del documento.
 - Seguramente, no muy práctico, salvo en especificaciones sencillas.
 - Word dispone de un modo revisión que permite visualizar más claramente los cambios realizados.

Seguimiento del estado de los requisitos

10



Atributos de requisitos

Algunos **atributos** útiles para los requisitos son los siguientes:

- Estado.
- Prioridad.
- Autor y fecha de creación.
- Versión actual del requisito.
- Fuente.
- Razonamiento.
- Release o iteración asignada.

Demasiados atributos pueden ser contraproducentes; seleccionar unos pocos atributos clave y añadir más si se considera que aportan valor.

Seguimiento del estado de los requisitos

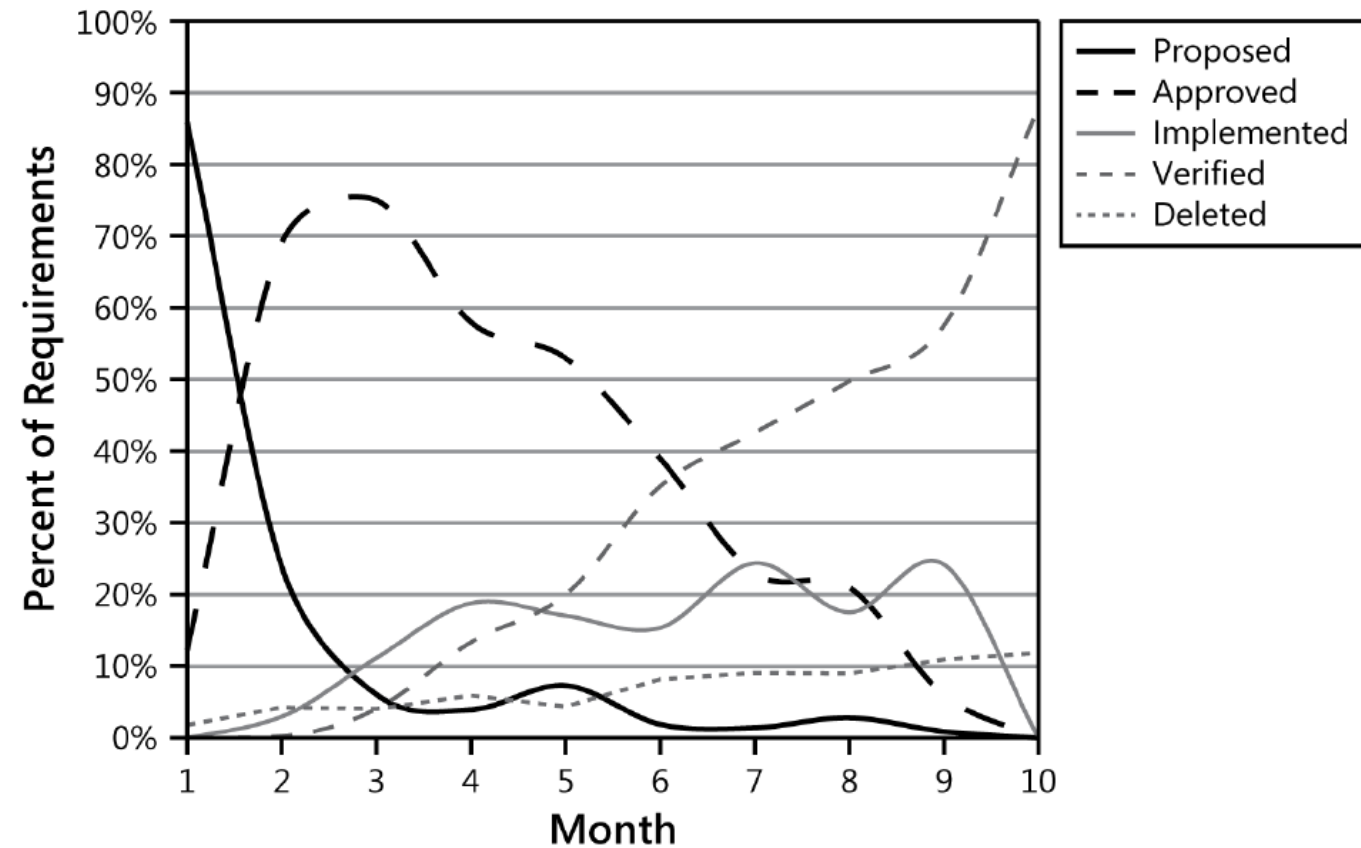
- El **estado** era uno de los atributos recomendados.
- Realizar el seguimiento del estado de cada requisito individual **permite medir** con cierta precisión el **progreso** del proyecto.
- **Evita estimaciones** excesivamente **optimistas** => **sobrecostes y retrasos.**

Seguimiento del estado de los requisitos

Estados comunes:

- **Propuesto.** Por una fuente autorizada.
- **En progreso.** Requisito en construcción.
- **Redactado.** La versión inicial del requisito se ha completado.
- **Aprobado.** Acordado por stakeholders y asignado a una línea base.
- **Implementado.** El código que materializa el requisito ha sido implementado.
- **Verificado.** El requisito satisface los criterios de aceptación (completo).
- **Aplazado.** Requisito aprobado que se retrasa a una futura iteración.
- **Eliminado.** Requisito aprobado y descartado. Incluir motivación y responsable.
- **Rechazado.** Requisito no aprobado y descartado.

Seguimiento del estado de los requisitos



¿Cuándo finalizará una iteración del desarrollo?

Gestión de problemas

- También es recomendable **registrar** y **clasificar** los **problemas** (*issues*) que surjan con los requisitos.
- **Etiquetas** comunes son:
 - Requisito **poco claro**.
 - Requisito **omitido**.
 - Requisito **incorrecto**.
 - Requisito **duplicado**.
 - Requisito **innecesario**.
- Tales problemas deberían almacenarse de manera organizada y actualizarse según se resuelvan (*open/closed issues*).

Control de cambios

16



Control de cambios

El control de cambios **permite a** la organización:

- **Evaluar** los cambios propuestos antes de realizarlos.
- **Hacer** los cambios **visibles** para todos los **stakeholders**.
- **Notificar** a todos los **participantes** los cambios aprobados.
- Realizar cambios de un **modo consistente y eficiente**.
 - Evitar cambios subversivos.

Corrupción del alcance

- Que tengan lugar cambios en los requisitos de la línea base es **inevitable**.
 - La **introducción de nueva funcionalidad en la línea base** o modificaciones notables en la misma **se denomina "crecimiento de requisitos"**.
 - El **negocio no es estático**: las **necesidades** de los clientes **cambian**, el contexto de negocio **evoluciona**.
- Típicamente, los **requisitos crecen** entre un 1% y un 4% por mes.
- Denominamos corrupción del alcance, o síndrome del lavadero, al **crecimiento descontrolado** de requisitos.
 - El proyecto **incorpora** constantemente **nueva funcionalidad sin considerar las consecuencias**.

Corrupción del alcance

De cara a **evitar la corrupción del alcance**:

- Acordar y **documentar** los objetivos de negocio, **alcance** y limitaciones del sistema.
 - Contrastar cada cambio propuesto contra todo ello.
- Tratar con todos los *stakeholders* de relevancia en la **obtención** reduce los requisitos omitidos o malinterpretados.
 - El prototipado también ayuda en el mismo sentido.

Corrupción del alcance

- Gerald Marvin Weinberg, en un artículo de American Programmer en 1995, afirma que la técnica más efectiva para evitar la corrupción del alcance es decir

No.

- Los equipos de desarrollo suelen recibir gran **presión** para satisfacer cualquier cambio.
 - La filosofía de "El cliente siempre tiene la razón" es, cuanto menos, peligrosa.
- Alternativa diplomática: "ahora no".

Política de control de cambios

Se debe definir una política que **defina** la gestión de los cambios propuestos:

- **Todos** los **cambios** deben **ceñirse al proceso** de control de cambios.
- **No** se realizará diseño o **implementación** alguna sobre **cambios no aprobados** (salvo exploratorios).
- Proponer un cambio **no** garantiza su aprobación.
- Los **cambios** deben ser **accesibles** para todos los **stakeholders**.
- Se debe **analizar el impacto** de **cada cambio** propuesto.
- Se debe **registrar el motivo** de aprobación/rechazo de un cambio.

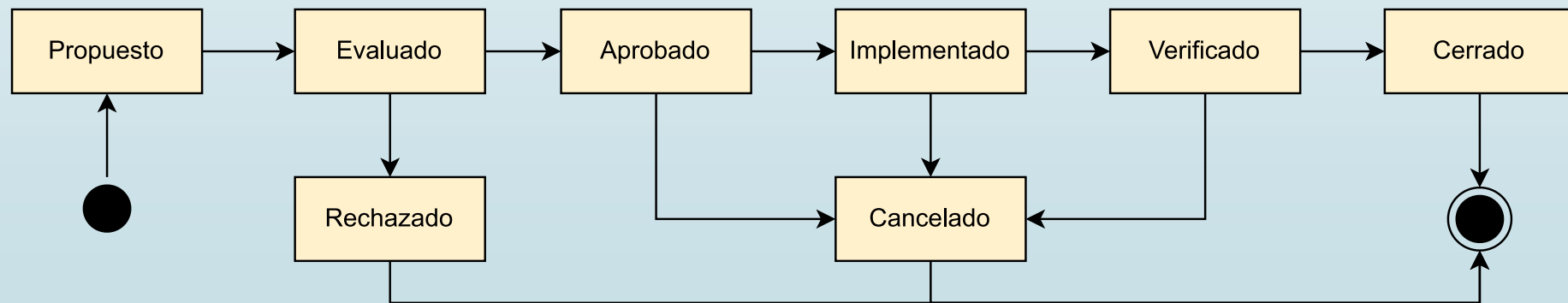
Proceso de control de cambios

- Procedimiento para **analizar** y **evaluar** los cambios propuestos a una línea base.
- El proceso debería ser claro (bien documentado), simple y eficiente.
 - Si no cumple estas condiciones, será ignorado.

Proceso de control de cambios

Las **tareas** a realizar en el proceso son las siguientes:

1. **Evaluar** la solicitud de cambio.
2. Realizar una **decisión** de cambio.
3. **Implementar** el cambio.
4. **Verificar** el cambio. (*Peer review*)



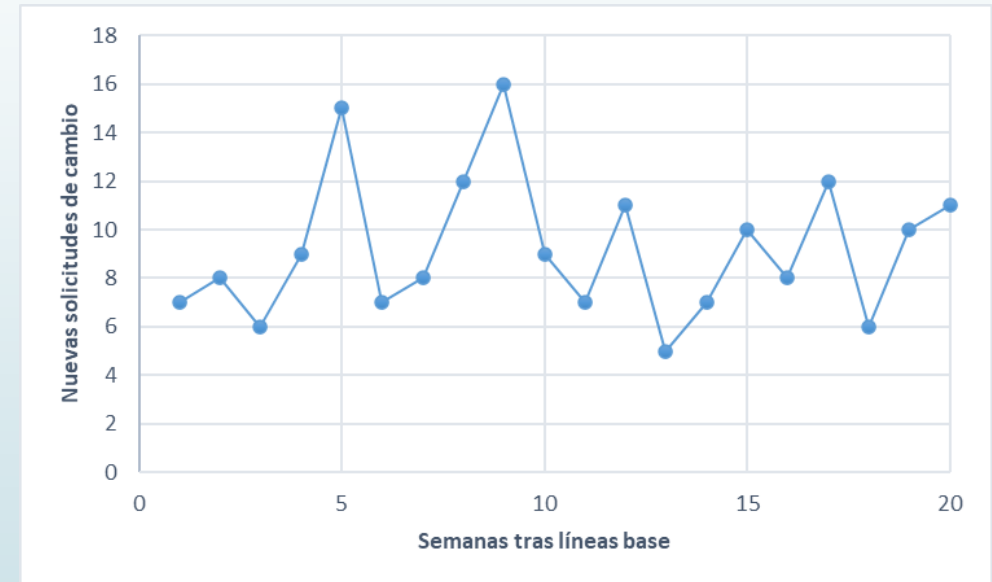
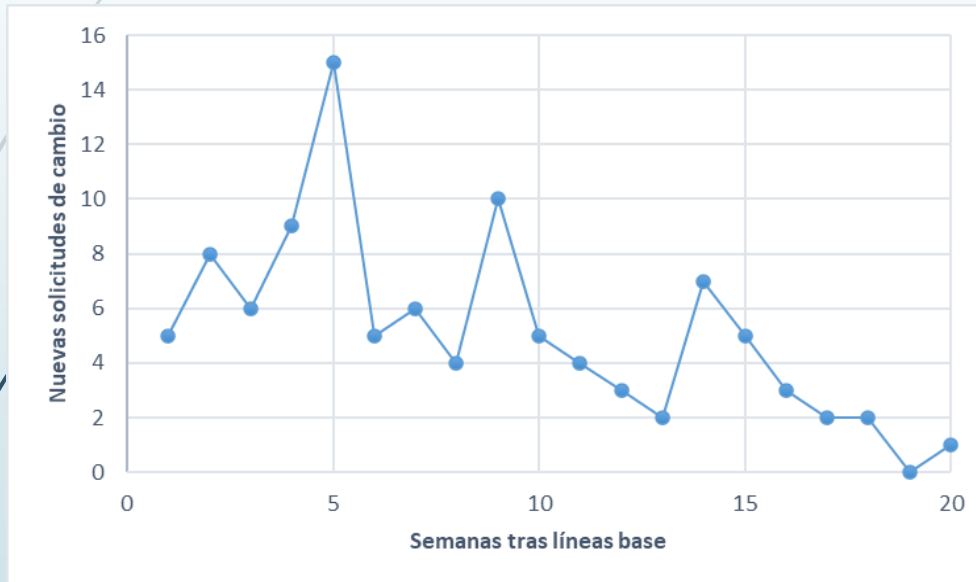
Comité de control de cambios (CCC)

- Toma las **decisiones** relativas a los cambios propuestos.
 - Suele existir de facto en cualquier proyecto.
 - Oficializar su existencia normaliza su composición y procedimientos.
 - Sus responsabilidades dependen de la escala de la organización y el proyecto.
- La composición varia, pero **suele incluir**:
 - **Dirección** del proyecto.
 - **Analista** de negocio.
 - **Desarrollo**.
 - **QA**.
 - **Marketing** o representantes del cliente.

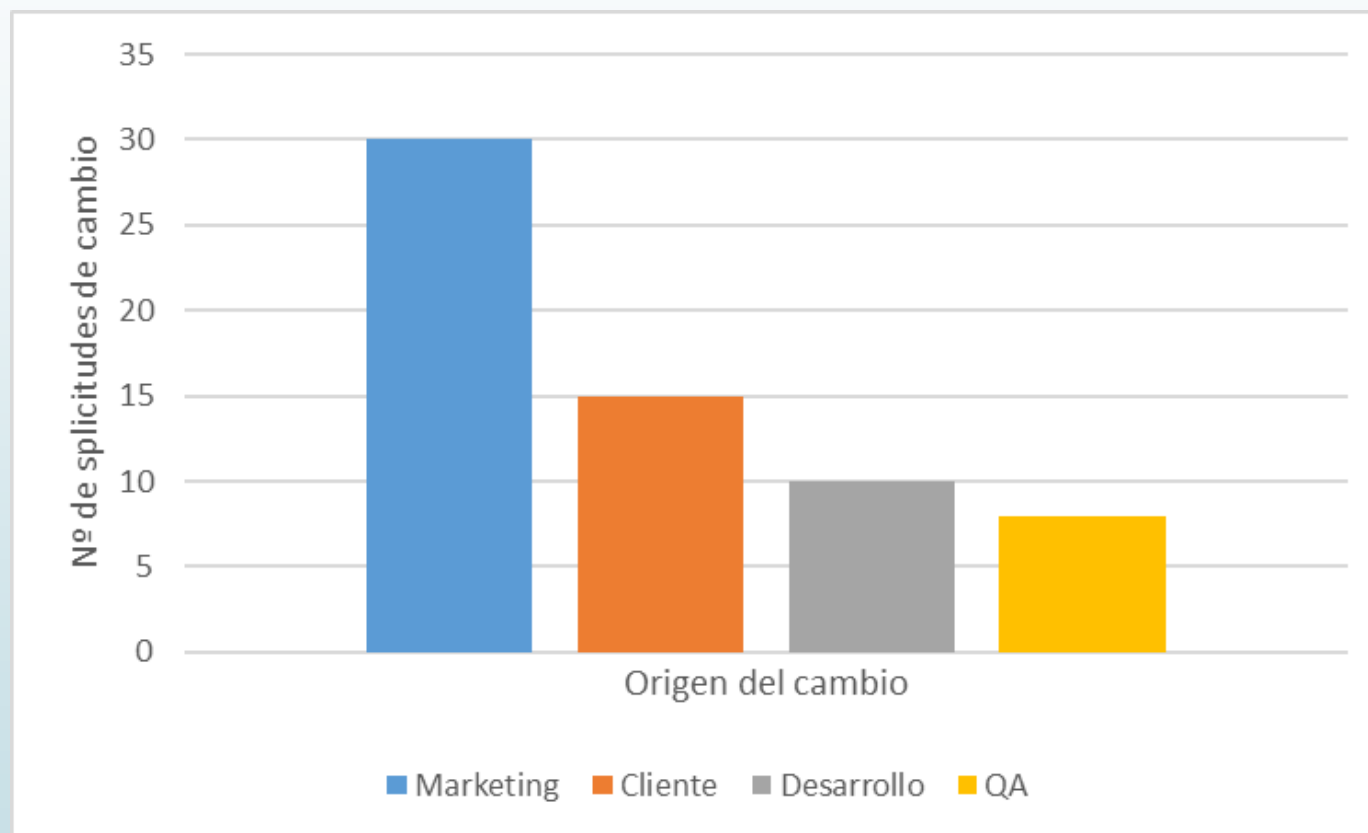
Métrica de cambios

- Medir la **actividad** de los cambios en un proyecto:
 - Es una **métrica para la estabilidad** de los requisitos.
 - Aporta **información para futuras mejoras** en el proceso.
- Los siguientes aspectos son de particular interés:
 - Total de solicitudes de cambios; abiertas y cerradas.
 - N° acumulativo de requisitos añadidos, modificados o eliminados.
 - N° de solicitudes realizadas por cada origen (stakeholders).
 - N° de cambios realizados sobre cada requisito de línea base.
 - Esfuerzo dedicado al control de cambios.

Métrica de cambios



Métrica de cambios



Estabilidad de cambios

Según su estabilidad, los requisitos se pueden clasificar en:

- **Duraderos**. Requisitos **estables** que derivan de la actividad principal de la organización y que están directamente relacionados con el dominio.
- **Volátiles**. Requisitos con **probabilidad de cambio** durante el desarrollo o el ciclo de vida del sistema.

Los **volátiles**, a su vez, pueden clasificarse en [Harker et al., 1993]:

- **Mutables**. Varían como **consecuencia de cambios** en el entorno operativo.
- **Emergentes**. Surgen al **incrementarse la comprensión** del cliente del sistema durante el desarrollo de este.
- **Consecuentes**. Resultan de los **cambios** en los procesos que tienen lugar al introducir el sistema en la organización.
- **Compatibles**. Requisitos que **dependen de otros sistemas**, y que deben adaptarse a cambios en aquellos.

X-MEN

Mutantes
Emergentes

Consecuencias
Compatibilidad

MECC

Impacto de cambios

- ▶ Durante la evaluación de los cambios propuestos, se ha de llevar a cabo un **análisis del impacto** de los mismos.
 - ▶ Otorga una comprensión de las implicaciones del cambio...
 - ▶ ... y por tanto nos permite tomar una decisión razonada acerca de su aprobación.
- ▶ Incluso los cambios en apariencia más inocuos pueden resultar inesperadamente complejos.
 - ▶ *Cambiar un mensaje de error. ¿Cuán difícil puede ser?*

Impacto de cambios

El **procedimiento** general es el siguiente:

1. Estudiar las implicaciones del cambio.
2. Identificar los productos del trabajo afectados por el cambio.
3. Estimar el esfuerzo necesario para el cambio.
4. Identificar las tareas a realizar y su coordinación con las preexistentes.
5. Estimar el impacto en la planificación y coste del proyecto.
6. Asignar prioridad al cambio.
7. Informar al CCC.

Implicaciones?
Prod. Afectados?
Esfuerzo?
Tareas?
Impacto?
Prioridad?
Informar

Wieggers y Beatty proponen unas checklists y plantillas como referencia para llevar a cabo este procedimiento en *Software Requirements 3ed*. A continuación, se muestra un atisbo de todo ello.

Impacto de cambios

Algunas cuestiones a considerar para evaluar las **implicaciones** de un cambio:

- ¿Alguno de los requisitos de la línea base entra en conflicto con el cambio?
- ¿Cuáles son las consecuencias de no realizar el cambio?
- ¿Cuáles son los posibles riesgos de realizar el cambio?
- ¿Afectará el cambio al rendimiento u otros atributos de calidad?
- ¿Es el cambio factible dadas las restricciones técnicas y el equipo actual?
- ¿Exige el cambio recursos necesarios para el desarrollo y pruebas?
- ¿Cómo afectará el cambio a la planificación temporal?
- ¿Es necesario realizar prototipado para validar el cambio?
- ¿Cuánto del esfuerzo realizado se perderá si se acepta el cambio?

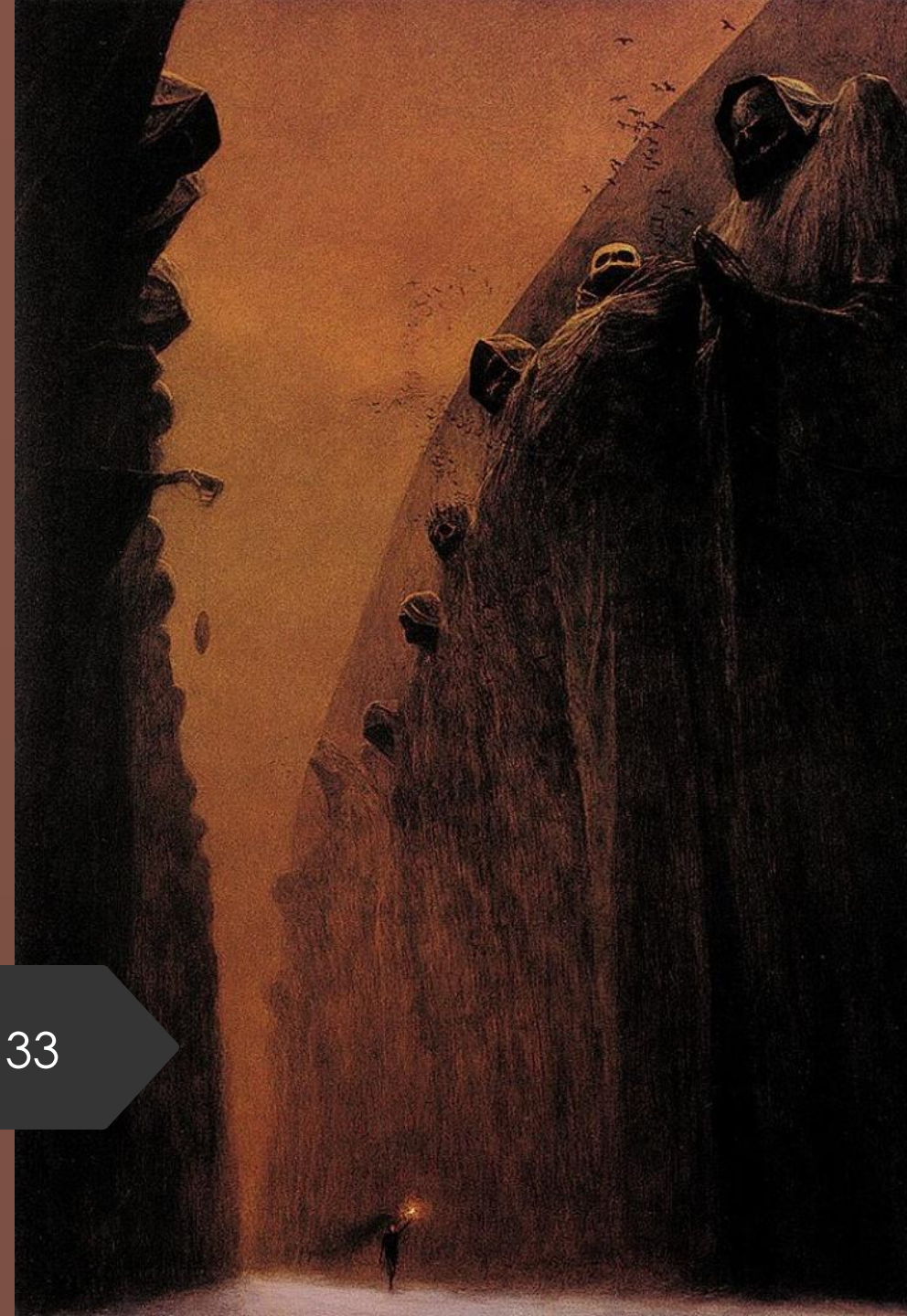
Impacto de cambios

Principales **productos** del trabajo **a examinar**:

- **Componentes** de diseño.
- Archivos del **código fuente**.
- **Bases de datos**.
- **Interfaces** de usuario.
- **Pruebas** unitarias, de integración, de sistema...
- Procedimientos, **archivos**, informes...
- **Documentación** y otros materiales de apoyo.
- Aplicaciones externas, componentes hardware, **librerías**...
- **Planes de la gestión** de proyectos.

Trazado de requisitos

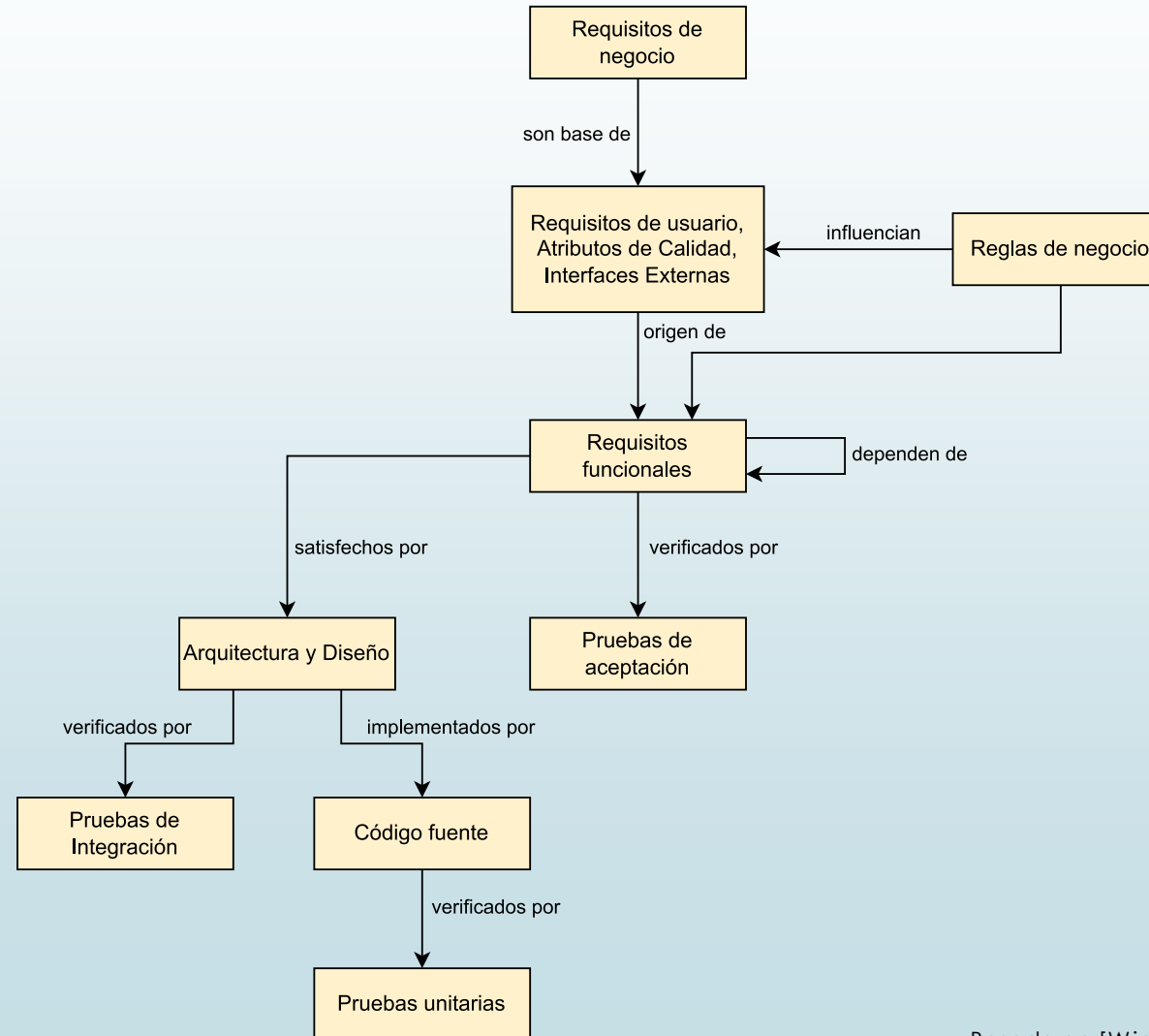
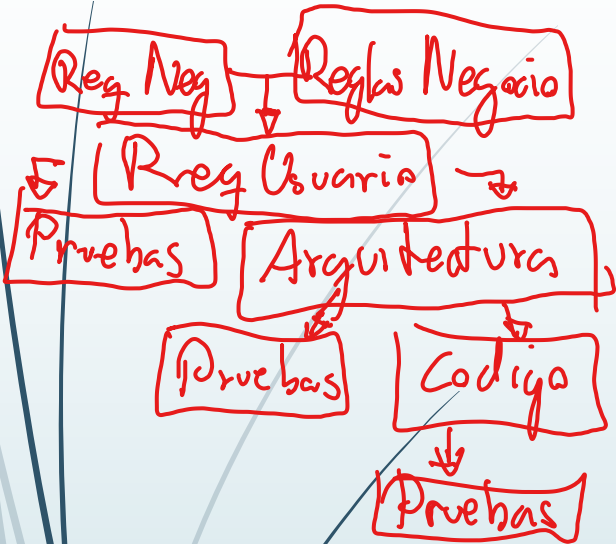
33



Trazado de requisitos

- *Trazabilidad* != trazado.
- Trazar requisitos con otros elementos del proyecto **permite:**
 - **Detectar** requisitos omitidos o innecesarios.
 - Realizar un **seguimiento del progreso** de la implementación.
 - Evaluar más fielmente el **análisis del impacto**.
 - **Asegurar** el **cumplimiento** de las **restricciones** impuestas.
 - **Ejecución** más **eficiente** del mantenimiento y las pruebas.

Trazado de requisitos





Matriz de trazabilidad

- Se emplean las denominadas **matrices de trazabilidad** para **trazar las conexiones entre requisitos y otros elementos del proyecto**.
- El formato y contenido de las mismas varía. Por ejemplo:

Requisito de usuario	Requisito Funcional	Componente de diseño	Código fuente	Pruebas unitarias
CU-33	RF1.4	Clase Solicitud	CrearSolicitud()	Solicitud.11 Solicitud.12
CU-33	RF8.8	Clase Solicitud	GuardarSolicitud() ()	Solicitud.42 Solicitud.43

- El contenido se rellena según avanza el desarrollo, no de antemano.*

Trazado de requisitos

- Conforme la **complejidad** de un sistema **aumenta**, realizar el trazado de los requisitos con el resto de elementos del proyecto **se torna muy costoso**.
- Los responsables del proyecto deberán **juzgar si es apropiado** invertir este coste, teniendo en cuenta los **riesgos** de no hacerlo.
 - Probablemente sea recomendable hacerlo, al menos a un nivel básico (requisitos de negocio – requisitos de usuario – requisitos funcionales).
- En determinados **casos críticos**, es (prácticamente) **obligatorio** realizar el **trazado**.
 - Ver [Software Approval Guidelines de la FAA](#).
 - Ver [Quality System Regulation de la FDA](#).



Tema 8

La Ingeniería de Requisitos en los Métodos Ágiles

1

Jorge Álvarez Fidalgo

"Un ejército victorioso gana primero y entabla la batalla después; un ejército derrotado lucha primero e intenta obtener la victoria después." - Sun Tzu.

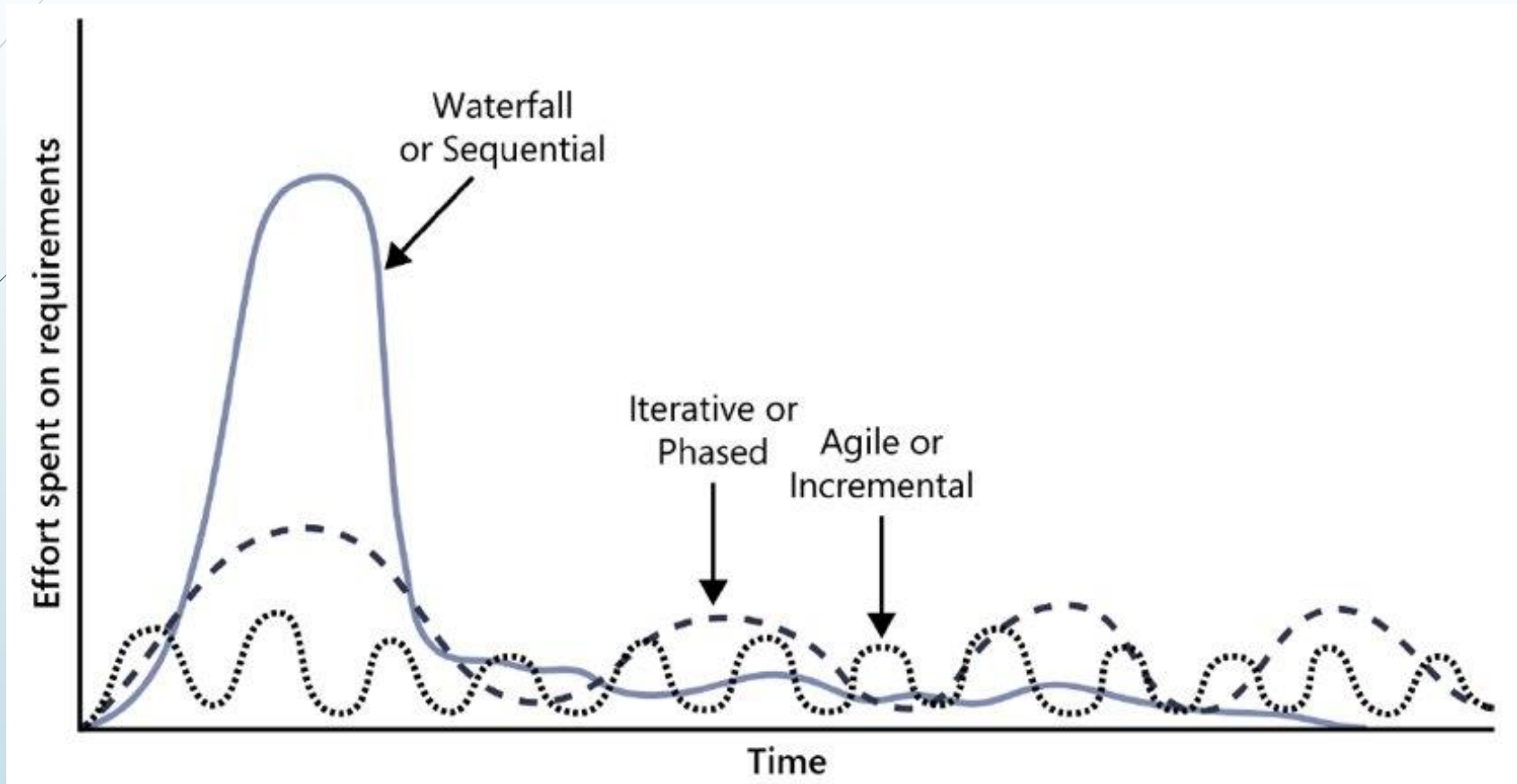
Motivaciones

- Las metodologías secuenciales tradicionales presentan una serie de problemas.
- La definición de una especificación de requisitos completa al inicio del proyecto, sobre la cual realizar las subsiguientes fases, genera una fuerte **dependencia** sobre la misma y lo hace propenso a **errores** costosos.
 - La rigidez de la metodología secuencial hace, en general, difícil adaptarse a los **cambios**.
- *Interesantemente, el mismo autor de la primera propuesta de modelo de cascada (W. Royce) señala que es arriesgado e invita a los fallos.*

Métodos ágiles

- Metodologías de desarrollo que favorecen la **cooperación** con los stakeholders y la entrega frecuente de **pequeños incrementos de funcionalidad útil**.
 - Scrum, XP, Lean... (nos centraremos en Scrum).
 - Procesos pensados para **adaptarse a los cambios**.
- El **desarrollo se divide en iteraciones**.
 - En cada una de ellas, se realiza un incremento de la funcionalidad.
 - El trato continuado con los *stakeholders* facilita detectar y corregir problemas.
- El objetivo es **proveer valor** (software funcional) **al final de cada iteración**.

Métodos ágiles



5

IR en un desarrollo ágil

Adaptación de lo ya conocido

Proceso de IR

- Las **fases** del proceso de IR siguen presentes:
 - Debemos obtener requisitos de los *stakeholders*, analizarlos, documentarlos y validar que se ajustan con los objetivos de negocio; así como gestionar los cambios que se puedan sugerir.
- Ahora bien, dada la naturaleza del desarrollo ágil:
 - **No** se elabora la especificación completa al principio.
 - Se emplean **historias de usuario** para construir el *Product Backlog*.
 - Los *stakeholders* están **involucrados** durante **todo** el **proceso**, no meramente al principio y al final.

Historias de usuario

- Son **descripciones simples** y cortas de una función contadas desde la **perspectiva** de la persona que la desea; normalmente, un usuario o cliente [Cohn 2010].
- Al igual que los casos de uso, pretenden describir las interacciones con el sistema que necesitan llevar a cabo distintos tipos de usuarios.
- No obstante, varían en su desarrollo:
 - Caso de uso > Especificación CU > Requisitos Funcionales/Pruebas.
 - Historia de usuario > Historias de usuario refinadas > Pruebas de aceptación.

Historias de usuario

- Las historias de usuario **deben ser** implementables en **una iteración**.
 - Si requieren **más de una iteración**, se denominan **épicas (*epics*)**.
 - Las historias de nivel más bajo que satisfacen los requisitos de negocio se denominan *características mínimas comercializables*; esto es, la mínima funcionalidad que aporta valor al cliente.

Historias de usuario

- Las historias de usuario son simples y directas...
 - ... pero eso conlleva un precio.
- Las historias de usuario **podrían no ser apropiadas para proyectos complejos o críticos.** [Glib & Glib 2011]
 - La especificación de un caso de uso nos permite examinarlo con gran detalle y derivar los requisitos y pruebas pertinentes.
 - Las historias de usuario requieren un equipo experimentado para evitar omisiones, dado su carácter menos riguroso.

¿Requisitos?

- Si las historias de usuario son el corazón del desarrollo ágil, ¿cómo encajan los **requisitos** en todo esto?
- Como suele ser habitual, adolecemos de una respuesta absoluta.
 - La facción purista adopta una postura maniqueísta: los requisitos –exclusivos a las metodologías tradicionales– son entidades distintas a las historias de usuario.
 - Una visión más pragmática es la siguiente: las historias de usuario son simplemente una nueva encarnación de los requisitos. Expresados desde la perspectiva del usuario y de alto nivel; pero requisitos, a fin de cuentas.
 - ¿Requisitos de usuario?

¿Requisitos?

- Esta es una cuestión principalmente **ontológica**.
 - Por tanto, interesante pero no muy práctica para nuestros fines.
- Empleemos tal metodología o nomenclatura, nuestra labor sigue siendo la misma:
- **Proporcionar descripciones de calidad de las funciones y características del producto que satisfagan las necesidades de los *stakeholders*.**

De ahora en adelante, me referiré indistintamente a requisitos e historias de usuario, por sencillez.

Obtención

*Product Owner
se comunica con cliente constantemente*

- El desarrollo ágil favorece una **comunicación constante** con el cliente en detrimento de una documentación más detallada.
- En Scrum, se emplea un rol denominado **Product Owner**.
 - Es una **figura híbrida entre un representante de cliente** (objetivos de negocio) y **analista de negocio/ingeniero de requisitos** (desarrolla y prioriza el *Product Backlog*).

Análisis

- Se intentan
usar modelos visuales
- Se incrementa la calidad
de las atribuciones
del software

- El uso de modelos visuales en las metodologías ágiles es igualmente **útil**.
 - Las historias de usuario no serán siempre suficientes.
- La diferencia respecto a las metodologías tradicionales reside en:
 - Elaborar únicamente los modelos **necesarios**.
 - Elaborar los modelos correspondientes a la iteración **actual**.
 - Emplear el nivel de detalle **suficiente** para transmitir la información deseada.
 - Mayor tendencia a la **informalidad**.

Análisis

- Los **atributos de calidad** poseen la importancia habitual.
 - Los más relevantes habrán de identificarse y priorizarse en una etapa temprana.
- No obstante, su **implementación** es más **sutil**.
 - Pueden **implicar múltiples historias** a lo largo de varias iteraciones.
 - Se han de tener **presentes** a lo largo del desarrollo.
- Similarmente a lo expuesto en su momento:
 - Se pueden derivar **nuevas** historias de usuario de los atributos de calidad.
 - Se definen **pruebas** (de aceptación) para cuantificarlos.

Especificación

- Las historias de usuario se almacenan en el denominado **Product Backlog** (cartera de producto), que evoluciona a lo largo del proyecto.
- En cada iteración, se **selecciona un conjunto** de historias de usuario.
 - Es entonces cuando se lleva a cabo un **refinamiento** de las mismas para extraer los detalles relevantes.
- Comúnmente, esto se traduce en la definición de **pruebas de aceptación**.
 - A ejecutar en la **misma** iteración y las siguientes (pruebas de regresión).
 - También podrían elaborarse los requisitos funcionales que conocemos, pero es menos común.

Especificación

- Se suelen emplear **tarjetas** para registrar las **historias de usuario**.
 - Emplear documentos tradicionales es menos habitual.
 - El principio "**Software funcional antes que documentación**" puede llevar a la idea de desechar la documentación por completo.
 - Por una vez, existe cierto consenso de que esto es mala idea.
- Los **requisitos no funcionales** también se pueden registrar del mismo modo.
 - Seguidos de las correspondientes pruebas de aceptación.
 - Pueden formularse como **historias** de usuario:
 - *Como técnico del centro de asistencia, quiero que el grafo de conocimiento responda a mis consultas en 5 segundos o menos para que el cliente no se impaciente.*

Especificación

- Una vez más, el nivel de detalle y rigor para la especificación de "requisitos" dependerá de nuestras circunstancias particulares:
 - La frecuencia y calidad de las **comunicaciones** con los *stakeholders*.
 - El valor de los requisitos de cara al **futuro**.
 - Mejoras, mantenimiento, auditorías, certificaciones, cuestiones contractuales.
 - La capacidad de las pruebas de aceptación de **describir** las capacidades y comportamiento del sistema.
 - **Extensión** y **complejidad** del sistema.

Validación

- Gran énfasis en las **pruebas de aceptación** como método de validación.
- Por ejemplo, para la historia definida hace dos diapositivas:
 - *Una consulta sobre el grafo de conocimiento requiere menos de 5 segundos, preferiblemente menos de 3, para retornar un resultado.*

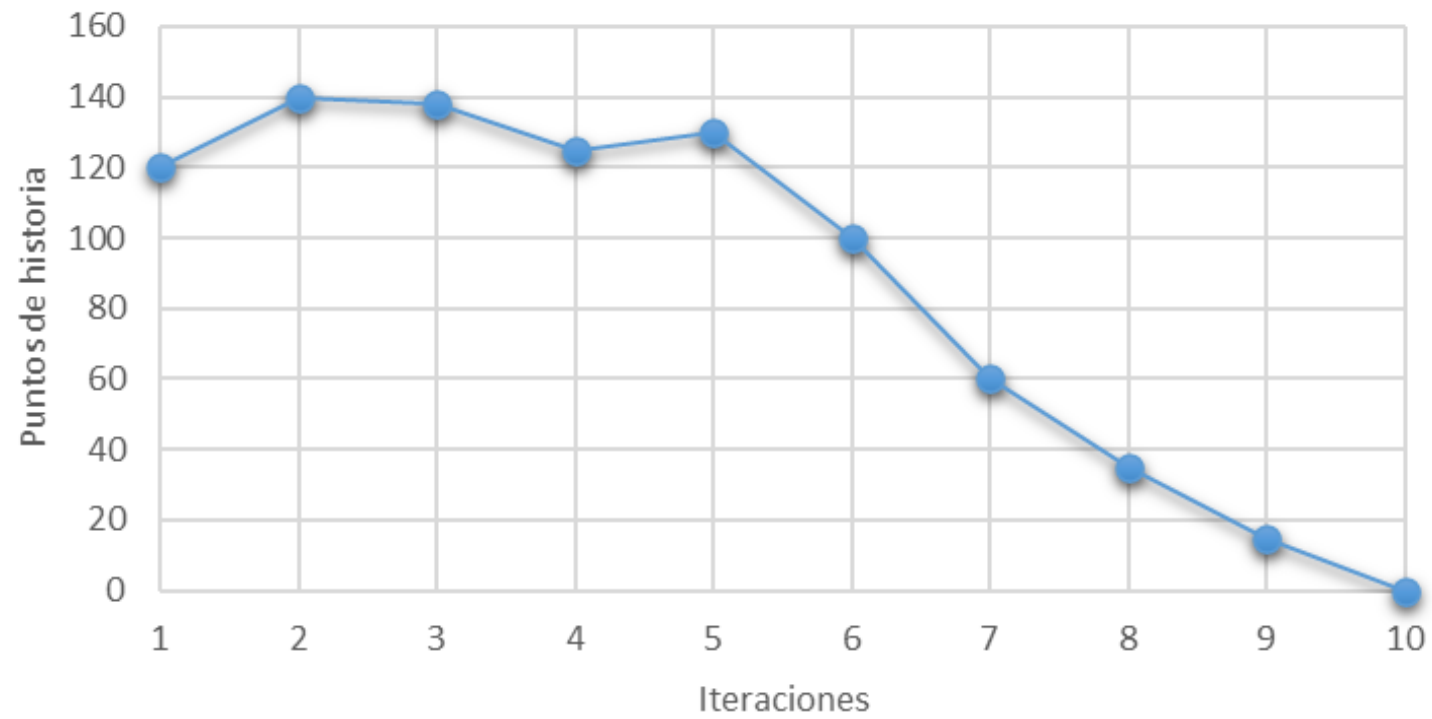
Línea base

- El *Product Owner* y el equipo acuerdan las historias a desarrollar en la siguiente iteración.
 - Se lleva a cabo en la **Planificación del Sprint**.
 - Se seleccionan del **Product Backlog según la prioridad**.
- Los **cambios** sobre los requisitos se consideran **para iteraciones futuras**.
 - Se introducen en el **backlog** y, según su prioridad, se determinará su introducción en las siguientes iteraciones.
 - Por tanto, la Gestión de requisitos es aquí bastante más limitada.

Gestión

- Se sigue realizando un **seguimiento** del estado (de las historias de usuario).
 - P.e. Leffingwell propone los siguientes estados: *en backlog, definida, en progreso, completado, aceptado, bloqueado*.
- Para medir el progreso del proyecto, se emplean diagramas de quemado (**burndown charts**).
 - Se estima el **esfuerzo** requerido para cada historia (*story points*).
 - Se indica, para cada iteración, los **puntos restantes** por completar en el backlog.

Diagrama de quema





Universidad de Oviedo

Epílogo

CHAOS

Jorge Álvarez Fidalgo

1

"Temedme mortales, ya que soy el Ungido, el Hijo Favorito del Caos, el Azote del Mundo. Los ejércitos de los Dioses van tras de mí, y por mi voluntad y mi espada, vuestras débiles naciones caerán." - Archaón el Elegido.

CHAOS

- El informe **CHAOS**, presentado por una institución independiente (The Standish Group) monitoriza proyectos de software y analiza sus resultados.
- Aquí se presentan algunos de los aspectos más interesantes que se pueden extrapolar del [informe de 2015](#).

Para los que no acudisteis presencialmente y leéis esto desde la calidez del hogar: a continuación, se presentan preguntas de interés cuya respuesta se encuentra en el informe. Podéis leer directamente el documento y extraeréis las mismas conclusiones; pero puede ser interesante que, antes, os planteéis dichas cuestiones por vuestra cuenta.

I – Proyecto exitoso

- ¿Qué define un proyecto exitoso?
- ¿Qué define un proyecto fracasado?
- ¿Existe un término medio?

I – Proyecto exitoso

- CHAOS propone las siguientes categorizaciones:
 - **Exitoso**. Cumple todas las métricas de calidad.
 - **Desafiado**. Proyecto finalizado, pero con incumplimientos de las métricas.
 - **Fallido**. Proyecto cancelado.
- Las métricas varían.
 - Antiguamente, empleaban **En presupuesto, A tiempo y Objetivos** [cumplidos].
 - En la última versión sustituyen Objetivos por **Satisfacción** [del cliente].
 - Este cambio reduce el % de éxito...

II – Tamaño del proyecto

- ¿Cómo afectará al éxito del proyecto?
- De aquí surge una pregunta ulterior: ¿cuándo *necesitamos* proyectos grandes?

III – Complejidad del proyecto

- ¿Qué diferencia hay con lo anterior?
- ¿Cómo afectará al éxito del proyecto?

IV – Metodología del proyecto

- ¿Se apreciarán diferencias entre proyectos de distintas metodologías de desarrollo? (Cascada vs ágiles)
- ¿Alguna característica del proyecto puede afectar al éxito de manera distinta según la metodología?

V – Experiencia del equipo

- Previsiblemente, afectará de algún modo al éxito del proyecto.
- ¿En qué magnitud?

VI – Requisitos como métrica

- ¿Cuántos proyectos emplearán los requisitos como métrica de éxito?

VII – Sectores

- ¿Habrá diferencia en el éxito de los proyectos según el sector del cliente?
- Si la hubiera, ¿cuál será el sector con más % de fracasos?

Factores de éxito

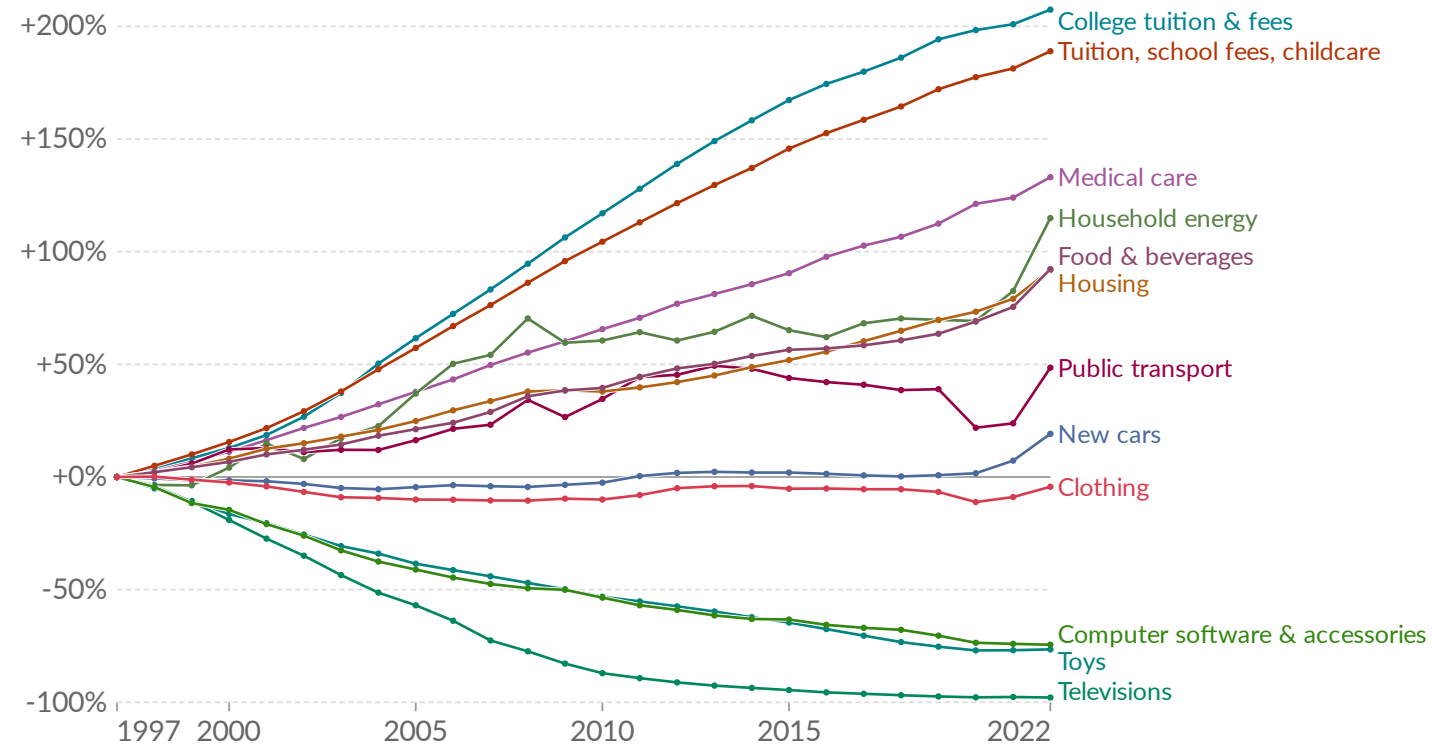
- ¿Cuáles serán los factores más importantes para el éxito de un proyecto?
- ¿Cuáles conciernen a la Ingeniería de Requisitos?

Impacto

Price changes in consumer goods and services in the United States

Our World
in Data

Price change in consumer goods and services in the United States, measured as the percentage change since 1997. Data is based on the consumer price index (CPI) for national average urban consumer prices.



Data source: United States Bureau of Labor Statistics (BLS)

OurWorldInData.org/technological-change | CC BY

Note: Some services – such as medical care – are not adjusted for quality. When adjusted for quality, some treatments have decreased in price rather than increased.